CSCI 566: Deep Learning and its Applications

Yue Zhao
Thomas Lord Department of Computer Science
University of Southern California

Credits to previous versions of USC CSCI566, CMU 10601/701, Stanford CS 229, 231n

USC Viterbi School of Engineering

Finding Your Project Team

Groups of 4 to 6, and 7 is fine. Mixing the two sections is encouraged. A DEN-only group needs one member at the November 30 poster session.

Sign up on the sheet linked on Piazza. Add your team once you have an idea, or use the Looking for Team tab.

No team by the end of week 3? Come to the September 14 class and we will group you. You still need to bring an idea.

Pre-proposal due Monday, September 14. Up to two pages, PDF, submitted on Gradescope as a group. Late groups have until Friday, September 18.

Cover the idea, why it is interesting, a solution sketch, expected outcomes, and a rough timeline.

Welcome to CSCI 566!

This class will teach you some exciting developments in Machine Learning, Computer Vision, NLP, Robotics, and other AI-related fields in the last decade!

New cool stuff: I will aim to bring someone to lectures from industry, academia, and more to discuss their life in ML and Data Science – this session will be interactive, and I hope it will be helpful for your career planning.

Piazza and Gradescope

- Course website: https://usc-fortis.github.io/CSCI566-F26/
- Piazza: https://piazza.com/class/msmpbcelj2p2fg
- Gradescope: https://www.gradescope.com/courses/1346598
- The Gradescope entry code is announced in class and on Piazza. Sections 30122 and 30359 share one Piazza and one Gradescope; if you are newly added to the course, add yourself to both.
- This lecture is pre-recorded. There is no live session on August 31. Post questions on Piazza.

TA Office Hours – Mine is After the Class

	Office hours	Where
Yue Zhao (instructor)	After each Monday class	SGM 124
Tiankai Yang	TBD	TBD
Jiate Li	TBD	TBD
Chen Chu	TBD	TBD



Section 30359 (DEN): every office hour has a remote option. Links are on the course website, not in this deck, so they stay current.

Midterm

- Monday, November 2, during the regular class block
- Worth 35 of the 100 course points
- Format: open book, no electronic devices. Mostly multiple choice, plus some computation questions with unambiguous single-number answers.
- Content: everything covered in class up to and including October 26. A narrowed scope is posted by October 19.
- Section 30359 (DEN): delivery and proctoring are arranged with DEN and posted on the course website before the exam. Both sections take the exam on the same day.

Discussions on The Depth of the Course

the instructors' answer, *where instructors collectively construct a single answer*

First of all, thanks for sharing your thoughts -- which are always acknowledged and appreciated.

Second, I also agree with most of the posts saying that since 567 is not a prerequisite this semester, it will be *risky* to assume all the students have the same level of knowledge in classical/non-deep ML. Thus, we will still have to take this into account.

Again, I think the solution can be multi-fold:

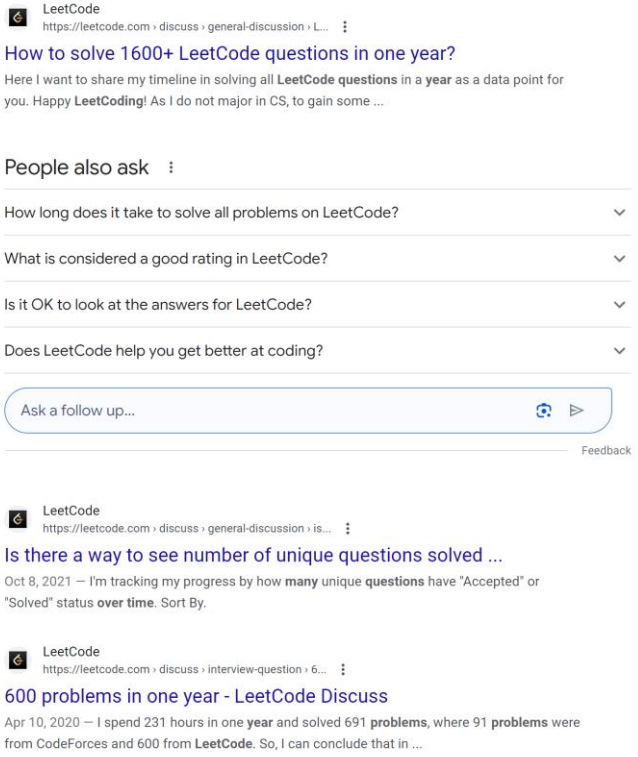
1. we will consider covering more advanced topics in the later part of the course
2. feel free to sign up for research-based projects (to be posted by TA), which can be open-ended, challenging, and rewarding
3. by design, 566 is an intro course for MS students -- you may consider more specialized courses if preferred. Let me name a few:

CSCI 655: Applied NLP
CSCI 599: 3D Vision
CSCI 599: Autonomous DecisionMaking

Discussions on The Depth of the Course

The correlation of getting a job and the course difficulty:

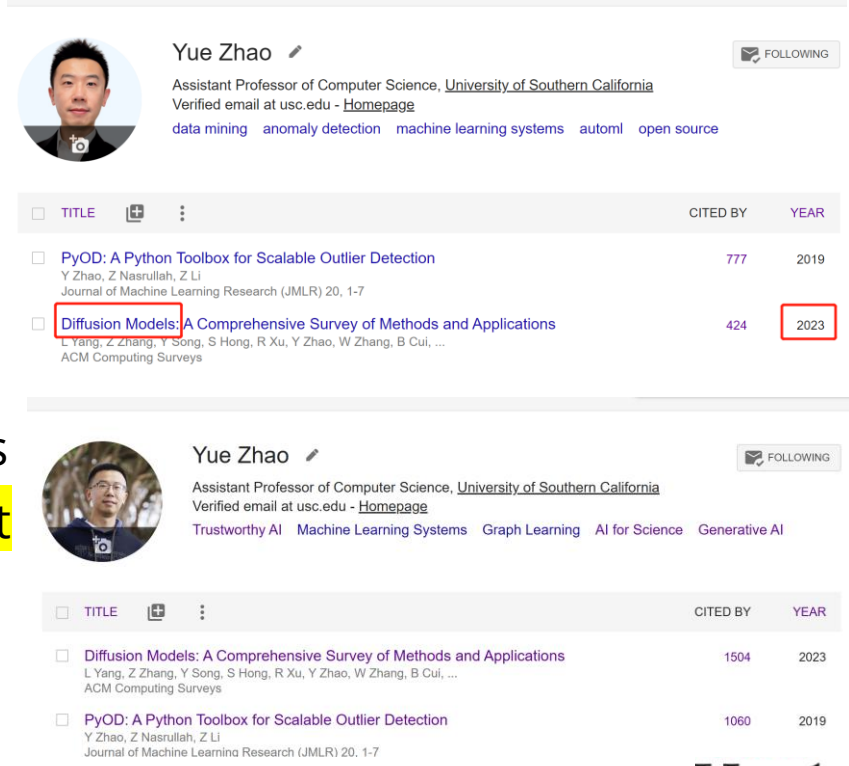
- A challenging course does not necessarily get you a better job
    - It will eat more of your time on **leetcode** and **networking** on advanced but less useful topics



Discussions on The Depth of the Course

The correlation of getting a job and the course difficulty:

- A challenging course does not necessarily get you a better job
  - It will eat more of your time on **leetcode** and **networking** on advanced but less useful topics
  - Advanced topics change so fast



Yue Zhao

FOLLOWING

Assistant Professor of Computer Science, University of Southern California
Verified email at usc.edu - Homepage
data mining anomaly detection machine learning systems automl open source

TITLE	CITED BY	YEAR
PyOD: A Python Toolbox for Scalable Outlier Detection Y Zhao, Z Nasrullah, Z Li Journal of Machine Learning Research (JMLR) 20, 1-7	777	2019
Diffusion Models: A Comprehensive Survey of Methods and Applications L Yang, Z Zhang, Y Song, S Hong, R Xu, Y Zhao, W Zhang, B Cui, ... ACM Computing Surveys	424	2023

Yue Zhao

FOLLOWING

Assistant Professor of Computer Science, University of Southern California
Verified email at usc.edu - Homepage
Trustworthy AI Machine Learning Systems Graph Learning AI for Science Generative AI

TITLE	CITED BY	YEAR
Diffusion Models: A Comprehensive Survey of Methods and Applications L Yang, Z Zhang, Y Song, S Hong, R Xu, Y Zhao, W Zhang, B Cui, ... ACM Computing Surveys	1504	2023
PyOD: A Python Toolbox for Scalable Outlier Detection Y Zhao, Z Nasrullah, Z Li Journal of Machine Learning Research (JMLR) 20, 1-7	1060	2019

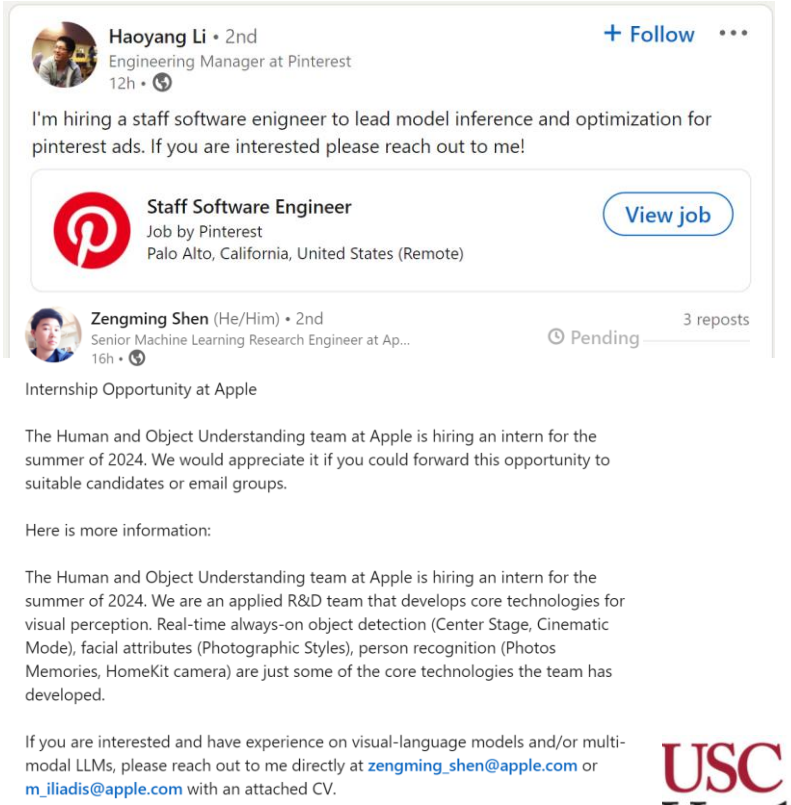
Discussions on The Depth of the Course

How to use LinkedIn more effectively?

1. Identify the post
2. Reach out the person (many times they leave the email)
3. Submit the application

Differentiate yourself:

1. Degree (hmmmm)
2. Unique skillsets, e.g., open-source developers
3. Perfect match to the job, e.g., papers



Haoyang Li • 2nd
Engineering Manager at Pinterest
12h •
+ Follow

I'm hiring a staff software enigneer to lead model inference and optimization for pinterest ads. If you are interested please reach out to me!

Staff Software Engineer
Job by Pinterest
Palo Alto, California, United States (Remote)
View job

Zengming Shen (He/Him) • 2nd
Senior Machine Learning Research Engineer at Ap...
16h •
3 reposts
Pending

Internship Opportunity at Apple

The Human and Object Understanding team at Apple is hiring an intern for the summer of 2024. We would appreciate it if you could forward this opportunity to suitable candidates or email groups.

Here is more information:

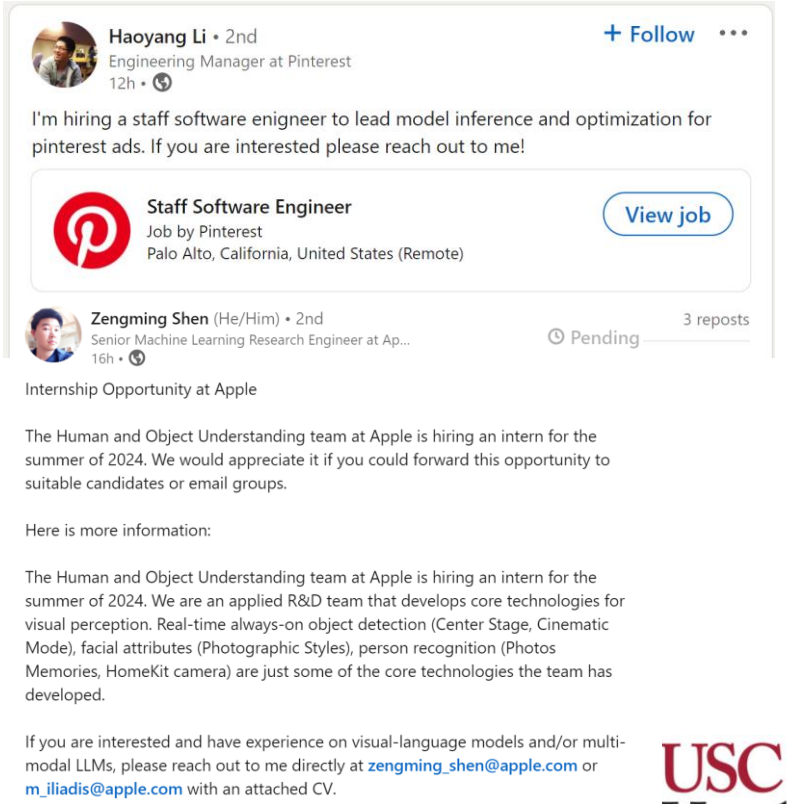
The Human and Object Understanding team at Apple is hiring an intern for the summer of 2024. We are an applied R&D team that develops core technologies for visual perception. Real-time always-on object detection (Center Stage, Cinematic Mode), facial attributes (Photographic Styles), person recognition (Photos Memories, HomeKit camera) are just some of the core technologies the team has developed.

If you are interested and have experience on visual-language models and/or multi-modal LLMs, please reach out to me directly at zengming_shen@apple.com or m_iliadis@apple.com with an attached CV.

Discussions on The Depth of the Course

How to use LinkedIn more effectively?

1. Do not spend too much time on any single position – they can be fake
2. Apply to the job which may not necessarily fit (who knows)
3. Put some emphasis on non-tech industries, e.g., healthcare, mining, finance.



Haoyang Li • 2nd
Engineering Manager at Pinterest
12h •
+ Follow

I'm hiring a staff software enigneer to lead model inference and optimization for pinterest ads. If you are interested please reach out to me!

Staff Software Engineer
Job by Pinterest
Palo Alto, California, United States (Remote)
View job

Zengming Shen (He/Him) • 2nd
Senior Machine Learning Research Engineer at Ap...
16h •
3 reposts
Pending

Internship Opportunity at Apple

The Human and Object Understanding team at Apple is hiring an intern for the summer of 2024. We would appreciate it if you could forward this opportunity to suitable candidates or email groups.

Here is more information:

The Human and Object Understanding team at Apple is hiring an intern for the summer of 2024. We are an applied R&D team that develops core technologies for visual perception. Real-time always-on object detection (Center Stage, Cinematic Mode), facial attributes (Photographic Styles), person recognition (Photos Memories, HomeKit camera) are just some of the core technologies the team has developed.

If you are interested and have experience on visual-language models and/or multi-modal LLMs, please reach out to me directly at zengming_shen@apple.com or m_iliadis@apple.com with an attached CV.

Diversify Your Skillset (Personal Story)



Yue Zhao
yzhao062

Assistant Professor of Computer Science @ University of Southern California (USC); Ph.D. @ CMU. Anomaly/Outlier Detection | ML Systems | AutoML

Edit profile

Sponsors dashboard

3.5k followers · **22** following

University of Southern California
Los Angeles, CA, USA
https://viterbi-web.usc.edu/~yzhao010/
@yzhao062
in/yzhao062

Pinned

Customize your pins

pyod Public
A Comprehensive and Scalable Python Library for Outlier Detection (Anomaly Detection)
Python 7.7k 1.3k

anomaly-detection-resources Public
Anomaly detection related books, papers, videos, and toolboxes
Python 7.7k 1.7k

WSAD Public
A Collection of Resources for Weakly-supervised Anomaly Detection (WSAD)
Python 114 5

Minqi824/ADBench Public
Official Implement of "ADBench: Anomaly Detection Benchmark", NeurIPS 2023.
Python 717 120

pytod Public
TOD: GPU-accelerated Outlier Detection via Tensor Operations
Python 162 22

pygod-team/pygod Public
A Python Library for Graph Outlier Detection (Anomaly Detection)
Python 1.1k 119

224 contributions in the last year

Contribution settings

2024
2023
2022
2021

Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
Mon
Wed
Fri

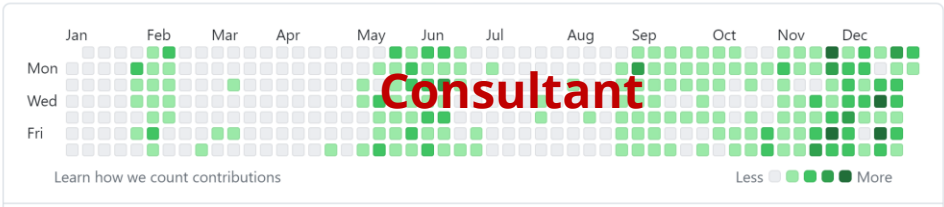
Learn how we count contributions

Less More

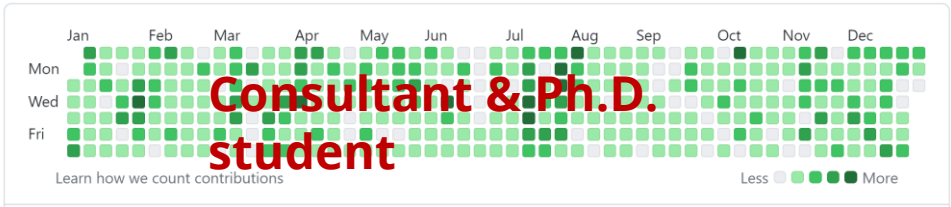
USC Viterbi School of Engineering

Diversify Your Skillset

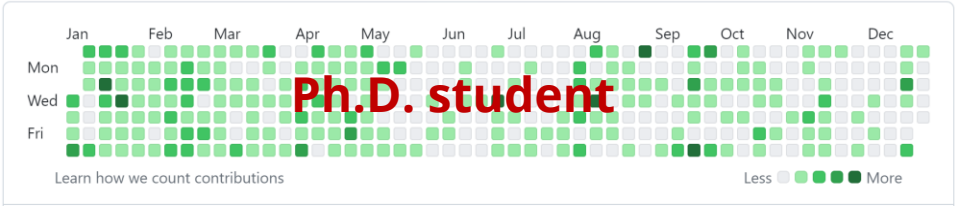
974 contributions in 2018



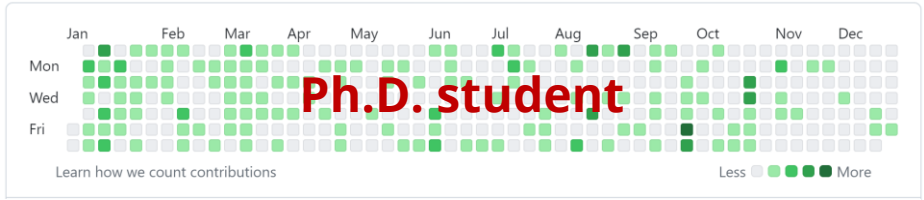
1,344 contributions in 2019



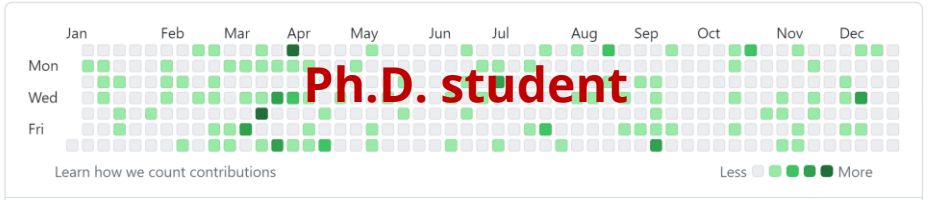
716 contributions in 2020



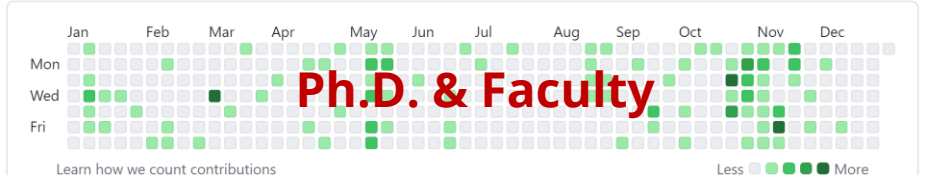
393 contributions in 2021



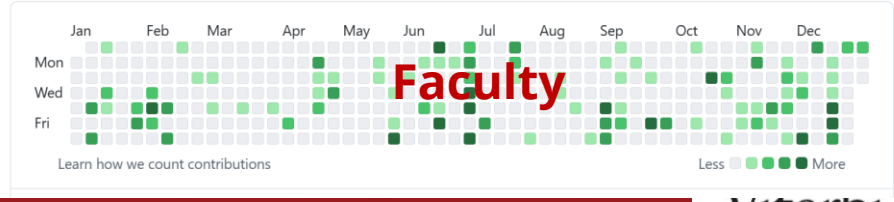
228 contributions in 2022



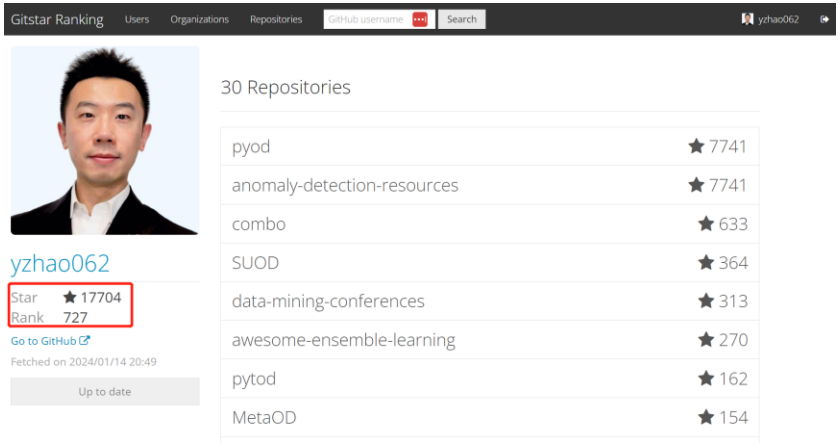
220 contributions in 2023

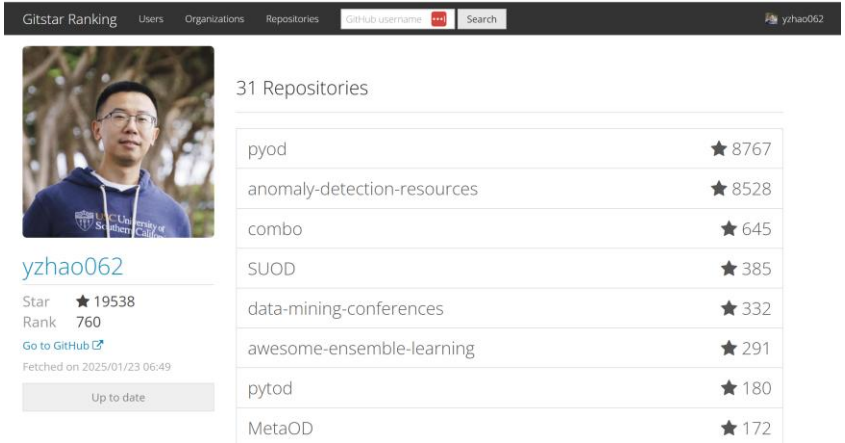


213 contributions in 2024

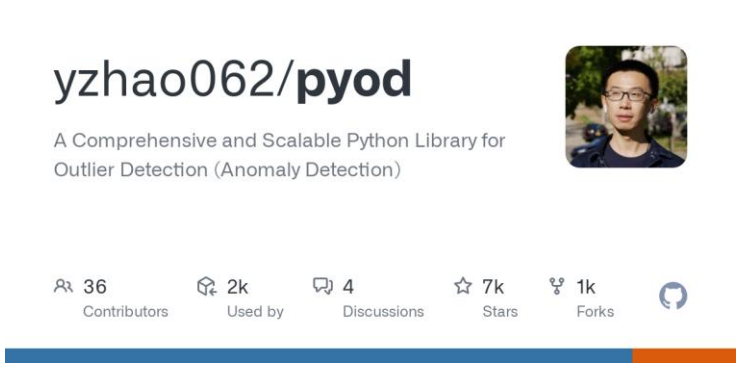


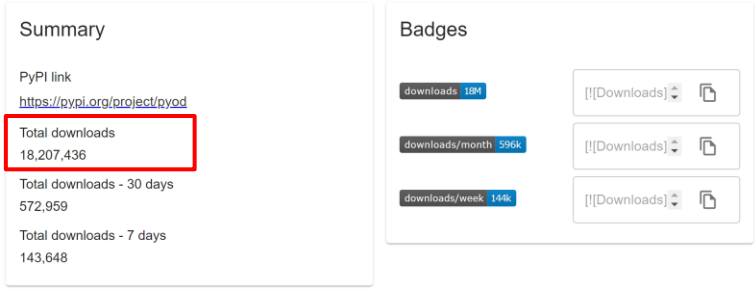
Diversify Your Skillset





Deployed Systems and Real-world Applications





PyOD library has **57M+ downloads**:

- **Morgan Stanley** for risk modeling
- **Tesla** for manufacturing quality control
- **NASA** for supply chain management
- **Microsoft** for security breach detection

Morgan Stanley

TESLA

NASA

Microsoft

Viterbi School of Engineering

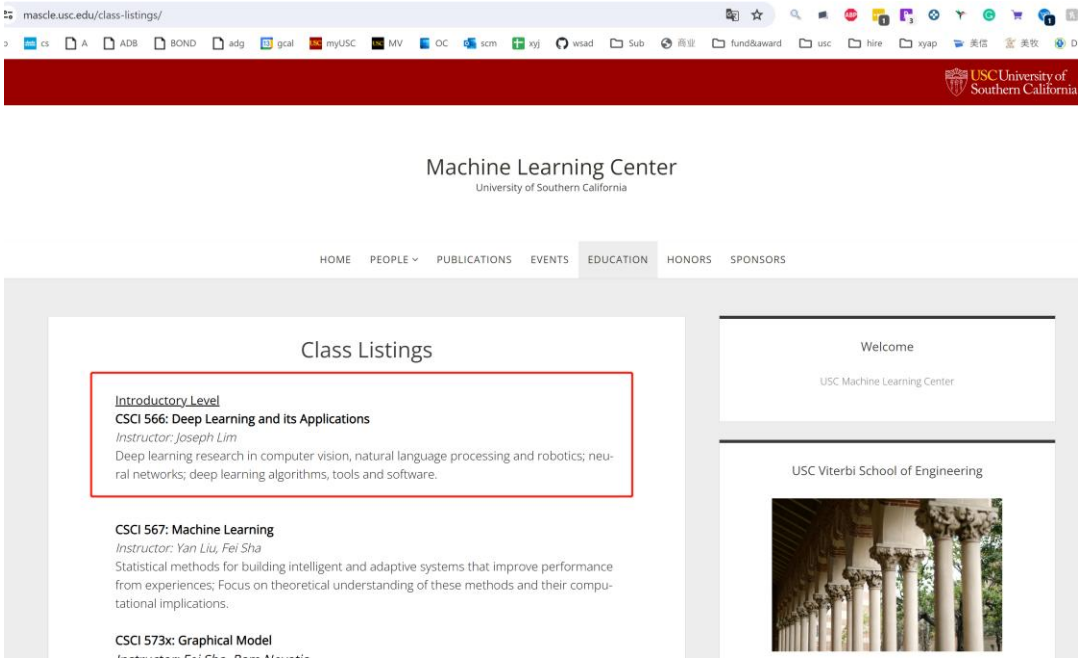
The Goal & Value Proposition of This Course

Have an understanding of deep learning topics:

- Understand when and where to use them

Knowing better about the industry:

- How to find a better job or career planning



USC Viterbi School of Engineering

Where to Ask

If your question is about	Post it to
Assignments, project, course content	Piazza, public post
Grades, extensions, personal matters	Piazza, private post to instructors
Anything you want discussed live	Office hours

Piazza is the primary channel given the size of the course. Please do not email staff individually. Office hours are on p. 4.

Logistics

- Time: Mondays, 5:00-8:20 PM. Fourteen meetings, August 24 to November 30. No class September 7 (Labor Day).
- Where: SGM 124 for section 30122; DEN@Viterbi for section 30359. Both sections get the same lecture at the same time.
- Course website: https://usc-fortis.github.io/CSCI566-F26/
- Discussion: Piazza. Use it for anything course-related rather than email.
- Guest lectures: industry and academic professionals join regularly.

Logistics

Course website: https://usc-fortis.github.io/CSCI566-F26/

Please use **Piazza** for any course-related communication
. The Fall 2026 channel is linked from the course website above.

This will be our primary place for answering the questions given the size of the course

Any non-necessary e-mail will be ignored.

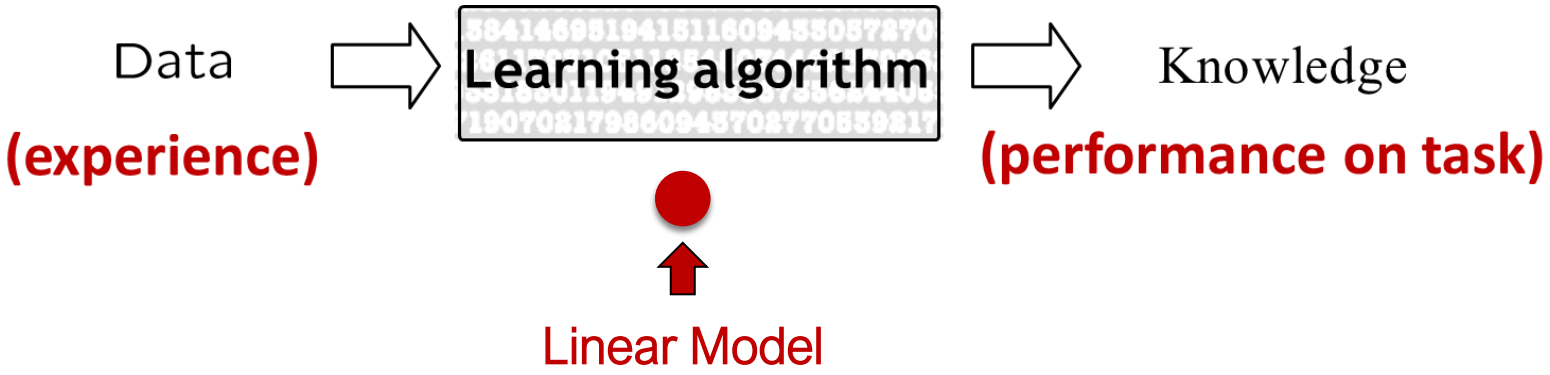
Compute and Cloud Credits

- **Start with the free notebooks.** Kaggle gives a P100 or two T4s, about 30 GPU hours a week. Colab also gives a free GPU, but does not publish the card or the limits.
- **Student offers.** Azure for Students gives $100 over 12 months with no credit card. The GitHub Student Pack adds a few GPU hours a month.
- **The $300 trial does not buy GPU time.** A trial account cannot attach a GPU. You must upgrade to a paid account first, and then anything past the credit bills your card.
- **A budget alert is not a brake.** Google says an alerts-only budget does not cap usage or spending. It emails you while the meter runs.
- **Course credit and CARC are not settled.** Wait for my announcement. Until then, stop or delete every paid VM and disk when you finish.

Recap from Week 1

Linear Models and Optimization

Recap: Tasks, Experience, Performance

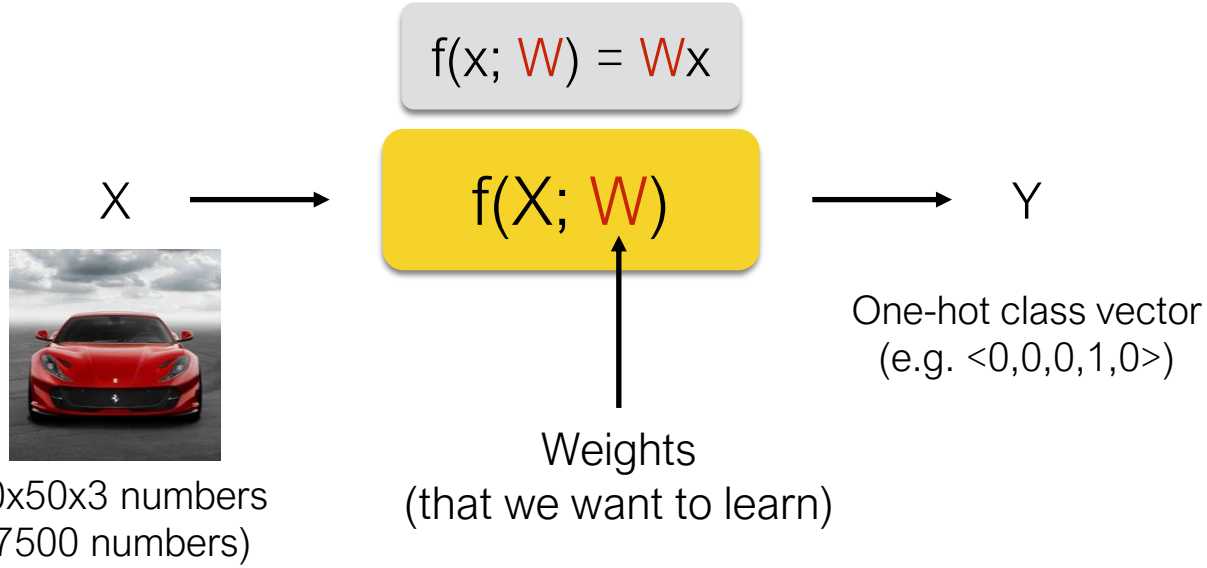


Recap: From Linear Classification to Deep Learning



Let's first talk about learning a simple function.

Recap: From Linear Classification to Deep Learning



Recap: From Linear Classification to Deep Learning

5x1 5x7500 7500x1

$f(x; W) = Wx + b$ 5x1



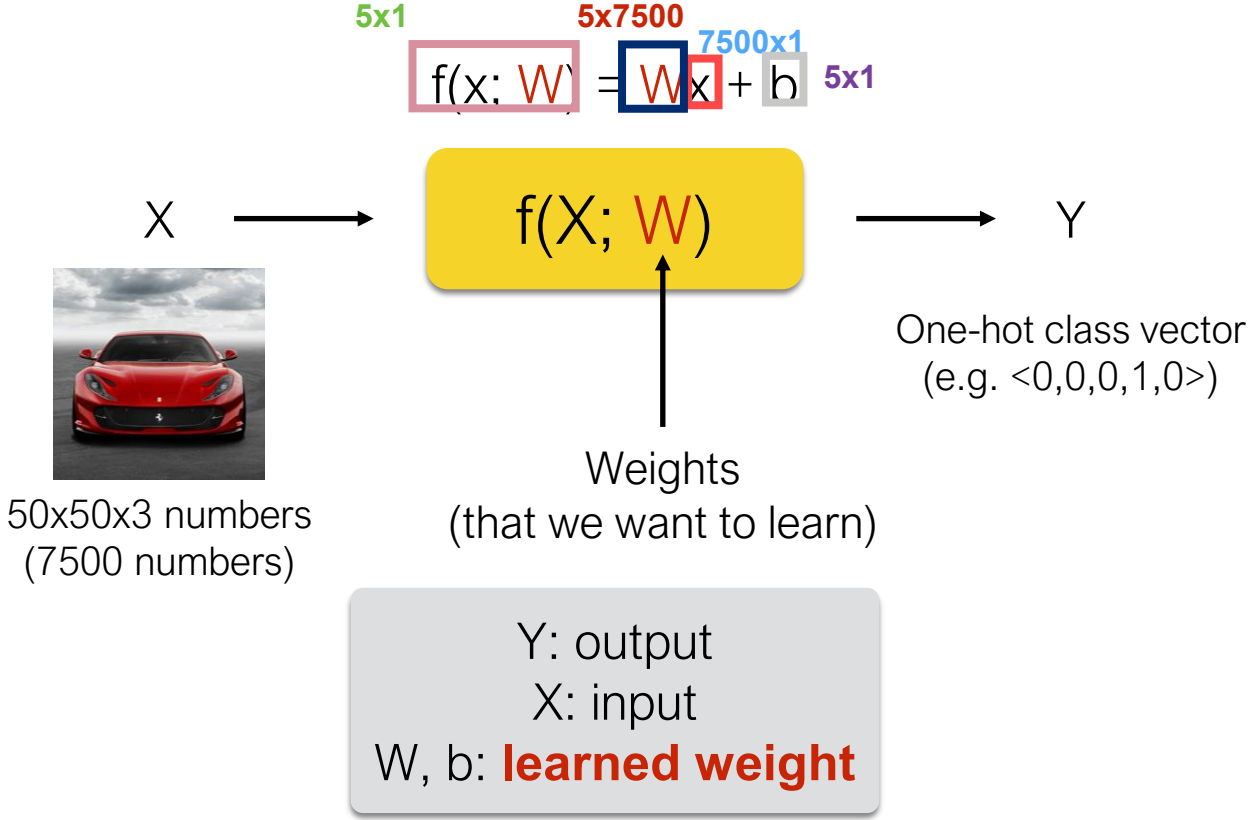
50x50x3 numbers
(7500 numbers)

X → f(X; W) → Y

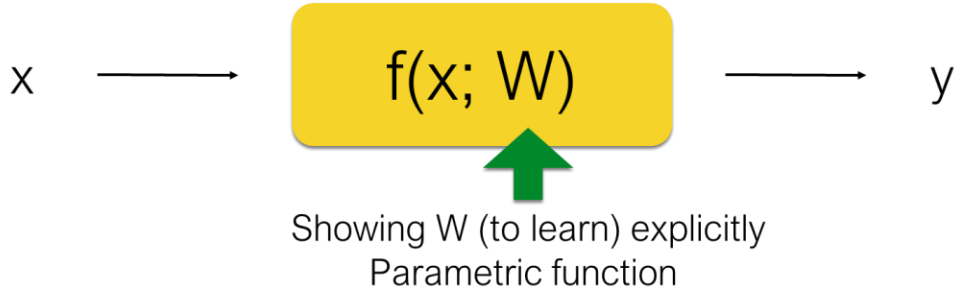
Weights
(that we want to learn)

One-hot class vector
(e.g. <0,0,0,1,0>)

Recap: From Linear Classification to Deep Learning

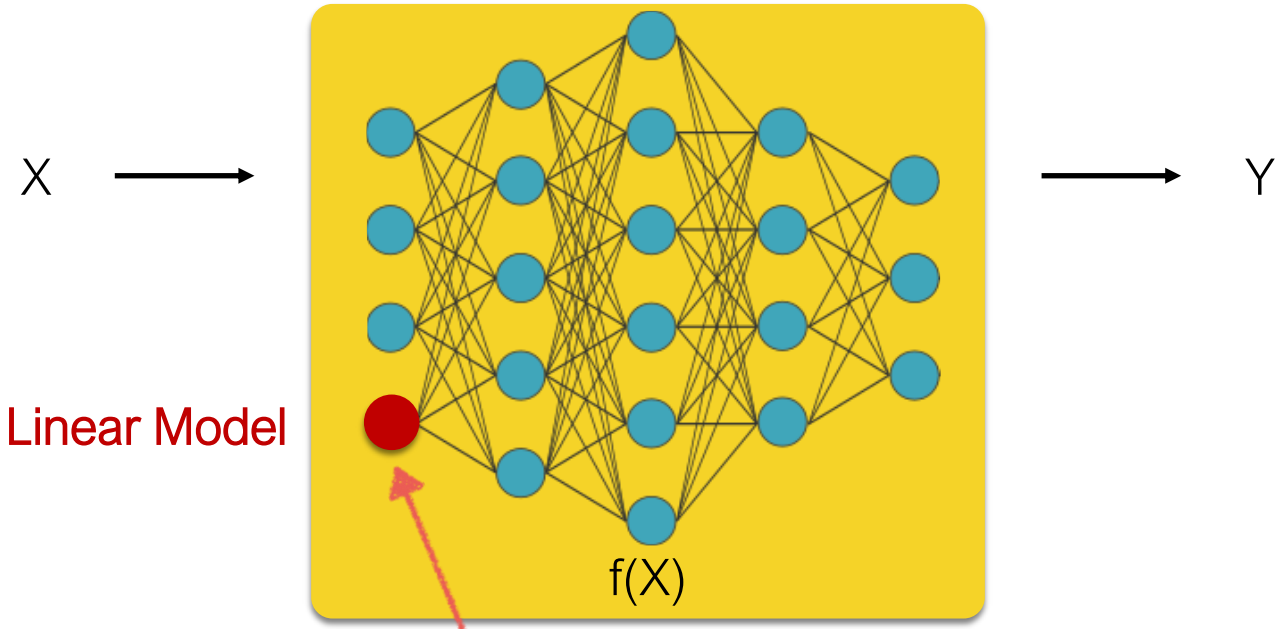


Summary of Loss Function and Optimization



- **Loss function (L)** measures how well learned W can map X to Y (compared to y*).
- **Optimization** finds the best W given a loss function L (i.e. finding W that minimizes L).

Recap: From Linear Classification to Deep Learning



Apparently, this is a unit used in deep neural networks too.

Alternatives to Learn Function f



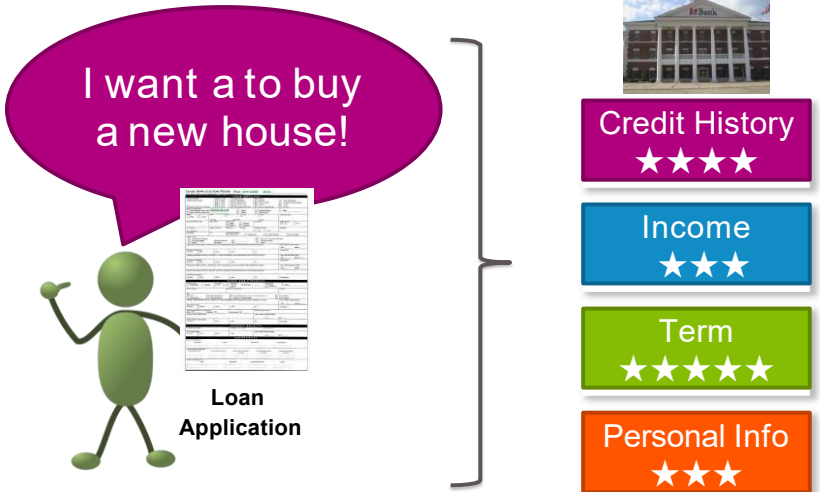
Of course, there are more approaches beyond linear models, which is parametric

Classical ML Algorithms

Decision Trees

Decision Tree

What makes a loan risky?



Credit to Stanford CS 229

Decision Tree

Credit history explained

Did I pay previous loans on time?

Example:
excellent, fair, or poor



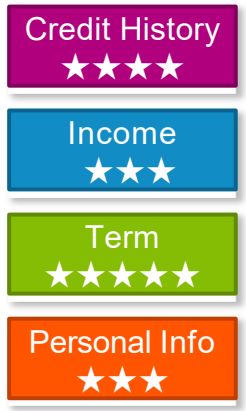
Credit to Stanford CS 229

Decision Tree

Income

What's my income?

Example:
$80K per year



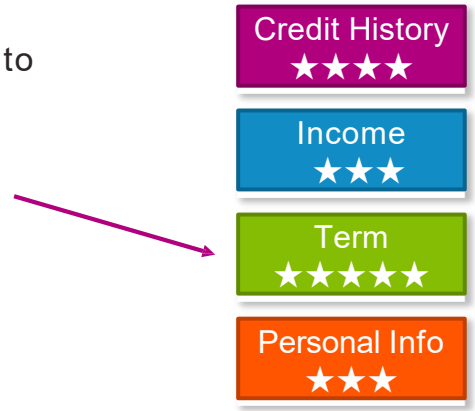
Credit to Stanford CS 229

Decision Tree

Loan terms

How soon do I need to pay the loan?

Example: 3 years, 5 years,…



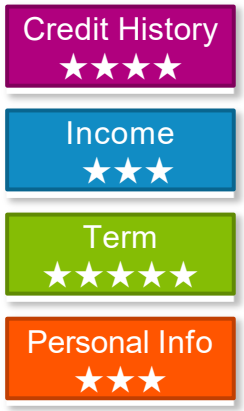
Credit to Stanford CS 229

Decision Tree

Personal information

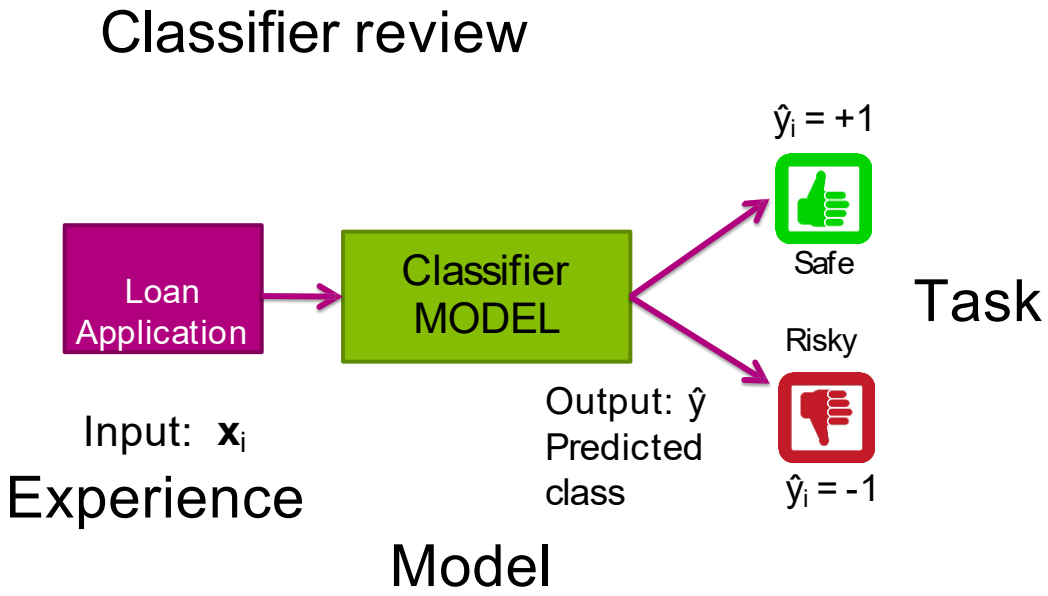
Age, reason for the loan, marital status,…

Example: Home loan for a married couple



Credit to Stanford CS 229

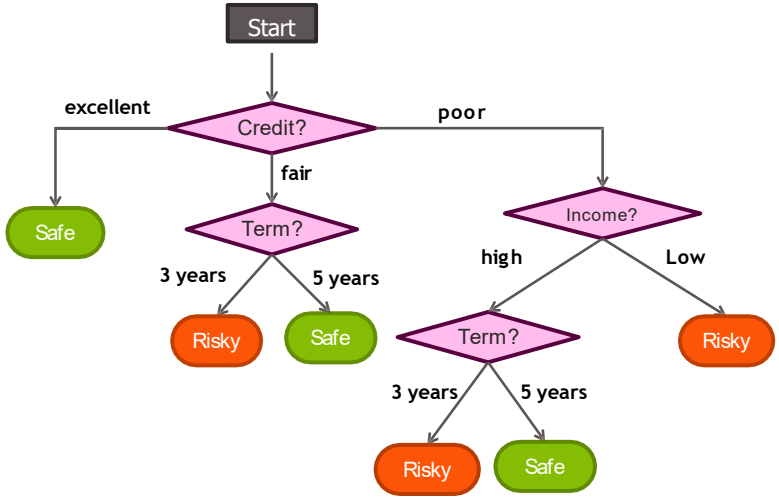
Decision Tree – Experience, Model, Task



Credit to Stanford CS 229

Decision Tree

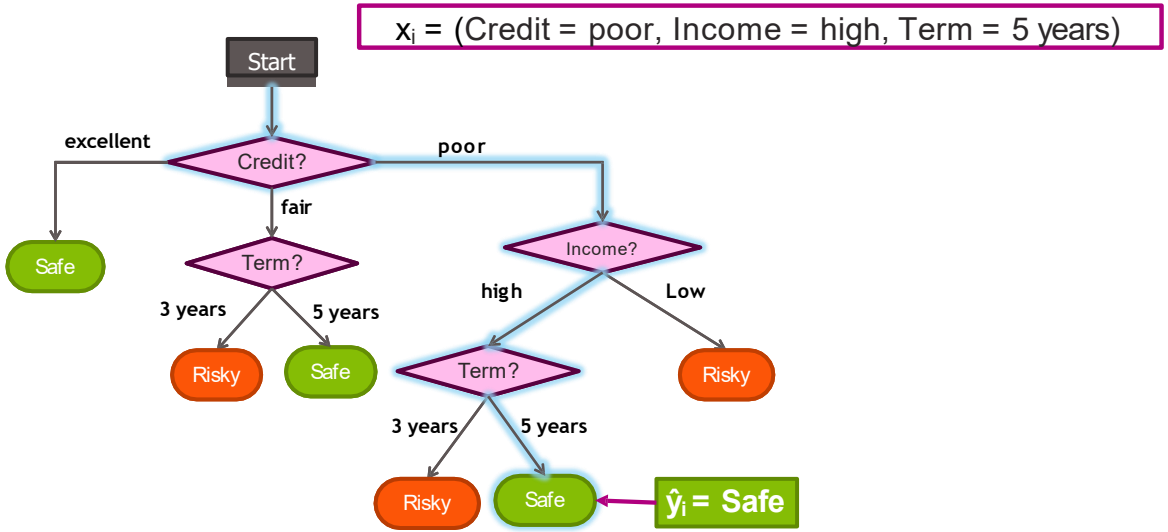
This module ... decision trees



Credit to Stanford CS 229

Decision Tree

Scoring a loan application



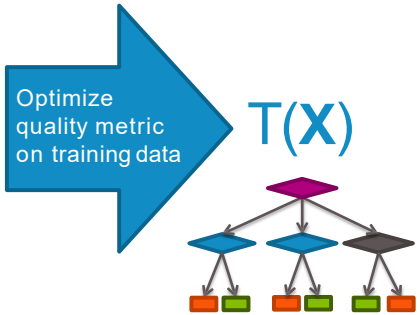
Credit to Stanford CS 229

Decision Tree

Decision tree learning problem

Training data: N observations (x_i, y_i)

Credit	Term	Income	y
excellent	3 yrs	high	safe
fair	5 yrs	low	risky
fair	3 yrs	high	safe
poor	5 yrs	high	risky
excellent	3 yrs	low	risky
fair	5 yrs	low	safe
poor	3 yrs	high	risky
poor	5 yrs	low	safe
fair	3 yrs	high	safe



Credit to Stanford CS 229

Decision Tree - Performance

Quality metric: Classification error

- Error measures fraction of mistakes

$$\text{Error} = \frac{\#\ \text{incorrect predictions}}{\#\ \text{examples}}$$

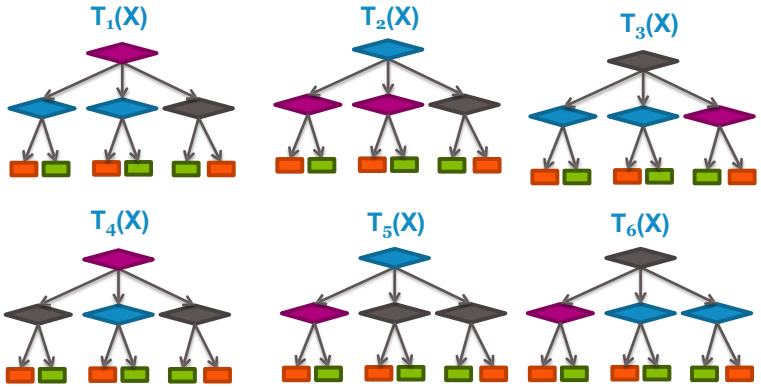
  - Best possible value : 0.0
  - Worst possible value: 1.0

Credit to Stanford CS 229

Decision Tree

How do we find the best tree?

Exponentially large number of possible trees makes decision tree learning hard!



Learning the smallest decision tree is an ***NP-hard*** *problem* [Hyafil & Rivest '76]

Credit to Stanford CS 229

Decision Tree

Our training data table

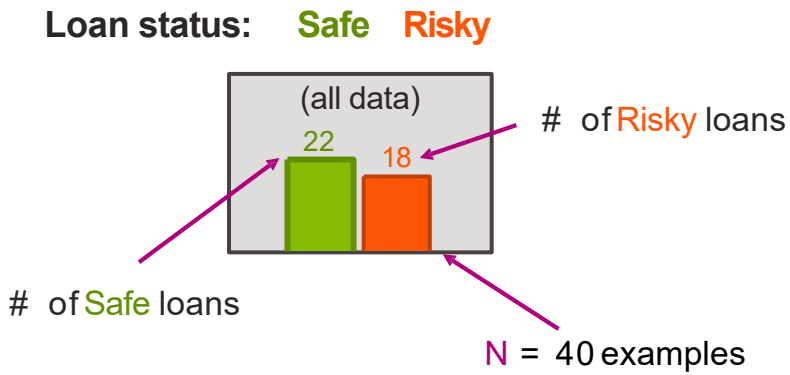
Assume N = 40, 3 features

Credit	Term	Income	y
excellent	3 yrs	high	safe
fair	5 yrs	low	risky
fair	3 yrs	high	safe
poor	5 yrs	high	risky
excellent	3 yrs	low	risky
fair	5 yrs	low	safe
poor	3 yrs	high	risky
poor	5 yrs	low	safe
fair	3 yrs	high	safe

Credit to Stanford CS 229

Decision Tree

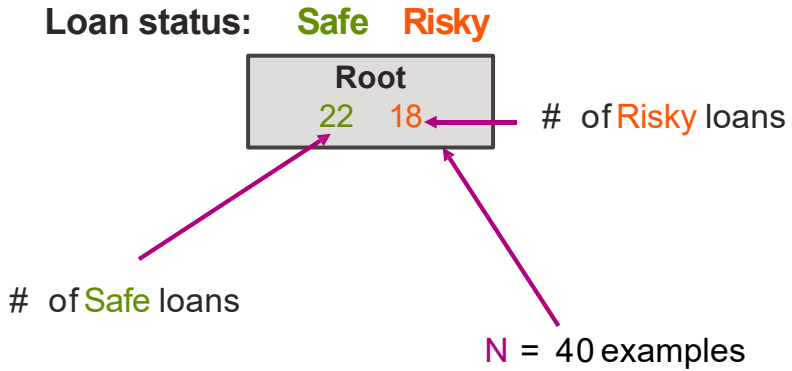
Start with all the data



Credit to Stanford CS 229

Decision Tree

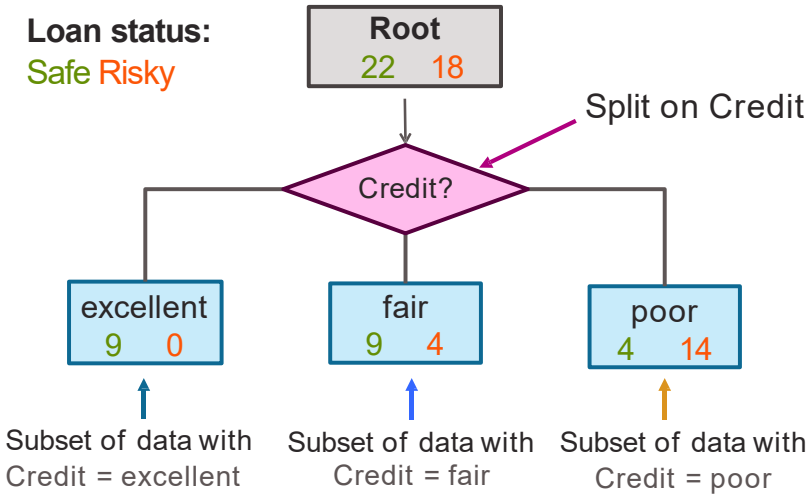
Compact visual notation: Root node



Credit to Stanford CS 229

Decision Tree

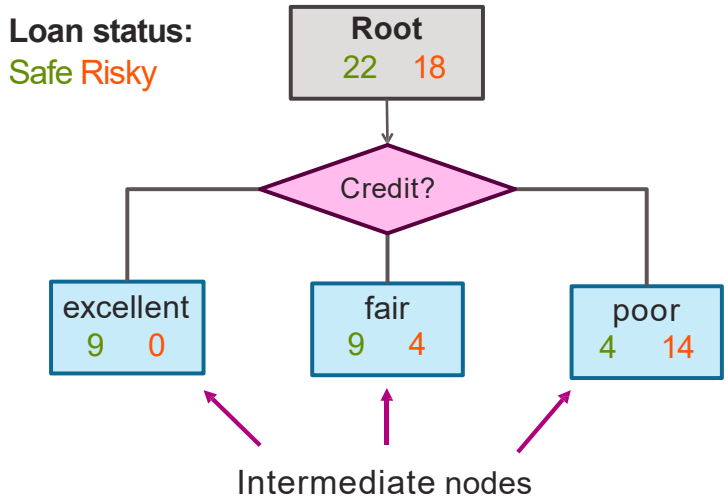
Decision stump: Single level tree



Credit to Stanford CS 229

Decision Tree

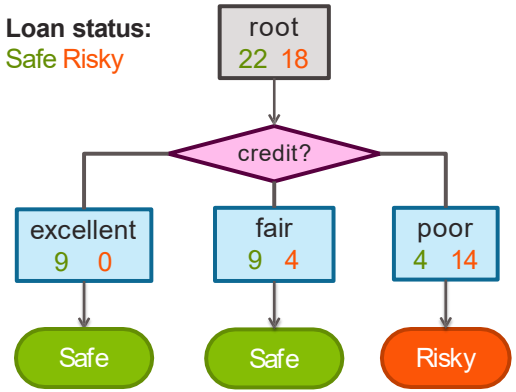
Visual notation: Intermediate nodes



Credit to Stanford CS 229

Decision Tree

Making predictions with a decision stump

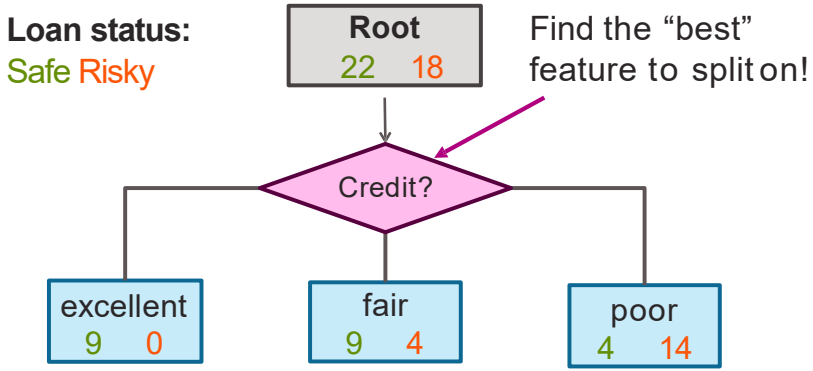


For each intermediate node, set $\hat{y}$ = majority value

Credit to Stanford CS 229

Decision Tree

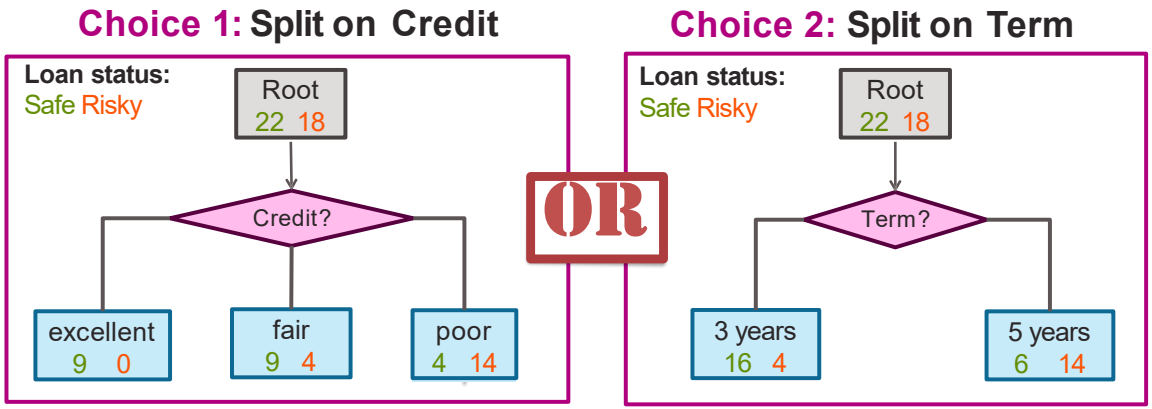
How do we learn a decision stump?



Credit to Stanford CS 229

Decision Tree

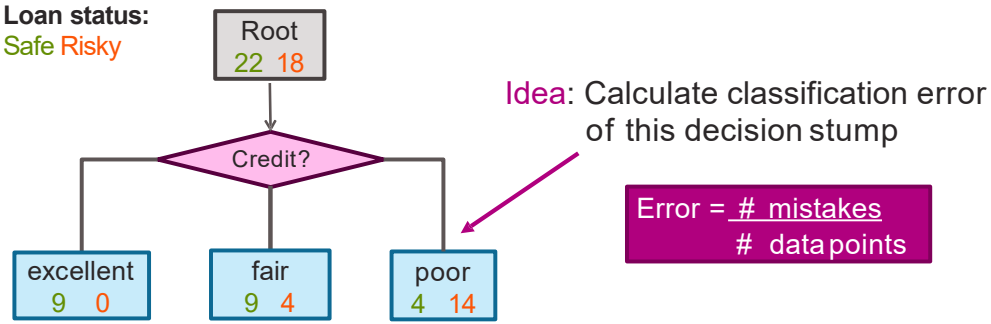
How do we select the best feature?



Credit to Stanford CS 229

Decision Tree

How do we measure effectiveness of a split?

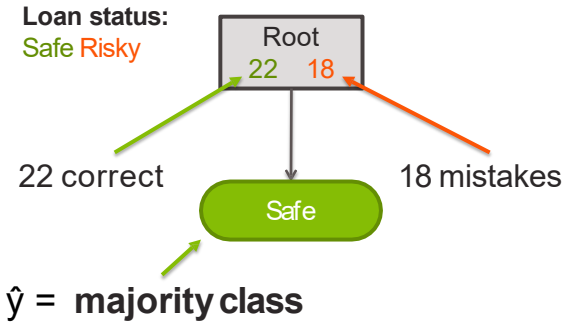


Credit to Stanford CS 229

Decision Tree

Calculating classification error

- Step 1: $\hat{y}$ = class of majority of data in node
- Step 2: Calculate classification error of predicting $\hat{y}$ for this data



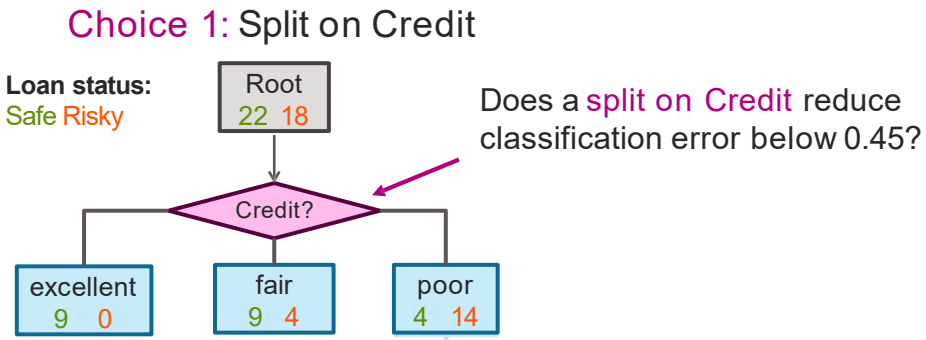
Error = $\frac{\#\ \text{mistakes}}{\#\ \text{data points}}$

Tree	Classification error
(root)	0.45

Credit to Stanford CS 229

Decision Tree

Choice 1: Split on Credit history?

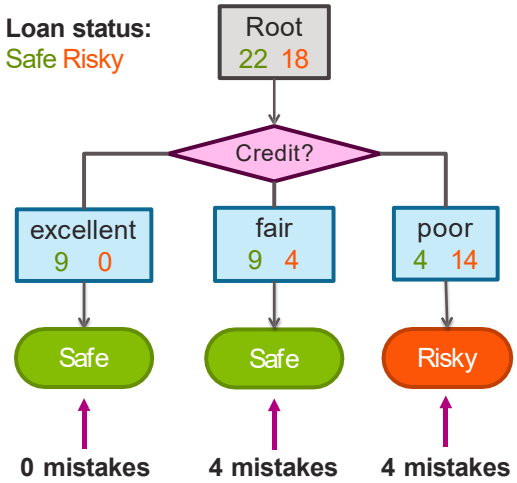


Credit to Stanford CS 229

Decision Tree

Split on Credit: Classification error

Choice 1: Split on Credit



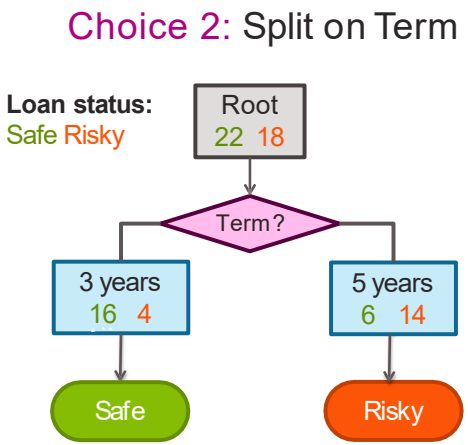
$$\text{Error} = \frac{\#\ \text{mistakes}}{\#\ \text{data points}}$$

Tree	Classification error
(root)	0.45
Split on credit	0.2

Credit to Stanford CS 229

Decision Tree

Choice 2: Split on Term?

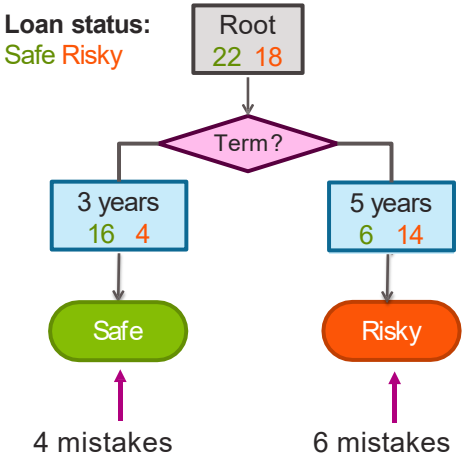


Credit to Stanford CS 229

Decision Tree

Evaluating the split on Term

Choice 2: Split on Term



Error = # mistakes / # data points

Tree	Classification error
(root)	0.45
Split on credit	0.2
Split on term	0.25

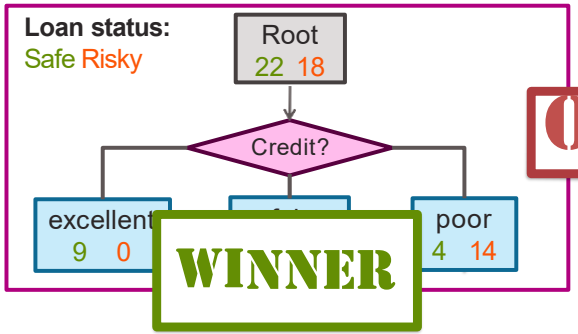
Credit to Stanford CS 229

Decision Tree

Choice 1 vs Choice 2: Comparing split on Credit vs Term

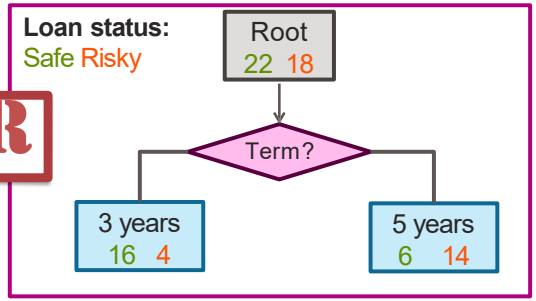
Tree	Classification error
(root)	0.45
split on credit	0.2
split on loanterm	0.25

Choice 1: Split on Credit



OR

Choice 2: Split on Term



Credit to Stanford CS 229

Decision Tree

Feature split selection algorithm

- Given a subset of data M (a node in a tree)
- For each feature $h_i(x)$:
  1. Split data of M according to feature $h_i(x)$
  2. Compute classification error of split
- Choose feature $h^*(x)$ with lowest classification error

Credit to Stanford CS 229

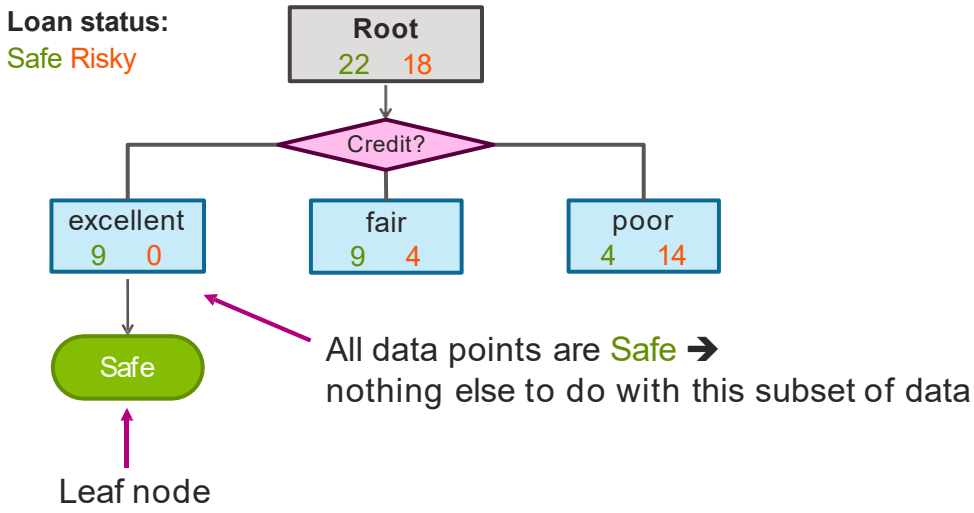
Decision Tree

Recursion & Stopping conditions

Credit to Stanford CS 229

Decision Tree

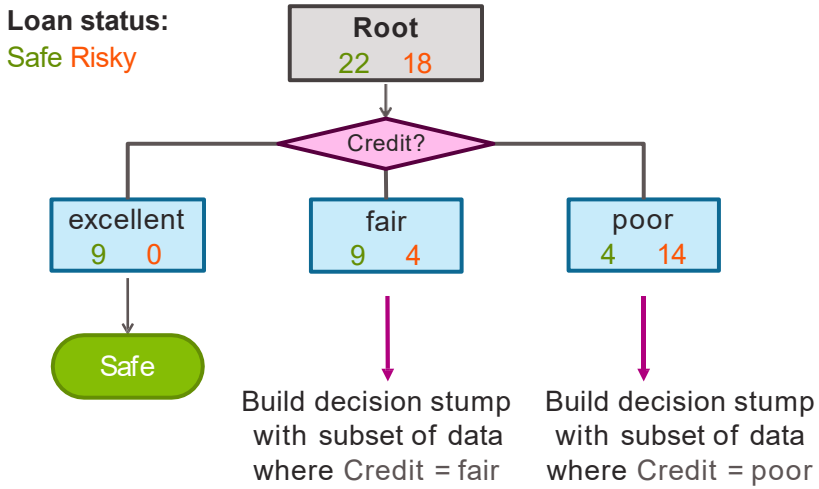
We've learned a decision stump, what next?



Credit to Stanford CS 229

Decision Tree

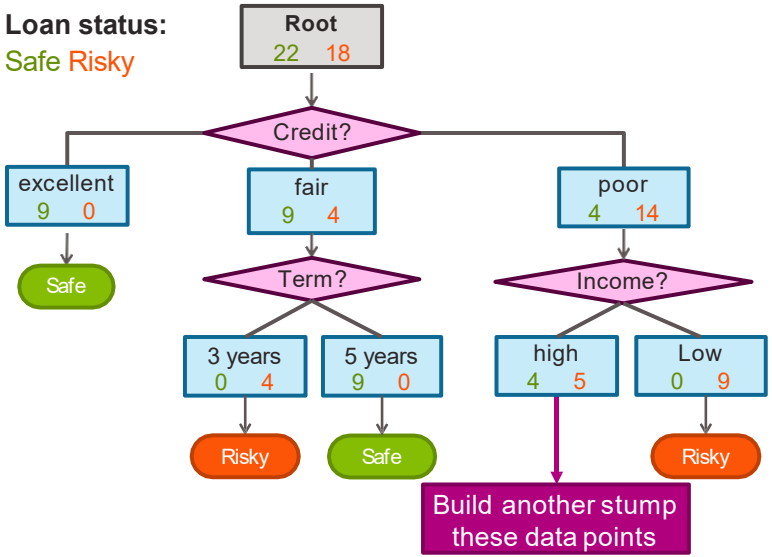
Tree learning = Recursive stump learning



Credit to Stanford CS 229

Decision Tree

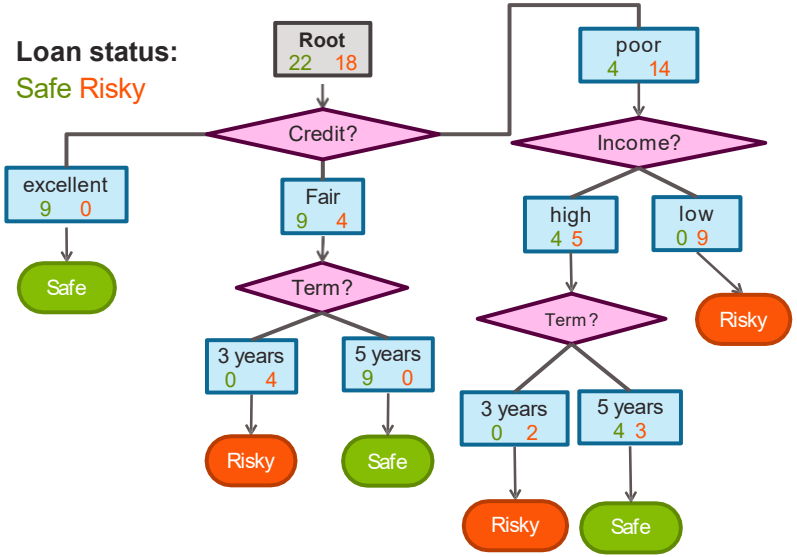
Second level



Credit to Stanford CS 229

Decision Tree

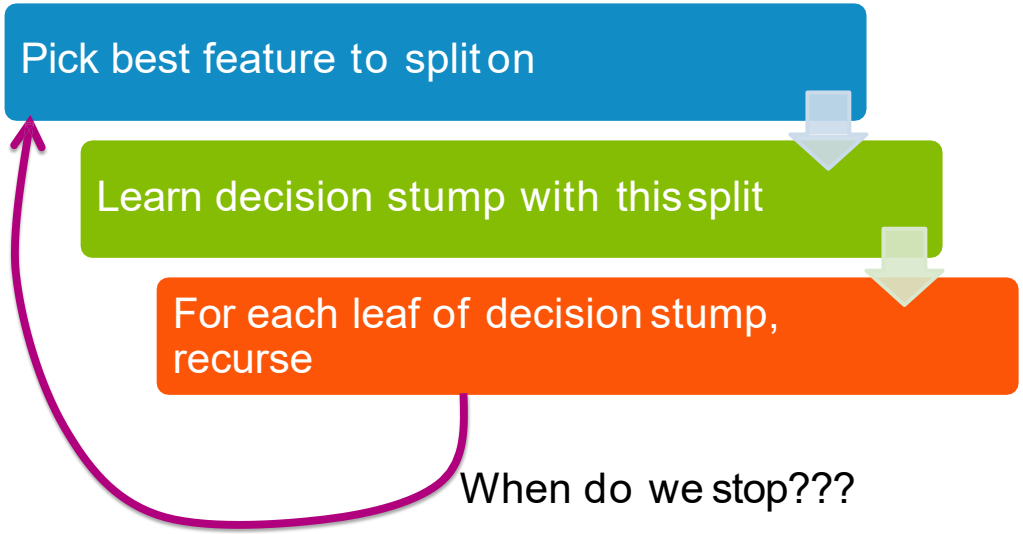
Final decision tree



Credit to Stanford CS 229

Decision Tree

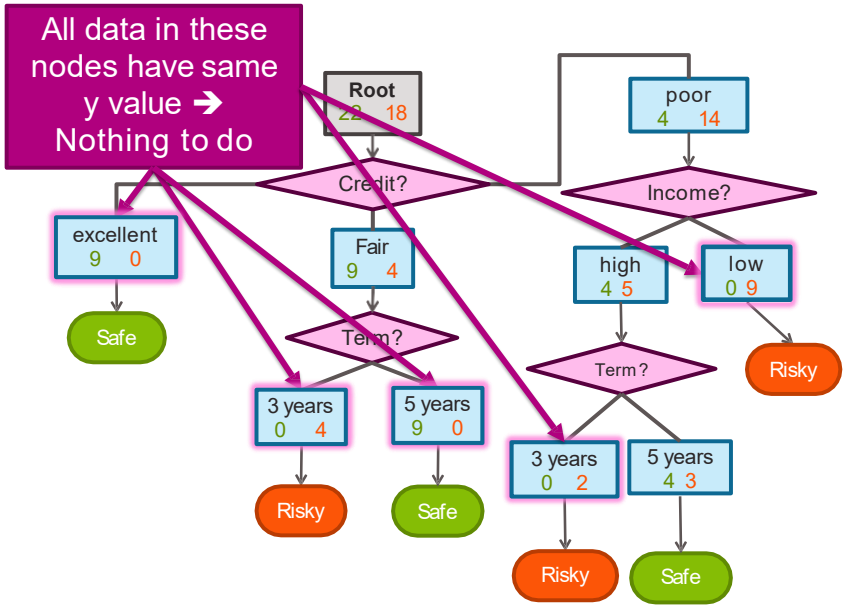
Simple greedy decision tree learning



Credit to Stanford CS 229

Decision Tree

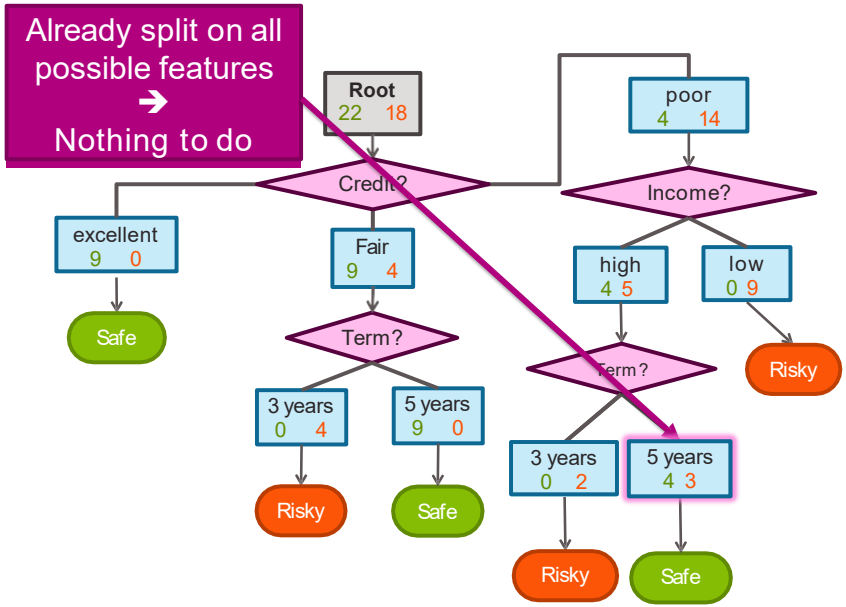
Stopping condition 1: All data agrees on y



Credit to Stanford CS 229

Decision Tree

Stopping condition 2: Already split on all features



Credit to Stanford CS 229

Decision Tree

Greedy decision tree learning

- Step 1: Start with an empty tree
- Step 2: Select a feature to split data
- For each split of the tree:
  - Step 3: If nothing more to do, make predictions
  - Step 4: Otherwise, go to Step 2 & continue (recurse) on this split

Pick feature split leading to lowest classification error

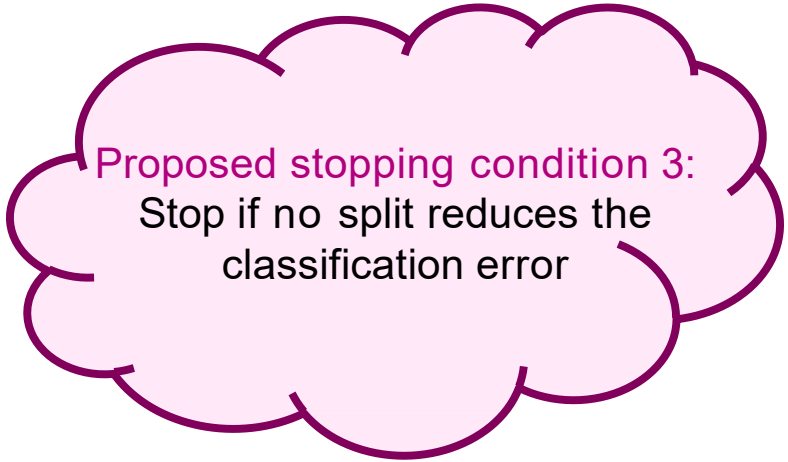
Stopping conditions

Recursion

Credit to Stanford CS 229

Decision Tree

Is this a good idea?



Credit to Stanford CS 229

Decision Tree

Stopping condition 3:
Don't stop if error doesn't decrease???

$y = x[1] \text{ xor } x[2]$

x[1]	x[2]	y
False	False	False
False	True	True
True	False	True
True	True	False

y values
True False

Root
2 2

Error = # mistakes / # data points

Tree	Classification error
(root)	0.5

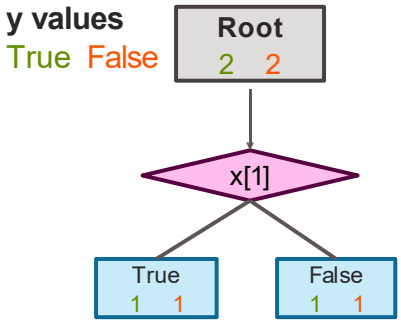
Credit to Stanford CS 229

Decision Tree

Consider split on x[1]

y = x[1] xor x[2]

x[1]	x[2]	y
False	False	False
False	True	True
True	False	True
True	True	False



Error = # mistakes / # data points

Tree	Classification error
(root)	0.5
Split on x[1]	0.5

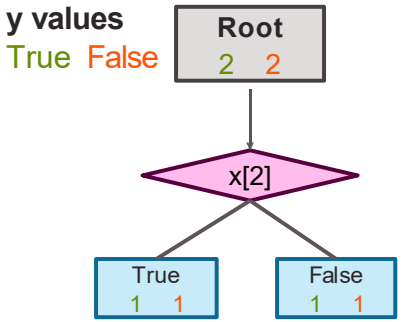
Credit to Stanford CS 229

Decision Tree

Consider split on x[2]

$y = x[1]$ xor $x[2]$

x[1]	x[2]	y
False	False	False
False	True	True
True	False	True
True	True	False



Error = $\frac{1+1}{2+2}$ = 0.5

Neither features improve training error… Stop now???

Tree	Classification error
(root)	0.5
Split on x[1]	0.5
Split on x[2]	0.5

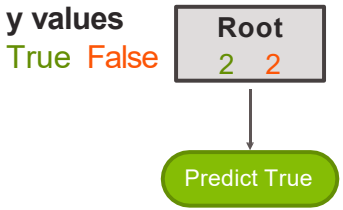
Credit to Stanford CS 229

Decision Tree

Final tree with stopping condition 3

$y = x[1] \text{ xor } x[2]$

x[1]	x[2]	y
False	False	False
False	True	True
True	False	True
True	True	False



Tree	Classification error
with stopping condition 3	0.5

Credit to Stanford CS 229

Decision Tree

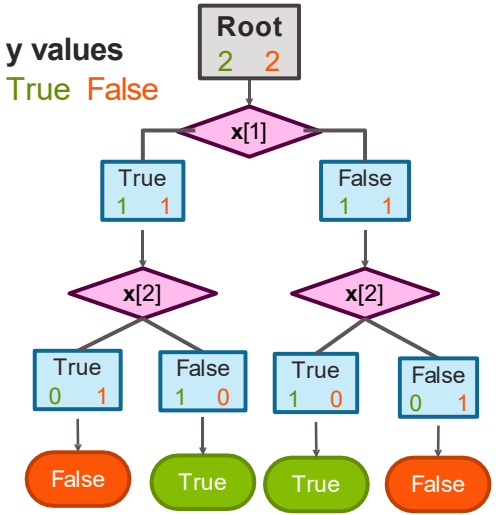
Without stopping condition 3

Condition 3 (stopping when training error doesn't' improve) **is not recommended!**

$y = x[1] \text{ xor } x[2]$

x[1]	x[2]	y
False	False	False
False	True	True
True	False	True
True	True	False

Tree	Classification error
with stopping condition 3	0.5
without stopping condition 3	

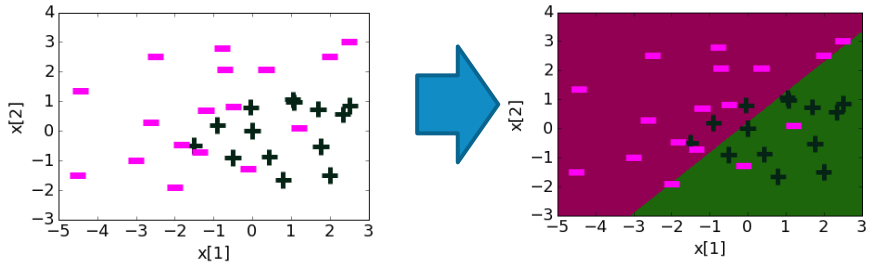


Credit to Stanford CS 229

Decision Tree

Linear classification

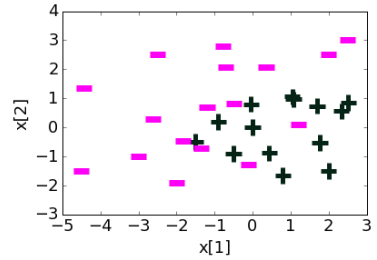
Feature	Value	Weight Learned
$h_0(x)$	1	0.22
$h_1(x)$	x[1]	1.12
$h_2(x)$	x[2]	-1.07

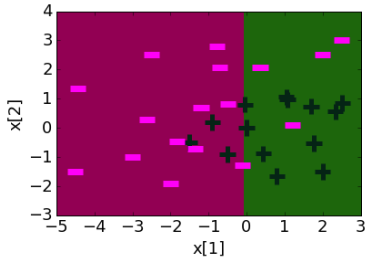


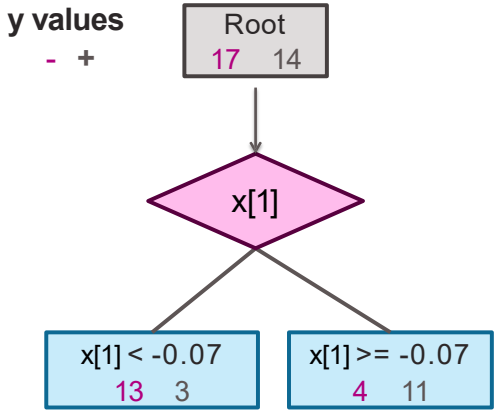
Credit to Stanford CS 229

Decision Tree

Depth 1: Split on x[1]



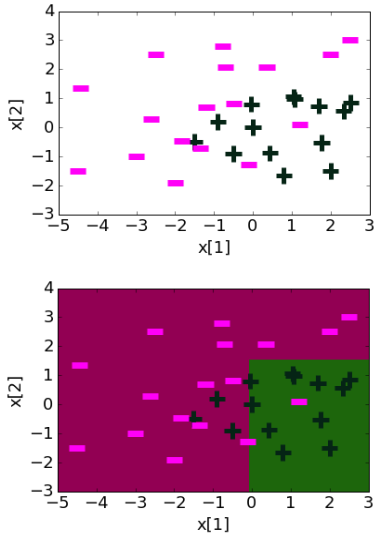


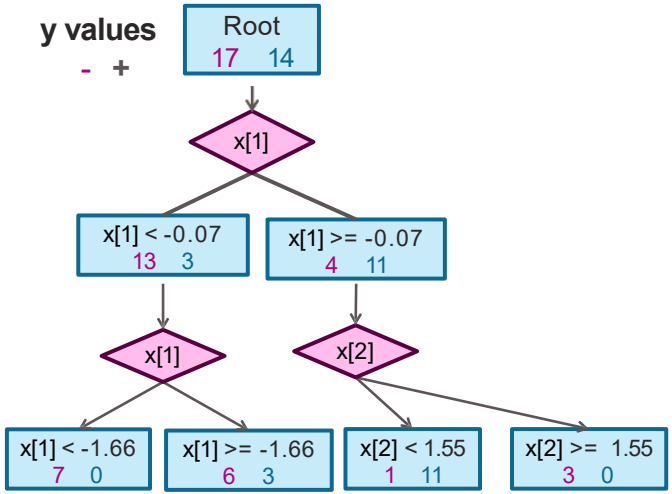


Credit to Stanford CS 229

Decision Tree

Depth 2



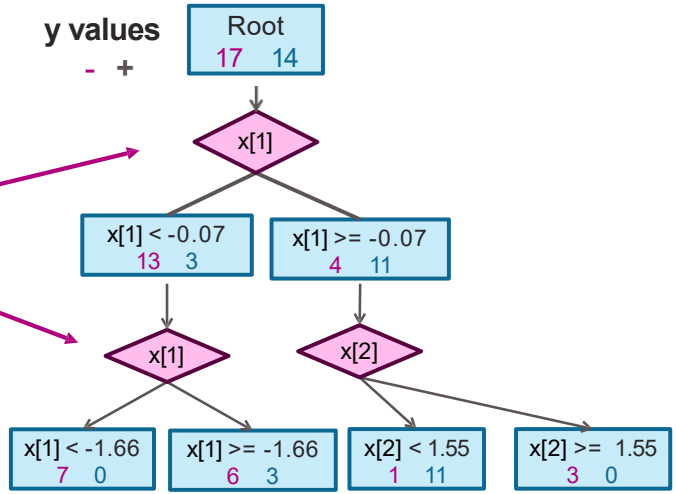


Credit to Stanford CS 229

Decision Tree

Threshold split caveat

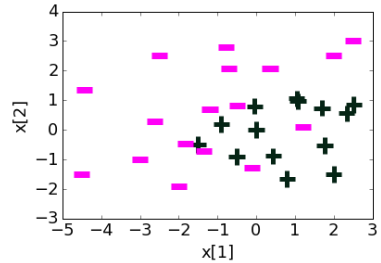
For threshold splits, same feature can be used multiple times



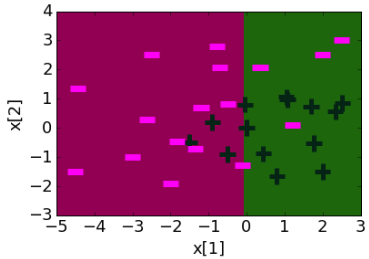
Credit to Stanford CS 229

Decision Tree

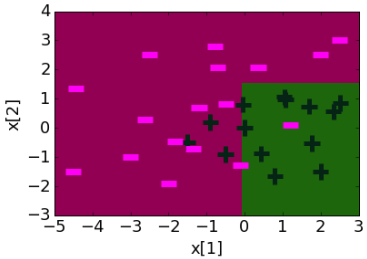
Decision boundaries



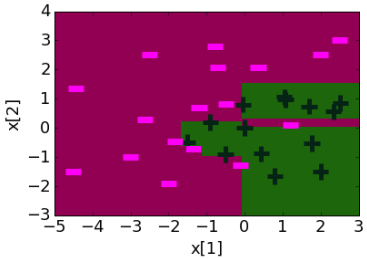
Depth 1



Depth 2



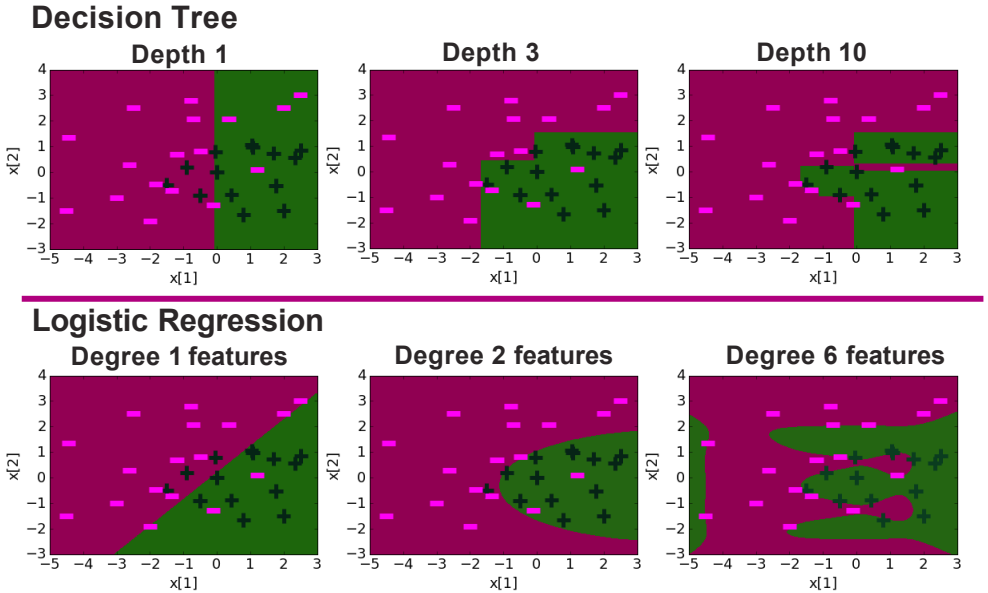
Depth 10



Credit to Stanford CS 229

Decision Tree

Comparing decision boundaries



Credit to Stanford CS 229

Decision Tree – Splitting Criteria

Buntine & Niblett (1992) compared 4 criteria (random, Gini, mutual information, Marshall) on 12 datasets

Medical Diagnosis Datasets: (4 of 12)

- **hypo:** data set of 3772 examples records expert opinion on possible hypo- thyroid conditions from 29 real and discrete attributes of the patient such as sex, age, taking of relevant drugs, and hormone readings taken from drug samples.
- **breast:** The classes are reoccurrence or non-reoccurrence of breast cancer sometime after an operation. There are nine attributes giving details about the original cancer nodes, position on the breast, and age, with multi-valued discrete and real values.
- **tumor:** examples of the location of a primary tumor
- **lymph**: from the lymphography domain in oncology. The classes are normal, metastases, malignant, and fibrosis, and there are nineteen attributes giving details about the lymphatics and lymph nodes

Table 1. Properties of the data sets

Data Set	Classes	Attr.s	Training Set	Test Set
hypo	4	29	1000	2772
breast	2	9	200	86
tumor	22	18	237	102
lymph	4	18	103	45
LED	10	7	200	1800
mush	2	22	200	7924
votes	2	17	200	235
votes1	2	16	200	235
iris	3	4	100	50
glass	7	9	100	114
xd6	2	10	200	400
pole	2	4	200	1647

Credit to CMU 10601

Table from Buntine & Niblett (1992)

Decision Tree – Splitting Criteria

Key Takeaway: GINI gain and Mutual Information are statistically indistinguishable!

Table 3. Error for different splitting rules (pruned trees).

	Splitting Rule			
Data Set	GINI	Info. Gain	Marsh.	Random
hypo	1.01 ± 0.29	0.95 ± 0.22	1.27 ± 0.47	7.44 ± 0.53
breast	28.66 ± 3.87	28.49 ± 4.28	27.15 ± 4.22	29.65 ± 4.97
tumor	60.88 ± 5.44	62.70 ± 3.89	61.62 ± 3.98	67.94 ± 5.68
lymph	24.44 ± 6.92	24.00 ± 6.87	24.33 ± 5.51	32.33 ± 11.25
LED	33.77 ± 3.06	32.89 ± 2.59	33.15 ± 4.02	38.18 ± 4.57
mush	1.44 ± 0.47	1.44 ± 0.47	7.31 ± 2.25	8.77 ± 4.65
votes	4.47 ± 0.95	4.57 ± 0.87	11.77 ± 3.95	12.40 ± 4.56
votes1	12.79 ± 1.48	13.04 ± 1.65	15.13 ± 2.89	15.62 ± 2.73
iris	5.00 ± 3.08	4.90 ± 3.08	5.50 ± 2.59	14.20 ± 6.77
glass	39.56 ± 6.20	50.57 ± 6.73	40.53 ± 6.41	53.20 ± 5.01
xd6	22.14 ± 3.23	22.17 ± 3.36	2.06 ± 3.37	31.86 ± 3.62
pole	15.43 ± 1.51	15.47 ± 0.88		

Info. Gain is another name for *mutual information*

Table from Buntine & Niblett (1992)

Decision Tree – Splitting Criteria

Table 4. Difference and significance of error for GINI splitting rule versus others.

	Splitting Rule		
Data Set	Info. Gain	Marsh.	Random
hypo	−0.06 (0.82)	0.26 (0.99)	6.43 (1.00)
breast	−0.17 (0.23)	−1.51 (0.94)	0.99 (0.72)
tumor	1.81 (0.84)	.74 (	7.86 (
lymph	−0.44 (0.83)	−0.1	
LED	0.12 (0.17)	0.3	
mush	0.00 (0.00)	5.8	
votes	0.11 (0.55)	7.3	
votes1	0.26 (0.47)	2.3	
iris	−0.10 (0.67)	0.5	
glass	1.01 (0.50)	0.9	
xd6	0.04 (0.11)	−0.0	
pole	0.03 (0.11)	−0.4	

Key Takeaway: GINI gain and Mutual Information are statistically indistinguishable!

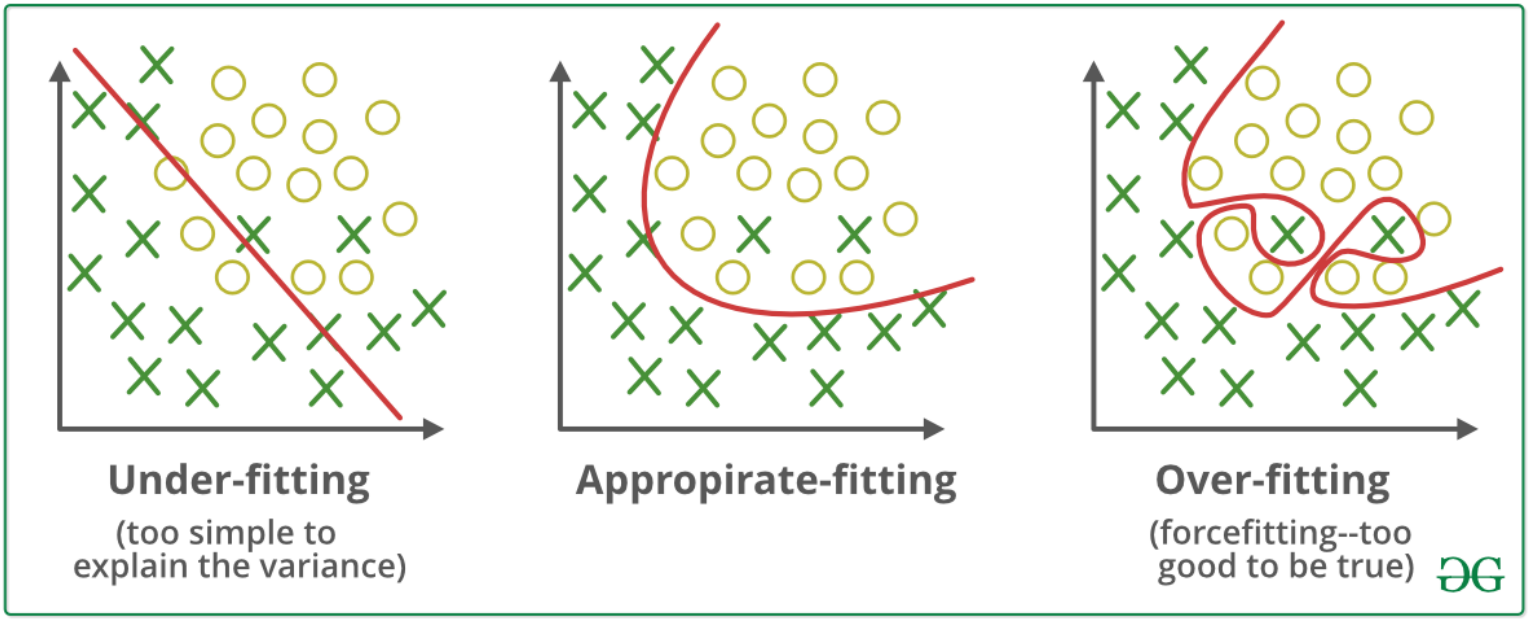
Results are of the form A.AA (B.BB) where:

1. A.AA is the **average difference in errors** between the two methods
2. B.BB is the **significance** of the difference according to a two-tailed **paired t-test**

Credit to CMU 10601

Table from Buntine & Niblett (1992)

Overfitting and Underfitting Revisited



We favor the case where the ML model appropriately depict the underlying data

DT Generalization – Overfitting Revisited

Question:

Which of the following would generalize best to unseen examples?

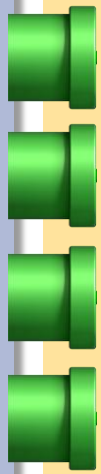
A. Small tree with low training accuracy

B. Large tree with low training accuracy

C. Small tree with high training accuracy

D. Large tree with high training accuracy

Answer:



Credit to CMU 10601

DT Generalization – Overfitting Revisited

Underfitting

- The model…
  - is too simple
  - is unable captures the trends in the data
  - exhibits too much bias
- *Example*: majority-vote classifier (i.e. depth-zero decision tree)
- *Example*: a toddler (that has not attended medical school) attempting to carry out medical diagnosis

Credit to CMU 10601

Overfitting

- The model…
  - is too complex
  - is fitting the noise in the data or fitting "outliers"
  - does not have enough bias
- *Example*: our "memorizer" algorithm responding to an irrelevant attribute
- *Example*: medical student who simply memorizes patient case studies, but does not understand how to apply knowledge to new patients

DT Generalization – Overfitting Revisited

- Given a hypothesis *h*, its…
  - …error rate over all training data: $error(h, D_{train})$
  - …error rate over all test data: $error(h, D_{test})$
  - …true error over all data: $error_{true}(h)$



In practice, $error_{true}(h)$ is **unknown**

- We say h overfits the training data if, for some alternative h′…

$error(h, D_{train}) < error(h', D_{train})$ but $error_{true}(h) > error_{true}(h')$

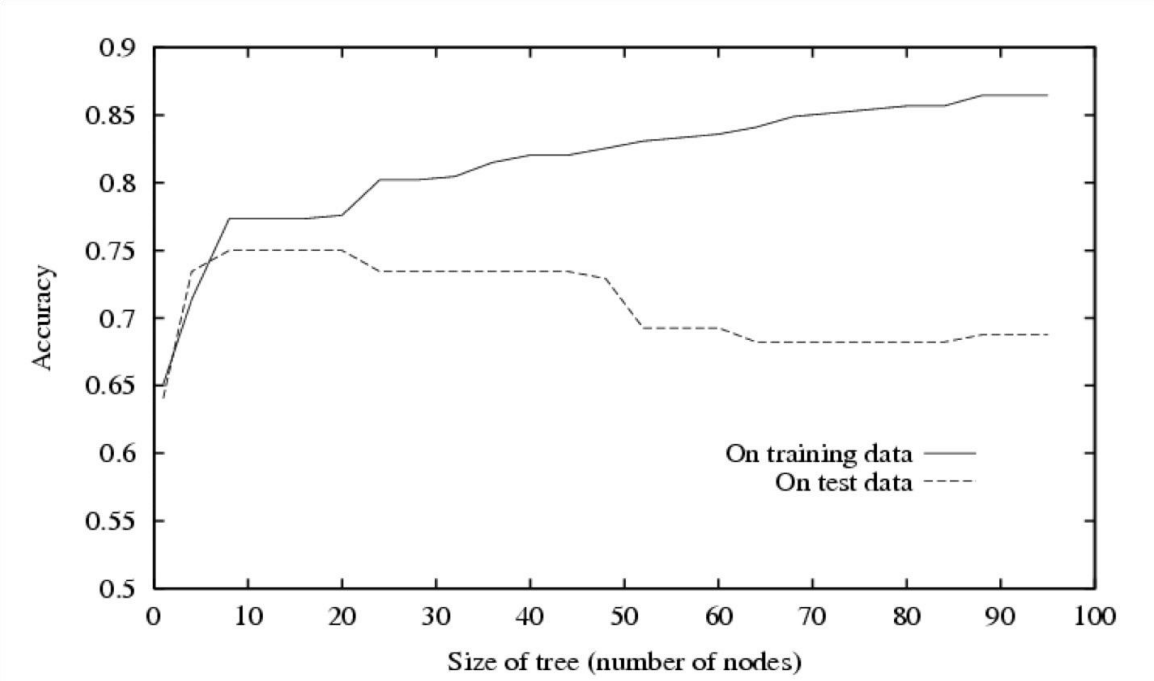
- Amount of overfitting =

$$error_{true}(h) - error(h, D_{train})$$

Credit to CMU 10601

Slide adapted from Tom Mitchell

DT Generalization – Overfitting Revisited



Credit to CMU 10601

Figure from Tom Mitchell

USC Viterbi School of Engineering

Underfitting and Overfitting

- Training error: model error on the training data
- Generalization error: model error on new data

Generalization error \ Training error	Low	High
Low	Good	Bug?
High	Overfitting	Underfitting

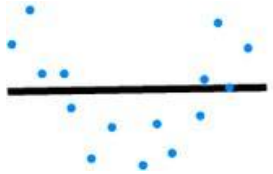
USC Viterbi School of Engineering

Alternative View: Data and Model Complexity

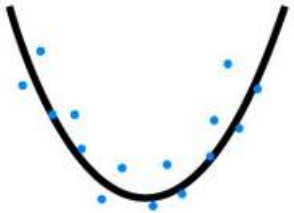
Data complexity

Model complexity	Low	High
Low	Normal	Underfitting
High	Overfitting	Normal

Underfitting



Normal



Overfitting



Image credit:hackernoon.com

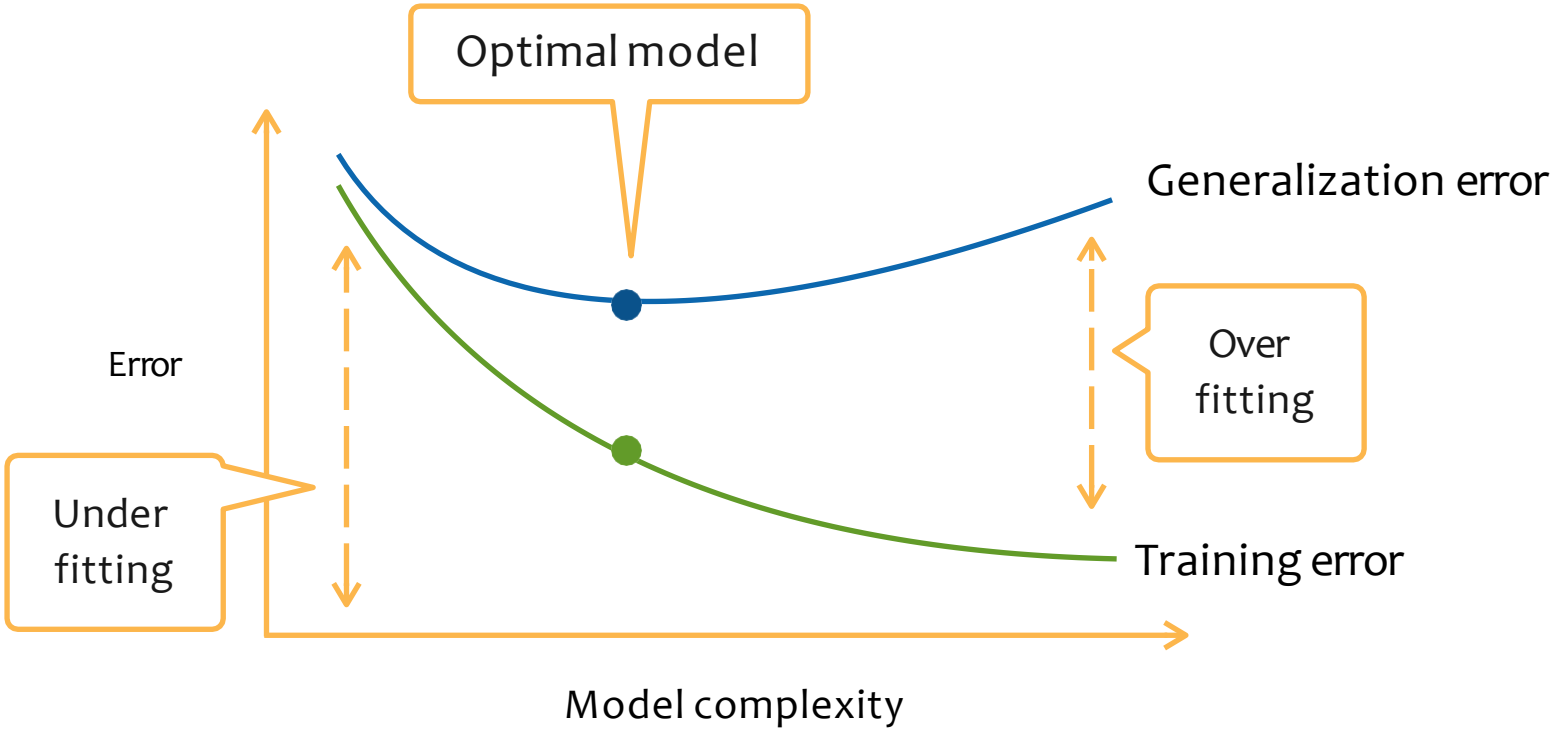
JSC Viterbi School of Engineering

Model Complexity

- The capacity of a set of function to fit data points
- In ML, model complexity usually refers to:
  - The number of learnable parameters
  - The value range for those parameters
- It's hard to compare between different types of ML models
  - E.g. trees vs neural network
- More precisely measure of complexity: VC dimension
  - VC dim for classification model:
    the maximum number of examples the model can shatter

USC Viterbi School of Engineering

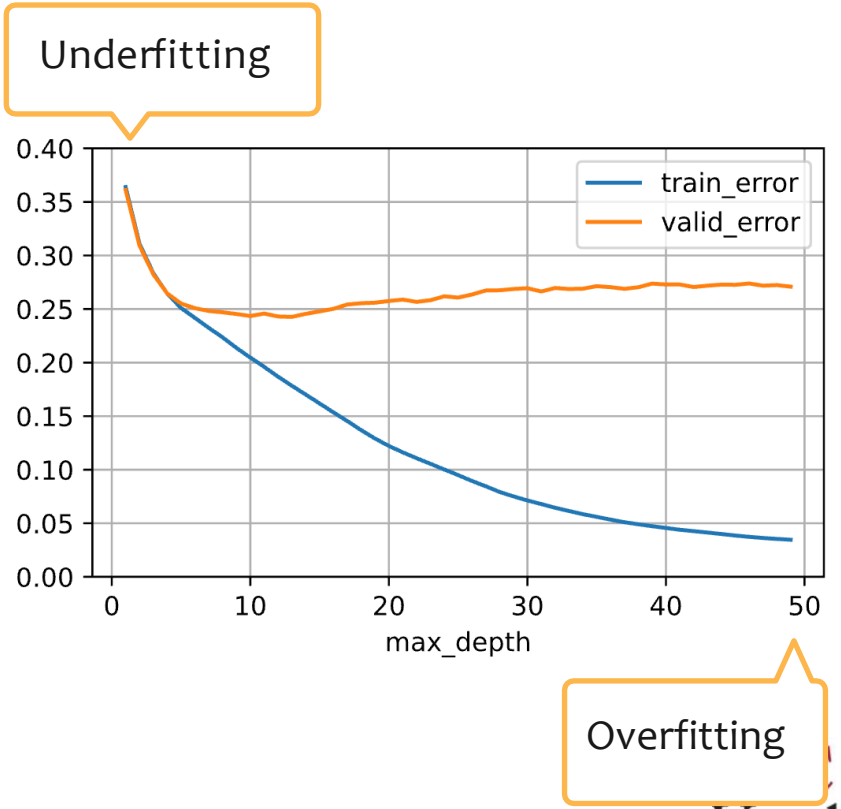
Model Complexity



USC Viterbi School of Engineering

Model Complexity Example: Decision Tree

- The tree size can be controlled by the number of levels
- Use scikit-learn **DecisionTreeRegressor(max_depth =n)** on house sales data



Viterbi School of Engineering

Data Complexity

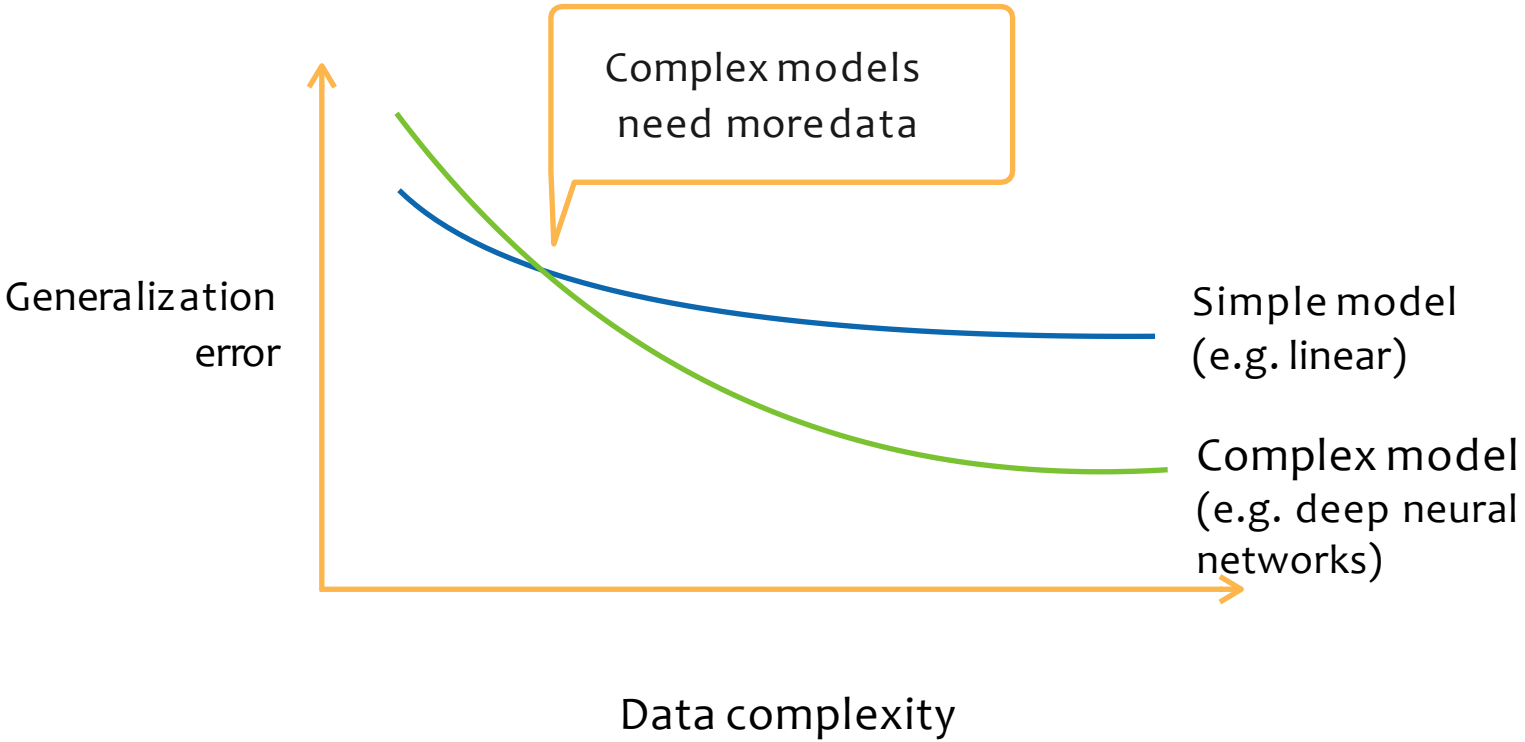
- Multiple factors matters
  - # of examples
  - # of features in each example
  - the separability of the classes
- Again, hard to compare among very different data
  - E.g a char vs a pixel
- More precisely, Kolmogorov complexity
  - A data is simple if it can be generated by a short program





USC Viterbi School of Engineering

Model Complexity vs Data Complexity



USC Viterbi School of Engineering

Model Selection

- Pick a model with a proper complexity for your data
  - Minimize the generalization error
  - Also consider business metrics
- Pick up a model family, then select proper hyper-parameters
  - Trees: #trees, maximal depths
  - Neural networks: architecture, depth (#layers), width (#hidden units), regularizations
  - **We will revisit the model selection later**

USC Viterbi School of Engineering

Summary

- **Model complexity:** the ability to fit various functions
- **Data complexity:** the richness of information
- **Model selection:** match model and data complexities

USC Viterbi School of Engineering

DT Generalization – Overfitting Revisited

For Decision Trees…

1. Do not grow tree beyond some **maximum depth**
2. Do not split if splitting criterion (e.g. mutual information) is **below some threshold**
3. Stop growing when the split is **not statistically significant**
4. Grow the entire tree, then **prune**

Credit to CMU 10601

DT Generalization – Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

Credit to CMU 10601

Memorization Is Not Overfitting

Memorization. A model can reproduce a particular training string.

Overfitting. Training fit improves while performance on new data worsens.

They can come apart. Fine-tuning Pythia, Llama3, and Mistral from 1.4B to 70B parameters, memorization rose sharply in the first few epochs, often before validation perplexity was best.

One number is not enough. Validation error can improve while verbatim leakage grows.

When privacy or copyright matters, track held-out performance and memorization together. A memorization-aware regularizer cut memorization by up to 40% across the model families tested.

When the Test Set Was Seen

The old rule. A final test item must be unseen, and used once.

A 2024 audit. Over 15 models across six multiple-choice QA benchmarks showed contamination from 1% to 45%.

It does not always inflate scores. Contaminated subsets showed boosts up to 14% on C-Eval and 7% on HellaSwag, but only a minimal increase on MMLU.

Patching the benchmark does not fix it. Across 10 models, 5 benchmarks, and 20 mitigation strategies, none balanced fidelity against contamination resistance.

So use validation or cross-validation for every choice you make, and keep the final test private. Rewording a leaked question is not a reset.

Linear Model Regularization: L1 and L2

What if $(\mathbf{A}^T\mathbf{A})$ is not invertible ?

r equations , p unknowns – underdetermined system of linear equations

many feasible solutions Need to constrain solution further

e.g. bias solution to "small" values of β (small changes in input don't translate to large changes in output)

$$\widehat{\beta}_{\text{MAP}} = \arg\min_{\beta} \sum_{i=1}^{n} (Y_i - X_i\beta)^2 + \lambda\|\beta\|_2^2$$

Ridge Regression (l2 penalty)

$$\widehat{\beta}_{\text{MAP}} = \arg\min_{\beta} \sum_{i=1}^{n} (Y_i - X_i\beta)^2 + \lambda\|\beta\|_1$$

Lasso

(l1 penalty)

Naming: the L2 penalty is called ridge in least squares, a Tikhonov special case. It matches weight decay under plain SGD, but not under the decoupled form in AdamW.

Credit to CMU 10601

Decision Tree in the Wild

- DTs are one of the most popular classification methods for practical applications
  - Reason #1: The learned representation is **easy to explain** a non-ML person
  - Reason #2: They are **efficient** in both computation and memory
- DTs can be applied to a wide variety of problems including **classification, regression, density estimation,** etc.

Credit to CMU 10601

Decision Tree in the Wild

Applications of DTs include…

- medicine, molecular biology, text classification, manufacturing, astronomy, agriculture, and many others

- **Decision Forests** learn many DTs from random subsets of features; the result is a very powerful example of an **ensemble method**

Credit to CMU 10601

Decision Tree Summary

What you can do now

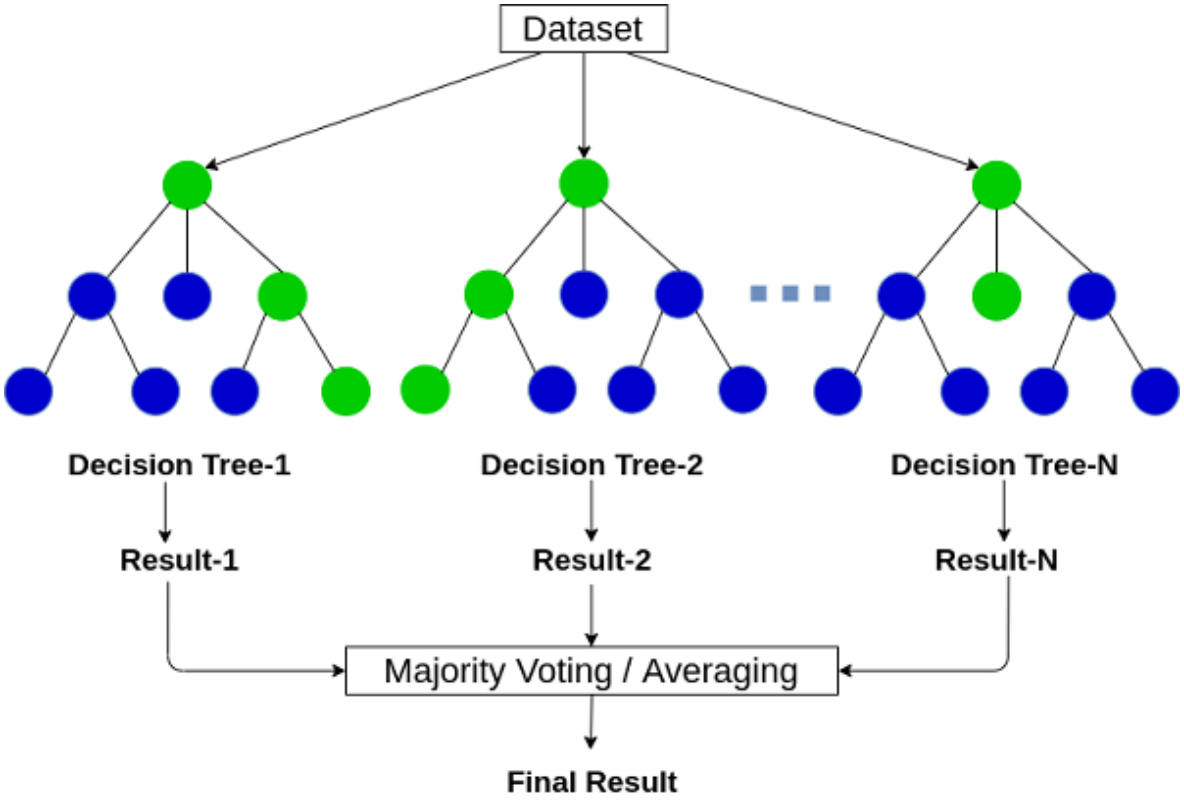
- Define a decision tree classifier
- Interpret the output of decision trees
- Learn a decision tree classifier using greedy algorithm
- Traverse a decision tree to make predictions
  - Majority class predictions
- Tackle continuous and discrete features
- Understand how to control overfitting and underfitting in DT

Credit to Stanford CS 229

Classical ML Algorithms

Ensembles

From A Decision Tree to Decision Forests



Bias, Variance, and Refit Stability

- Bias: the error left when the fitted rule is too simple for the pattern.
- Variance: how much predictions move when the training sample changes.
- Measure it without algebra: resample, refit, compare predictions at the same input.
- A single deep tree moves a lot. Averaging many of them moves less.
- That is the whole argument for the forest, stated before the mechanism.

Bootstrap AGGregatING: The Mechanism

- Bagging: draw n rows with replacement, fit one tree, repeat.
- Each sample holds about 63% unique rows, since $1 - (1 - 1/n)^n \rightarrow 1 - 1/e$.
- The rows left out are that tree's out-of-bag set, free validation data.
- Aggregate by voting for classification, averaging for regression.
- A random forest is bagging plus a random feature subset at every node.

Why Tree Count Differs From max_depth

- max_depth changes each tree's capacity; n_estimators changes how many are averaged.
- With tree settings fixed, adding trees approaches a limiting forest vote.
- So more trees converge; they do not overfit the way deeper trees do.
- Dispersion across trees is a heuristic indicator of ensemble disagreement.
- Calibrated uncertainty needs the infinitesimal jackknife, quantile forests, or conformal calibration.

Boosting: Fit Residuals Sequentially

- Fit h_m to the current residual, then add it: $F_m = F_{m-1} + h_m$.
- Squared loss, $F_0 = 0$, shrinkage 1. Stumps split after x = 4, 2, 4.
- The residuals you just computed are the negative gradient of squared loss.

x	y	h_1	r_1	h_2	r_2	h_3	r_3
1	-8	-4	-4	-4	0	1	-1
2	-8	-4	-4	-4	0	1	-1
3	0	-4	4	2	2	1	1
4	0	-4	4	2	2	1	1
5	8	8	0	2	-2	-2	0
6	8	8	0	2	-2	-2	0

One Gradient-Boosting Family, Three Implementations

- They differ in engineering and defaults, inside one algorithm family.
- XGBoost: regularized objective with a second-order loss approximation.
- LightGBM: histogram-binned features, best-first leaf-wise growth.
- CatBoost: ordered target statistics for categories, plus ordered boosting.
- An untuned baseline is not a baseline.

Tabular Baselines as of August 2026

- TabPFN-3 reports that one forward pass beats tuned and ensembled baselines.
- TabArena reports that boosted trees are still strong contenders, and that deep learning catches up under larger budgets with ensembling.
- Both can hold. They differ in time budget, ensembling, and data regime.
- Policy here: evaluate a tuned GBDT and TabPFN-3 under one protocol.

Credit to TabPFN-3 (2026) and TabArena (NeurIPS 2025)

Ensemble Summary

What you can do now

- Explain why averaging many trees lowers variance
- Describe bagging: resample with replacement, fit, aggregate
- Say what random forests add on top of bagging
- Name the knob that converges and the knob that overfits

A Detour, Not a Continuation

Tree Search Inside a Language Model

Why These Slides Sit Here

An analogy

- Decision trees are built greedily and never revisit a split.
- The next pages ask what happens when a search can revisit.
- The connection is an analogy about search, not a property of trees.
- Nothing here is required for the midterm; it explains why the idea persists.

How do we solve complicated problems

Decompose the problem into **intermediate steps** and solve each before giving the final answer

“After Jane gives 2 flowers to her mom she has 10 . . . then after she gives 3 to her dad she will have 7 . . . so the answer is 7.”

The paper shows sufficiently large LMs can generate chains of thought if demonstrations of chain-of-thought reasoning are provided as few-shot prompting.

Wei, Jason, et al. "Chain-of-thought prompting elicits reasoning in large language models." *NeurIPS.*

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ✗

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9. ✓

Wei, Jason, et al. "Chain-of-thought prompting elicits reasoning in large language models." *NeurIPS.*

Benefits

1. Allows models to **decompose** multi-step problems into intermediate steps so additional computation can be allocated to problems that require more reasoning steps.
2. Interpretable method to understand model's response
3. Potentially applicable to any task that require multi-step approach.
4. Can be induced simply by **including examples of chain of thought** sequences while prompting.

Wei, Jason, et al. "Chain-of-thought prompting elicits reasoning in large language models." *NeurIPS.*

Sensitivity to prompt engineering

Chain of thought is robust to:

1. Different annotators writing prompts
2. Annotators without machine learning background writing prompts
3. Different types, ordering and number of prompt examples
4. Different language models (LaMDA, GPT-3, and PaLM)

However Prompt Engineering improved performance significantly in many cases.

Wei, Jason, et al. "Chain-of-thought prompting elicits reasoning in large language models." *NeurIPS.*

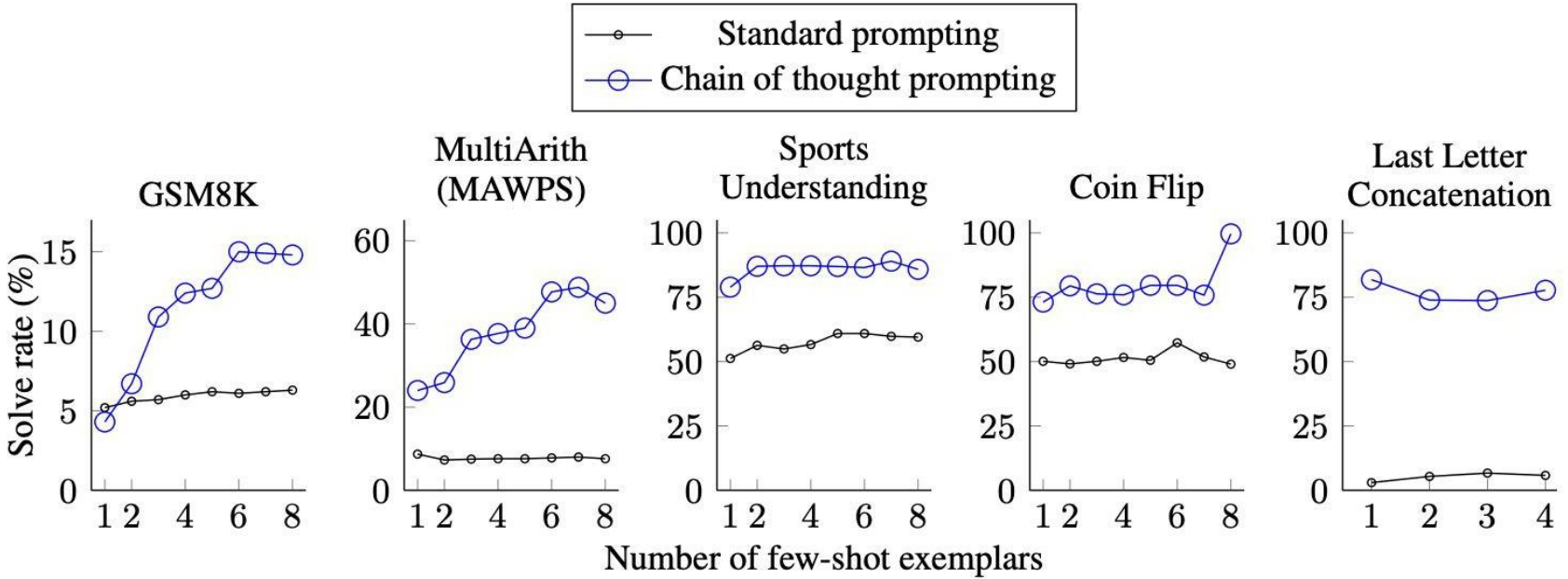


Figure 11: The improvement of chain of thought prompting over standard prompting appears robust to varying the number of few-shot exemplars in the prompt.

Results

Three key takeaways:

- chain-of-thought prompting is an emergent ability of **model scale**;
- chain-of-thought prompting has larger performance gains for **more-complicated problems**;
- chain-of-thought prompting via GPT-3 175B and PaLM 540B **compares favorably** to prior state of the art.

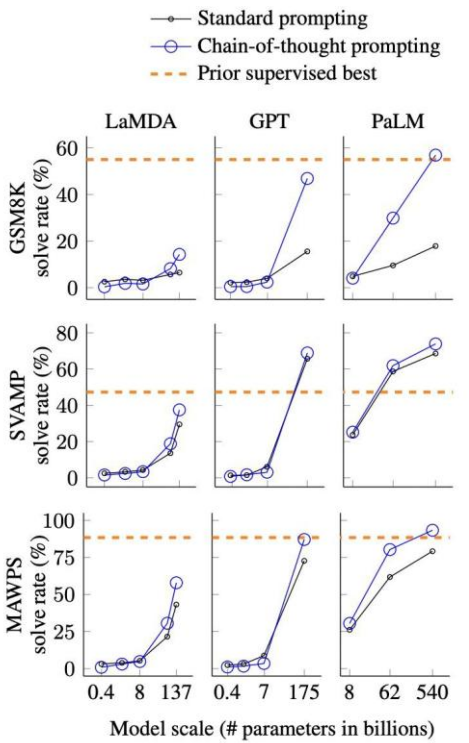


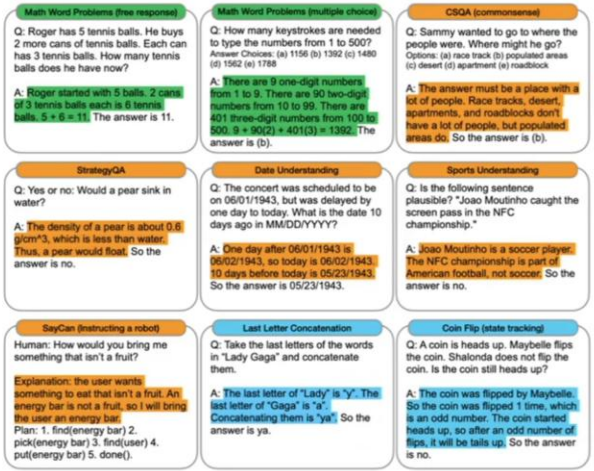
Figure 4: Chain-of-thought prompting enables large language models to solve challenging math problems. Notably, chain-of-thought reasoning is an emergent ability of increasing model scale.

Wei, Jason, et al. "Chain-of-thought prompting elicits reasoning in large language models." *NeurIPS.*

Tree of Thoughts in Large Language Models

How Language Models (LMs) Solve Problems

- LMs were invented to generate text
- LMs are used to solve problems by generating "reasoning" processes (e.g., chain-of-thought)
- Limitations of autoregressive generation:
  - **Token-level decision**:
    No global exploration
  - **One-by-one, left-to-right**:
    No local lookahead / backtrack



(Wei et al., 2022)

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*.

Tree of Thoughts in Large Language Models

Game of 24

Use numbers and basic arithmetic operations (+ - * /) to obtain 24. Each step, you are only allowed to choose two of the remaining numbers to obtain a new number.
Input: 4 4 6 8
Steps: 4 + 8 = 12 (left: 4 6 12); 6 - 4 = 2 (left: 2 12); 2 * 12 = 24 (left: 24)
Answer: (6 - 4) * (4 + 8) = 24
{more Input-Steps-Answer examples...}
Input: 4 5 6 10

Steps: 5 - 4 = 1 (left: 1 6 10); 10 - 1 = 9 (left: 6 9); 6 * 4 = 24 (left: 24)
Answer: (10 - (5 - 4)) * 6 = 24

Method	Success
IO prompt	7.3%
CoT prompt	4.0%
CoT-SC (k=100)	9.0%

Why hard even for GPT-4 + CoT?

Token decision with no lookahead: really hard to tell if first token "5" is good or bad

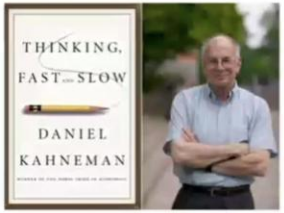
No backtrack: First couple tokens ("5 -") wrong, game over...
(In fact, 60% CoT samples fail after first step)

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*.

Tree of Thoughts in Large Language Models

How Humans Solve Problems

System 1	System 2
Fast	Slow
Subconscious	Conscious
Automatic	Effortful
Everyday Decisions	Complex Decisions
Error Prone	Reliable



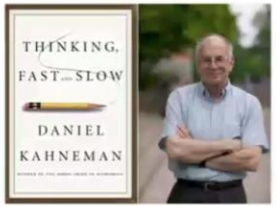
- Autoregressive generation is analogous to “System 1”

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*.

Tree of Thoughts in Large Language Models

How Humans Solve Problems





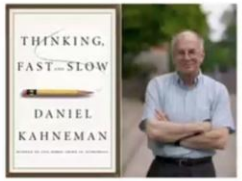
- Autoregressive generation is analogous to “System 1”
- Can be augmented by “System 2” control structures

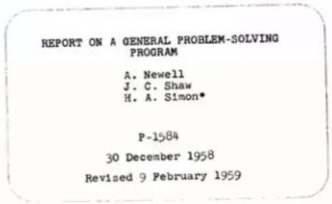
Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*.

Tree of Thoughts in Large Language Models

How Humans Solve Problems






REPORT ON A GENERAL PROBLEM-SOLVING PROGRAM

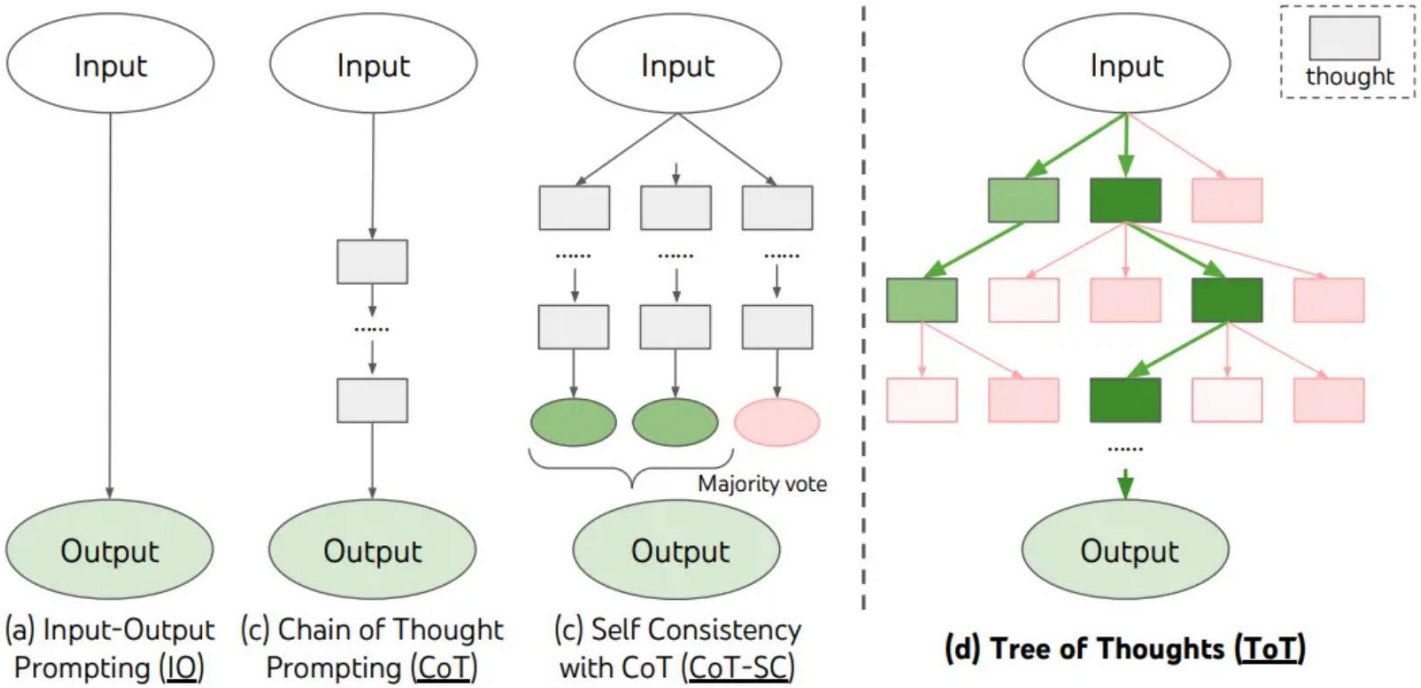
A. Newell
J. C. Shaw
H. A. Simon*

P-1584
30 December 1958
Revised 9 February 1959

- Autoregressive generation is analogous to "System 1"
- Can be augmented by "System 2" control structures
  - Tree search

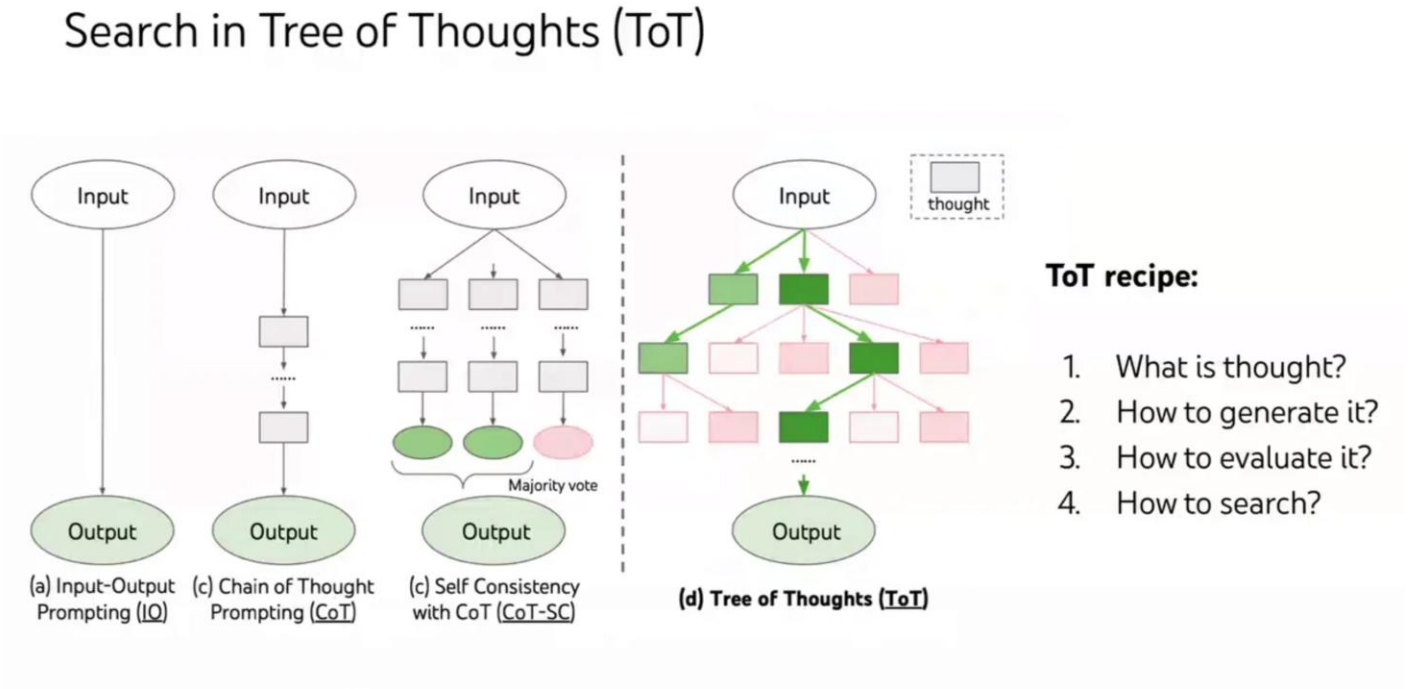
Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*.

Tree of Thoughts in Large Language Models



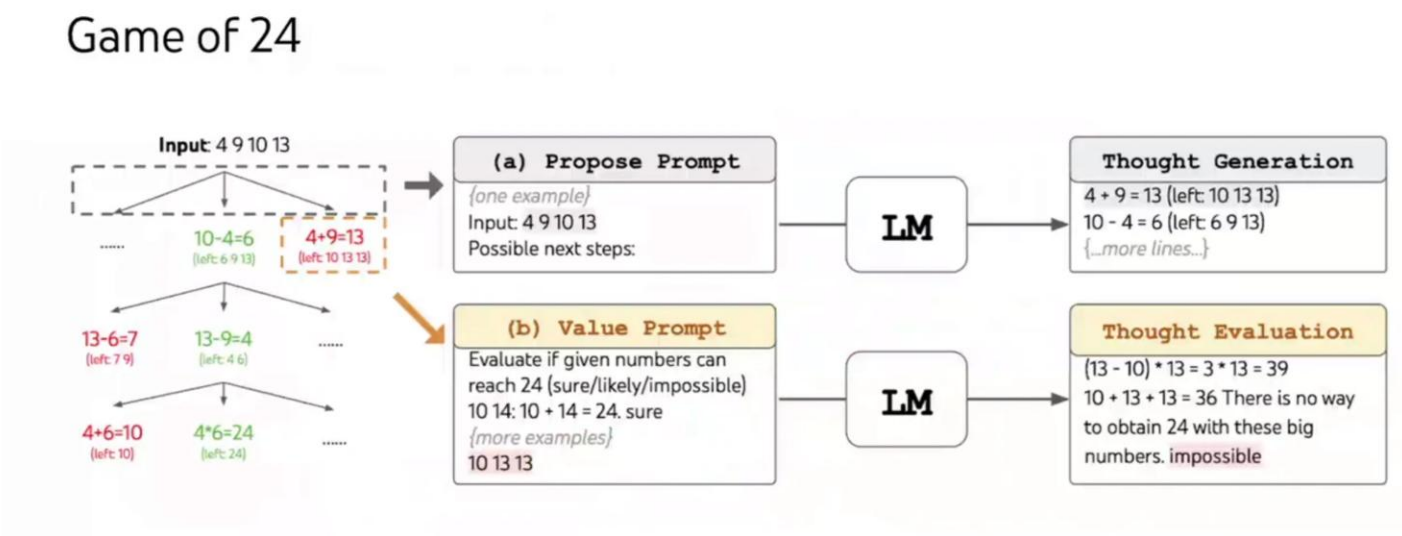
Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*.

Tree of Thoughts in Large Language Models



Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*.

Tree of Thoughts in Large Language Models



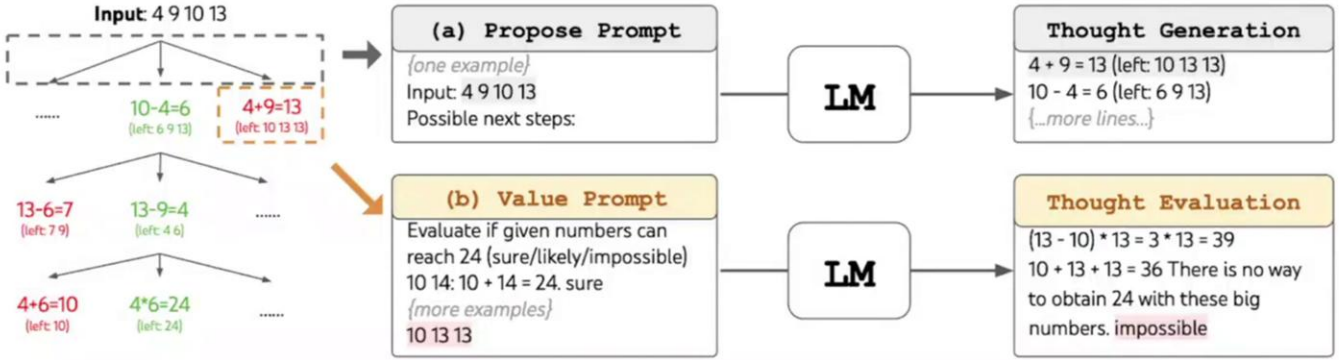
1. Thought: each intermediate equation
2. Generate thoughts: **propose**
3. Evaluate thoughts: **value**
4. Search algorithm: **BFS** (depth=4, breadth<=5)

- Number of candidates
- Number of steps/thought
- (breadth first search)

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*.

Tree of Thoughts in Large Language Models

Game of 24



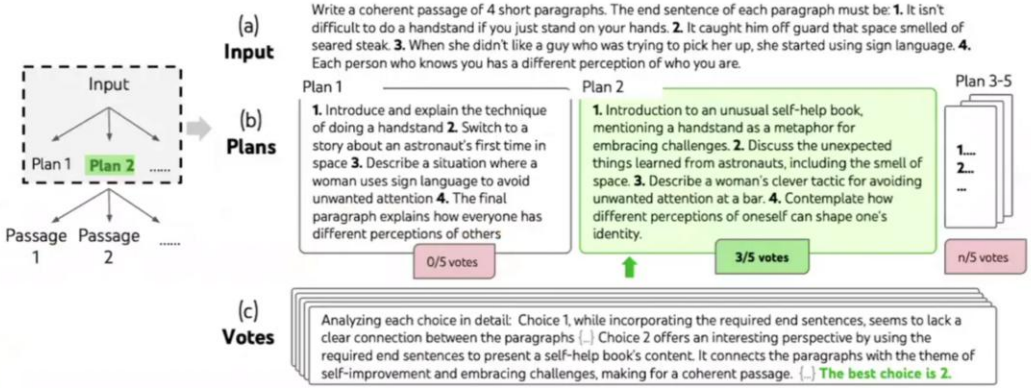
1. Thought: each intermediate equation
2. Generate thoughts: **propose**
3. Evaluate thoughts: **value**
4. Search algorithm: **BFS** (depth=4, breadth<=5)

Method	Success
IO prompt	7.3%
CoT prompt	4.0%
CoT-SC (k=100)	9.0%
ToT (ours) (b=1)	45%
ToT (ours) (b=5)	**74%**

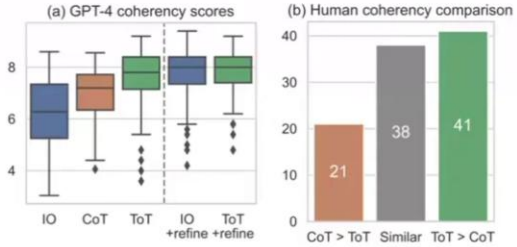
Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*.

Tree of Thoughts in Large Language Models

Creative Writing



1. Thought: just one intermediate plan
2. Generate thoughts: **sample**
3. Evaluate thoughts: **vote**
4. Search algorithm: **BFS** (depth=2, breadth=5)

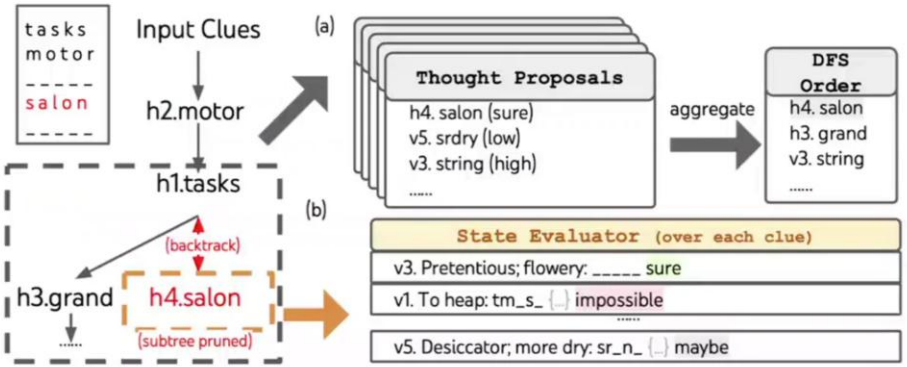


Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*.

USC Viterbi School of Engineering

Tree of Thoughts in Large Language Models

Mini Crosswords



Method	Success Rate (%) Letter	Word	Game
IO	38.7	14	0
CoT	40.6	15.6	1
ToT (ours)	**78**	**60**	**20**
+best state	82.4	67.5	35
-prune	65.4	41.5	5
-backtrack	54.6	20	5

1. Thought: each clue word
2. Generate thoughts: **propose**
3. Evaluate thoughts: **value**
4. Search algorithm: **DFS** (depth=5-10)

Importance of ...

- Backtrack
- Subtree pruning

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*.

Tree of Thoughts in Large Language Models

Tasks

	Game of 24	Creative Writing	5x5 Crosswords
Input	4 numbers (4 9 10 13)	4 random sentences	10 clues (h1. presented;..)
Output	An equation to reach 24 (13-9)*(10-4)=24	A passage of 4 paragraphs ending in the 4 sentences	5x5 letters: SHOWN; WIRRA; AVAIL; ...
Thoughts	3 intermediate equations (13-9=4 (left 4,4,10); 10-4=6 (left 4,6); 4*6=24)	A short writing plan (1. Introduce a book that connects...)	Words to fill in for clues: (h1. shown; v5. naled; ...)
#ToT steps	3	1	5-10 (variable)

- **Modularity** to use different generation, evaluation, and search strategies
- **Flexibility** to solve diverse problems under different resource constraints
- **Significant performance gain** without re-training LMs

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*.

USC Viterbi School of Engineering

Tree of Thoughts in Large Language Models

Summary

- Language model + Tree search complement each other (like System 1/2)
  - LM reasoning provides flexible and versatile heuristics for search
  - Search augments deliberation and exploration for LM reasoning

- Future directions:
  - Explore **harder problems** that challenge SoTA LMs like GPT-4
  - Develop **better LMs** for self-evaluation
  - Apply **classical insights** for frontier LM research

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*.

2026: The Tree Moved Inside

One path. A trained reasoning model continues, checks, revises, and sometimes backtracks before it answers.

Many leaves. Generate several complete answers, then vote on them or verify them.

Partial tree. Branch, score, and prune before completion. Tree of Thoughts is in this category, and this category is still active in 2026.

Products now expose the budget. GPT-5.6, Claude Opus 4.6, Gemini 3 Deep Think, and DeepSeek V4 all ship an adjustable reasoning control.

Value-guided search with a 14B generator, a 1.5B checker, and 64 generations scored 61.0% across four math contests; full 671B DeepSeek-R1 scored 60.8% in one pass. This is not compute matched.

Classical ML Algorithms

Model Metrics

Model Metrics

- Loss measures how good the model in predicting the outcome in supervised learning
- Other metrics to evaluate the model performance
  - Model specific: e.g., accuracy for classification, mAP for object detection
  - Business specific: e.g., revenue, inference latency
- We select models by multiple metrics
  - Just like how you choose cars



USC Viterbi School of Engineering

Metrics for Binary Classification

- Accuracy: # correct predictions / # examples

```
sum(y == y_hat) / y.size
```

- Precision: # True positive / # (True positive + False positive)

```
sum((y_hat == 1) & (y == 1)) / sum(y_hat == 1)
```

- Recall: # True positive / # Positive examples

```
sum((y_hat == 1) & (y == 1)) / sum(y == 1)
```

- Be careful of division by 0
- One metric that balances precision and recall
  - F1: the harmonic mean of precision and recall: $2pr/(p + r)$

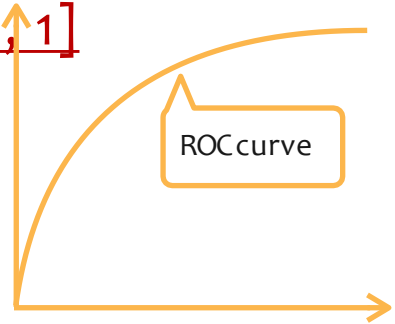
USC Viterbi School of Engineering

AUC-ROC

- Measures how well the model can separate the two classes
- Choose decision threshold θ, predict positive if $o \geq \theta$ else neg
- In the range [0, 1]

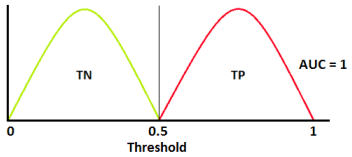
$$\text{TPR} = \frac{\#\text{ True Pos predictions}}{\#\text{ pos examples}}$$

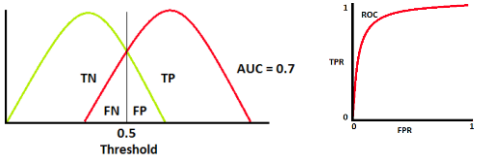
$$\text{FPR} = \frac{\#\text{ False Pos predictions}}{\#\text{ neg examples}}$$



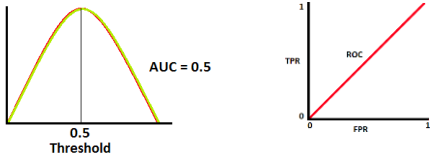
Distribution of output o

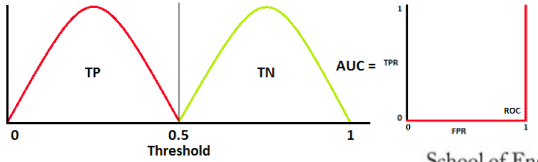
Ideal separation





Non separable





bi School of Engineering

When ROC Hides False Alarms

- Take 10,000 scored cases with 1% positives, ranked 10, 20, ..., 1000.
- Cut at the top 1,000: recall 1.00, but precision only 0.10.
- ROC calls that FPR 0.091, because 900 false alarms sit among 9,900 negatives.
- ROC-AUC reads 0.954, but PR-AUC reads 0.098 against a chance line of 0.01.
- Chance is 0.50 for ROC-AUC and the positive rate for PR-AUC. Report both here.

The Rare Positive Is a Safety Incident

A guardrail is one binary decision per exchange: pass, or escalate for a closer look.

ChatGPT Agent, July 2025. The first-stage classifier reports recall 0.960, precision 0.737, F1 0.834. OpenAI says it picked a threshold with at least 0.95 recall and accepted the lower precision.

Constitutional Classifiers++, January 2026. In a one-month shadow run on live traffic, 5.5% of exchanges escalated and 0.05% were finally flagged.

That is a 1-in-2000 positive rate. Accuracy and ROC-AUC stay comfortable while the false alarms decide whether the system can ship.

Neither report gives a PR curve on one labeled production sample. Attack tests and ordinary traffic are different datasets, and they do not combine into one confusion matrix.

Classical ML Algorithms

k Nearest Neighbor



USC Viterbi School of Engineering

Nearest Neighbor Algorithm

def train($\mathcal{D}$):

Store $\mathcal{D}$

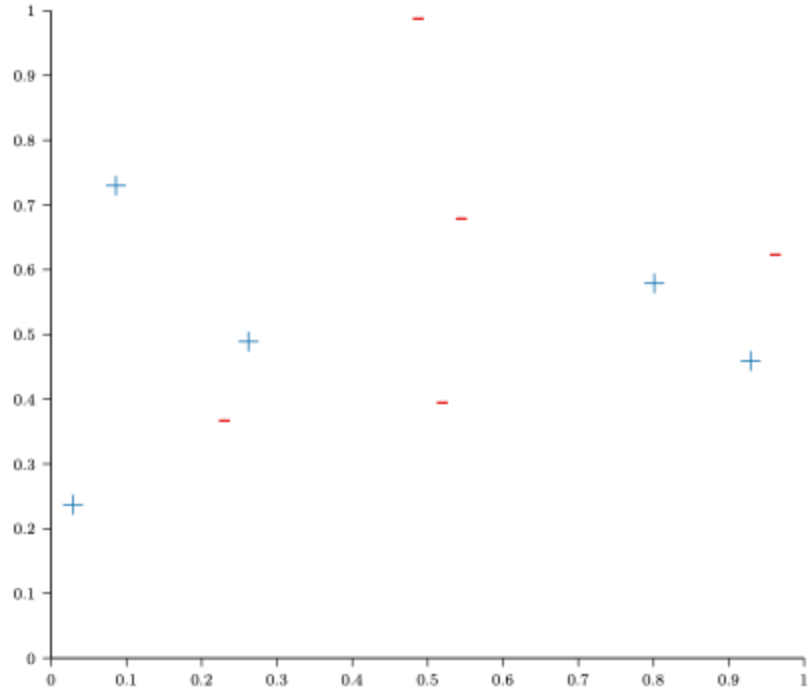
def h(x'):

Let $x^{(i)}$ = the point in $\mathcal{D}$ that is nearest to x'

return $y^{(i)}$

Credit to CMU 10601

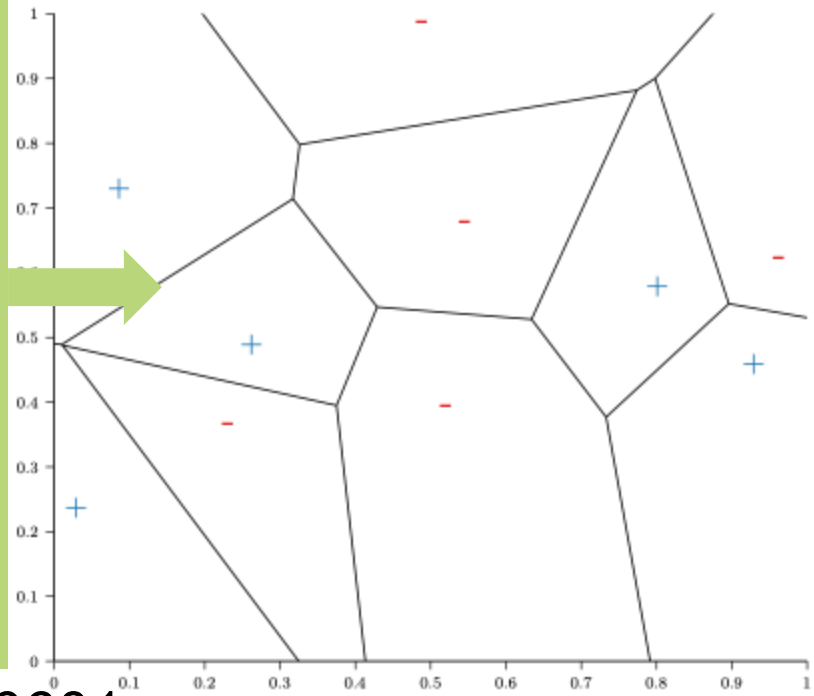
Nearest Neighbor Algorithm



Credit to CMU 10601

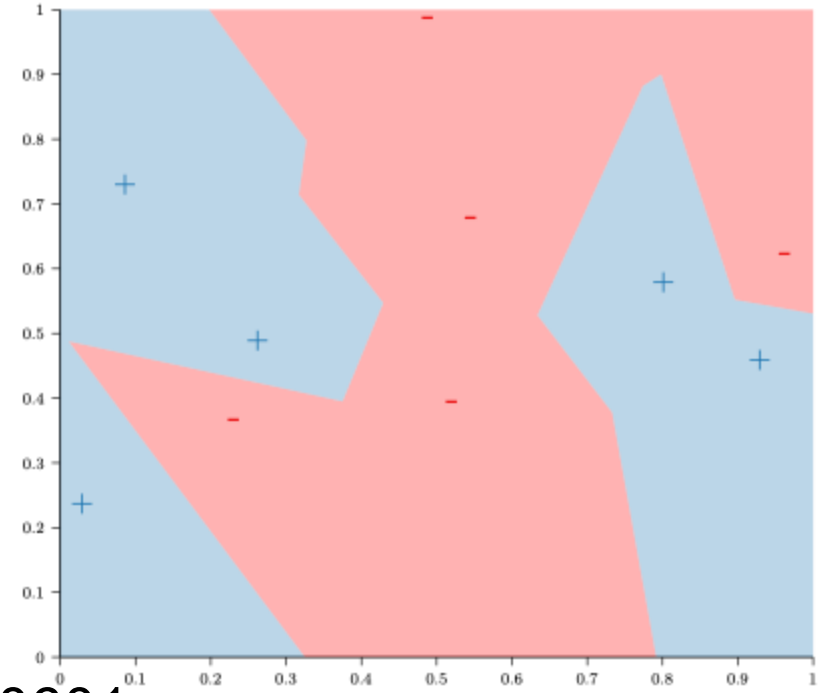
Nearest Neighbor Algorithm

- This is a **Voronoi diagram**
- Each **cell** contain one of our training examples
- **All points within a cell** are **closer** to that training example, than to any other training example
- **Points on the Voronoi line segments** are **equidistant** to one or more training examples



Credit to CMU 10601

Nearest Neighbor Decision Boundary



Credit to CMU 10601

Nearest Neighbor Algorithm

- Requires no training!
- Always has zero training error!
  - ***A data point is always its own nearest neighbor***

5
7

Credit to CMU 10601

k Nearest Neighbor Algorithm

def set_hyperparameters(k, d)**:**
    Store k
    Store $d(\cdot, \cdot)$

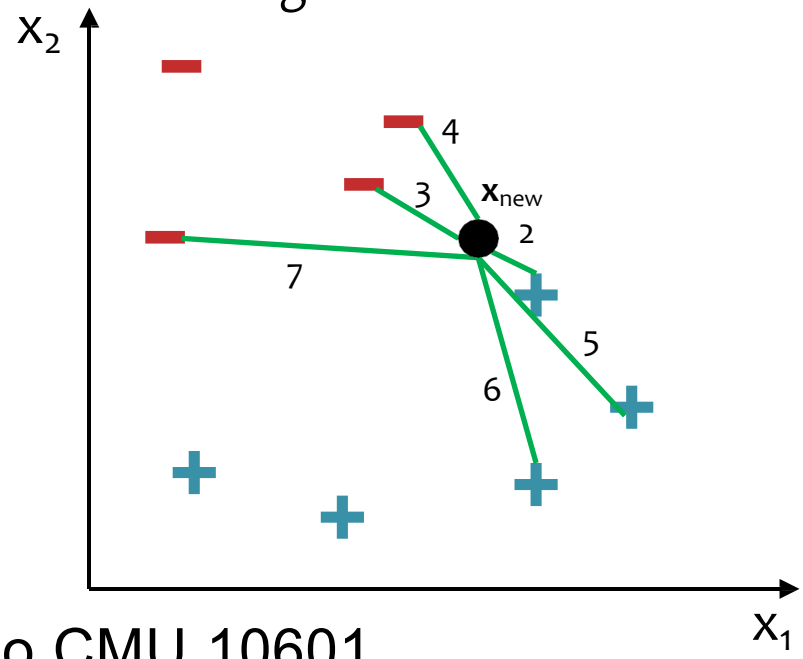
def train($\mathcal{D}$)**:**
    Store $\mathcal{D}$

def h(x')**:**
    Let S = the set of k points in $\mathcal{D}$ nearest to x'
        according to distance function
        d(**u**, **v**)
    Let v = majority_vote(S)
    return v

Credit to CMU 10601

k Nearest Neighbor Algorithm

Suppose we have the training dataset below.



How should we label the new point?

It depends on k:

if k=2, $h(\mathbf{x}_{new}) = +1$

if k=3, $h(\mathbf{x}_{new}) = -1$

if k=5, $h(\mathbf{x}_{new}) = +1$





Credit to CMU 10601

k Nearest Neighbor Algorithm

Distance Functions:

- KNN requires a **distance function**

$$d : \mathbb{R}^M \times \mathbb{R}^M \rightarrow \mathbb{R}$$

- The most common choice is **Euclidean distance**

$$d(\boldsymbol{u}, \boldsymbol{v}) = \sqrt{\sum_{m=1}^{M} (u_m - v_m)^2}$$

- But there are other choices (e.g. **Manhattan distance**)

$$d(\boldsymbol{u}, \boldsymbol{v}) = \sum_{m=1}^{M} |u_m - v_m|$$

Credit to CMU 10601

How Does k Change the Decision?

- What is the effect of k?
- Will a large k make the decision boundary coarse or smooth?
- How will that affect the overfitting and underfitting?

kNN on Fisher Iris Data

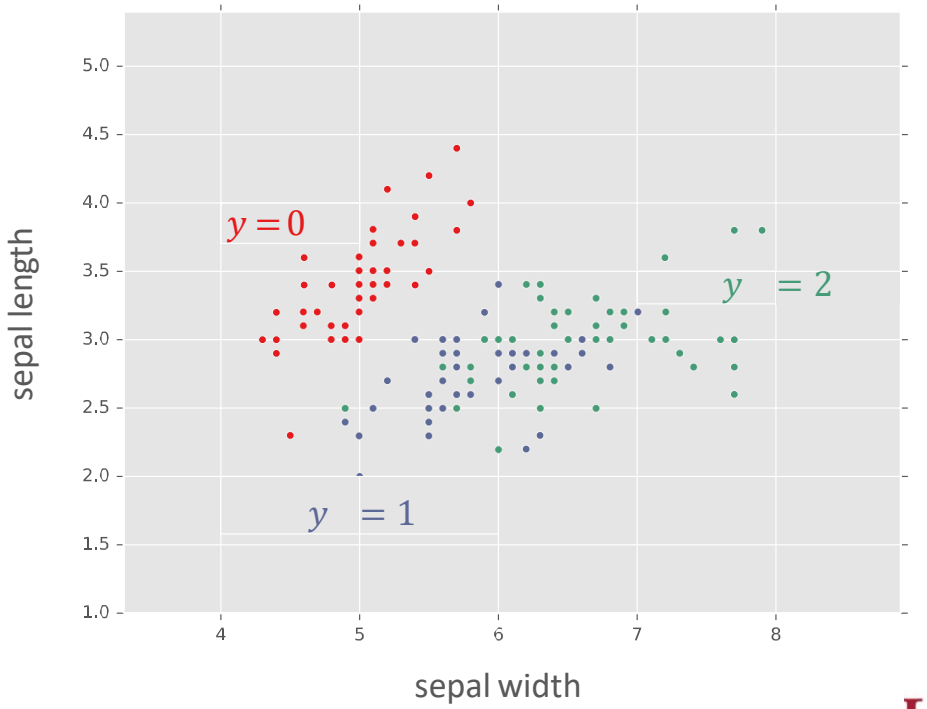


Figure courtesy of Matt Gormley

k=1

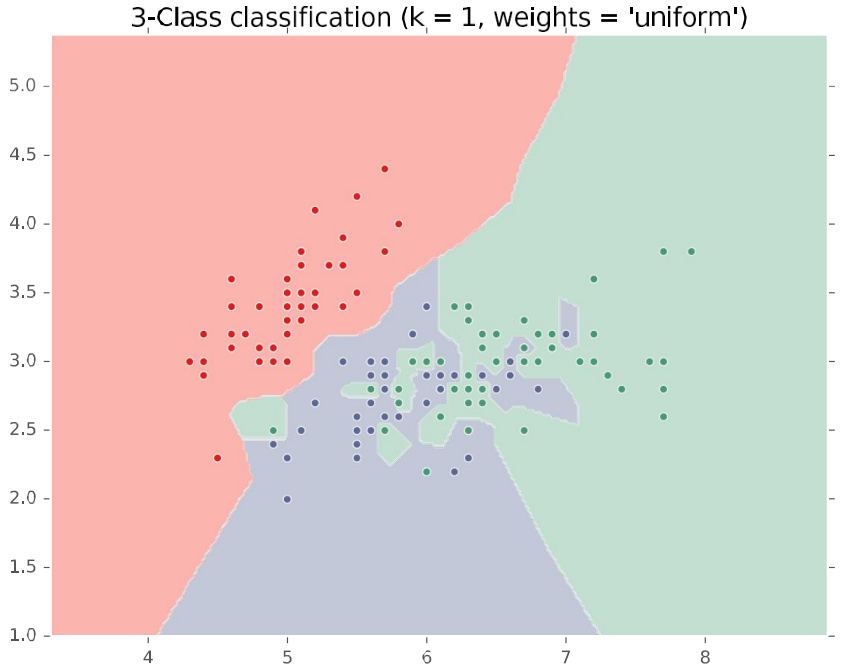


Figure courtesy of Matt Gormley

k=2

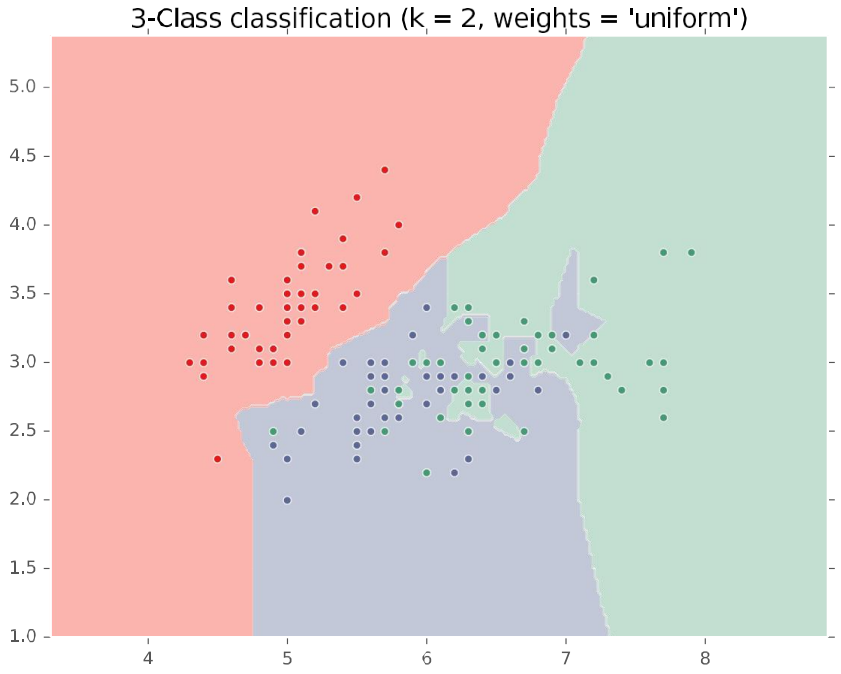


Figure courtesy of Matt Gormley

k=3

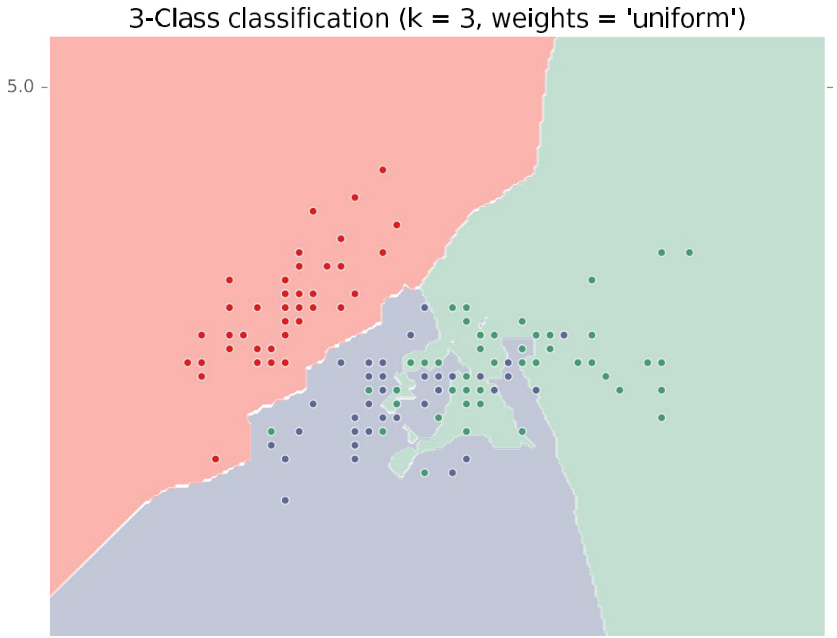


Figure courtesy of Matt Gormley

k=5

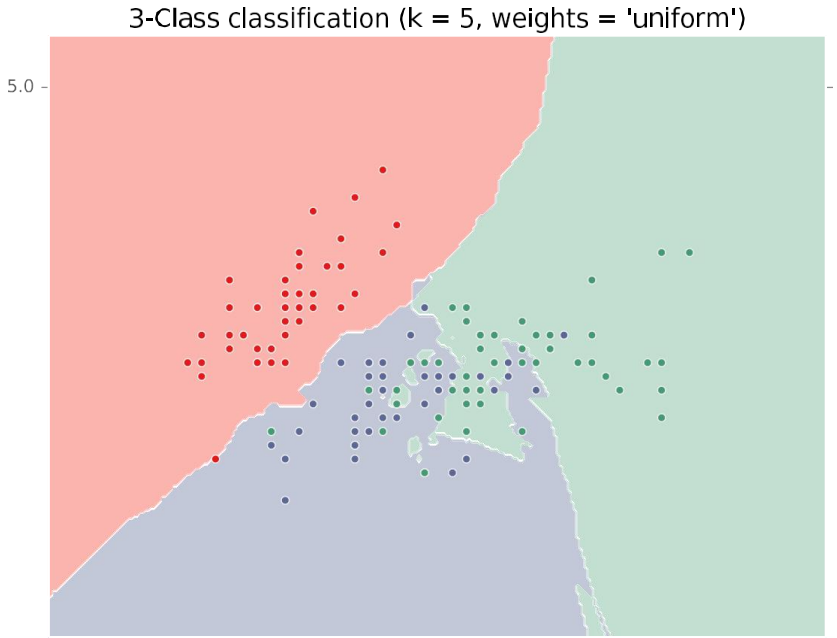


Figure courtesy of Matt Gormley

k=10

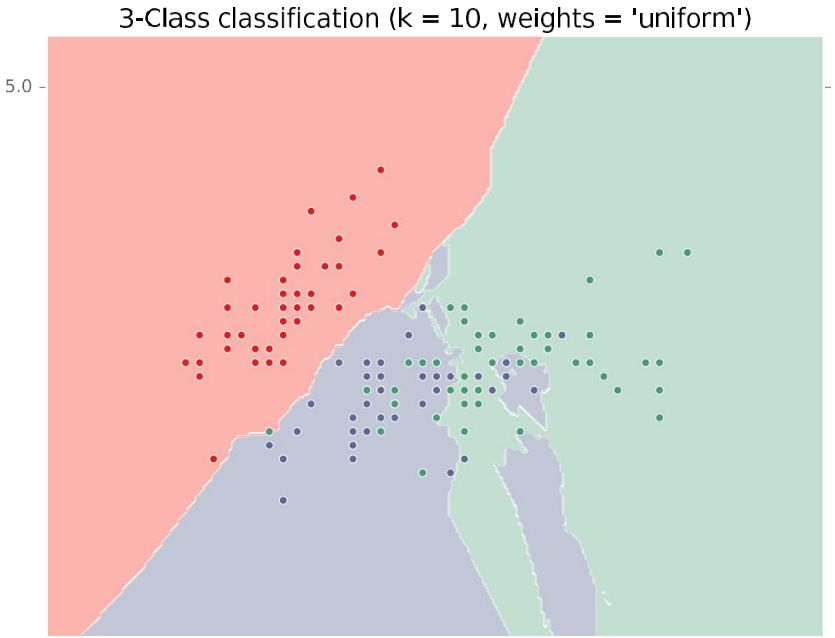


Figure courtesy of Matt Gormley

k=20

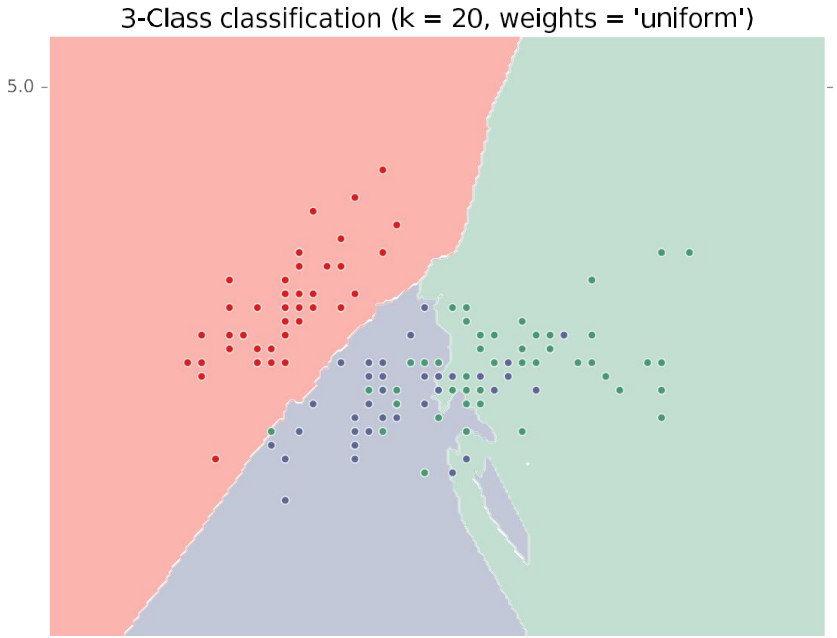


Figure courtesy of Matt Gormley

k=30

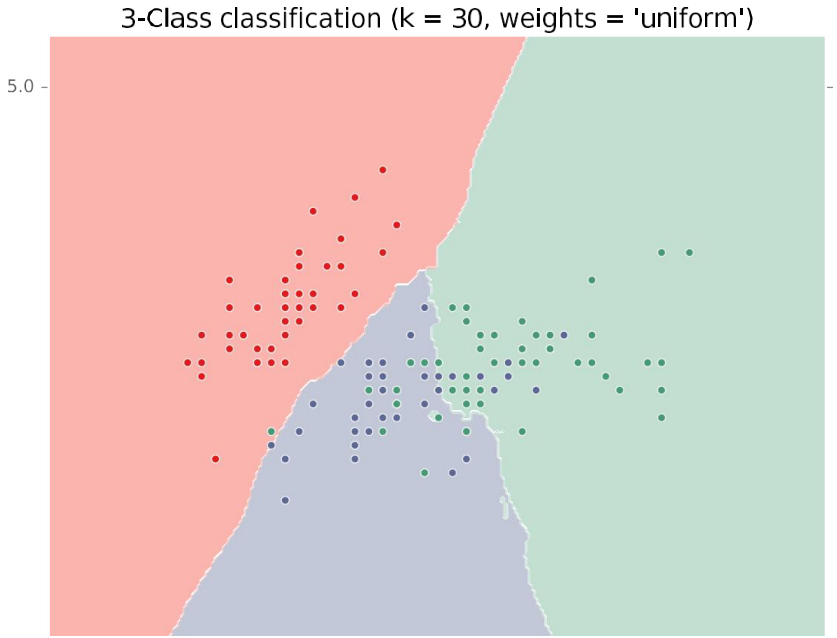
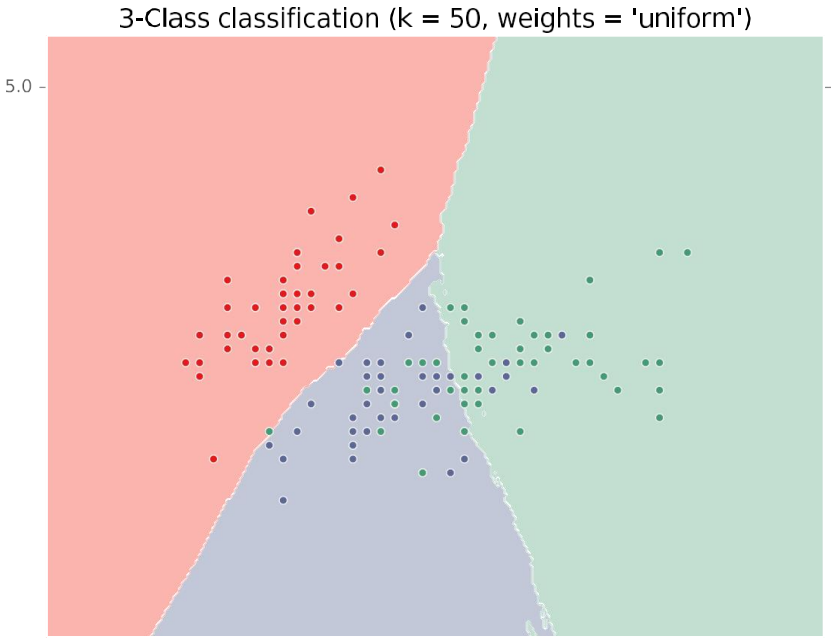
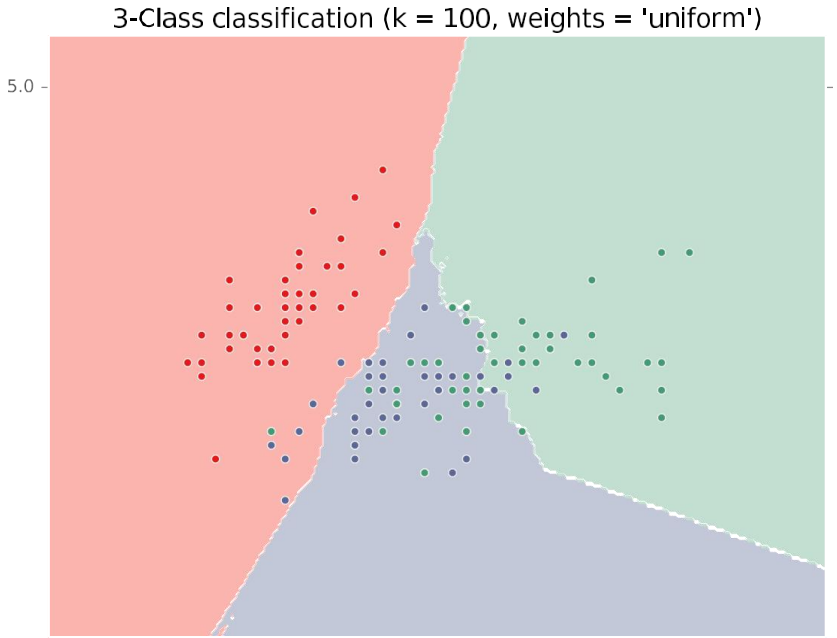


Figure courtesy of Matt Gormley

k=50



k=100



k=120

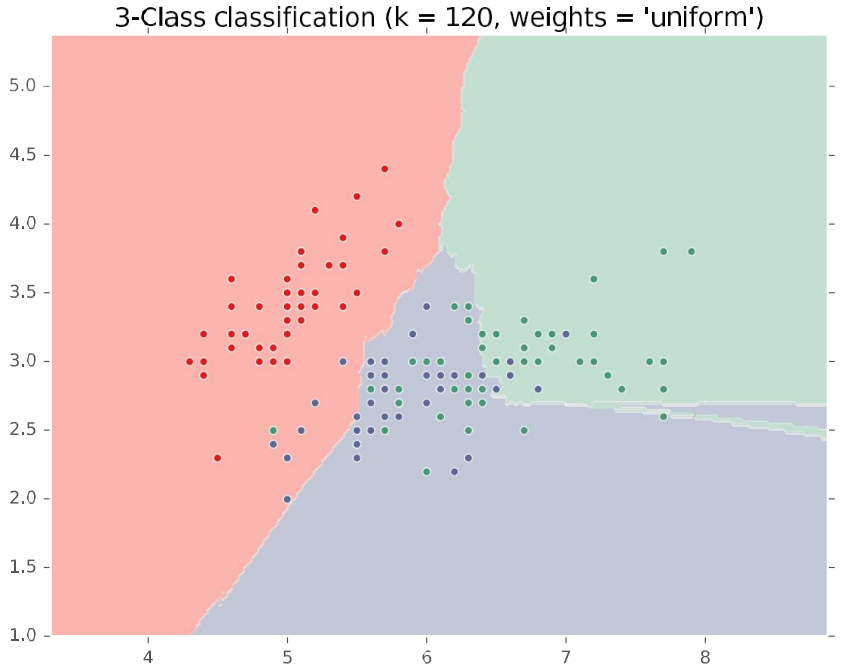


Figure courtesy of Matt Gormley

k=150

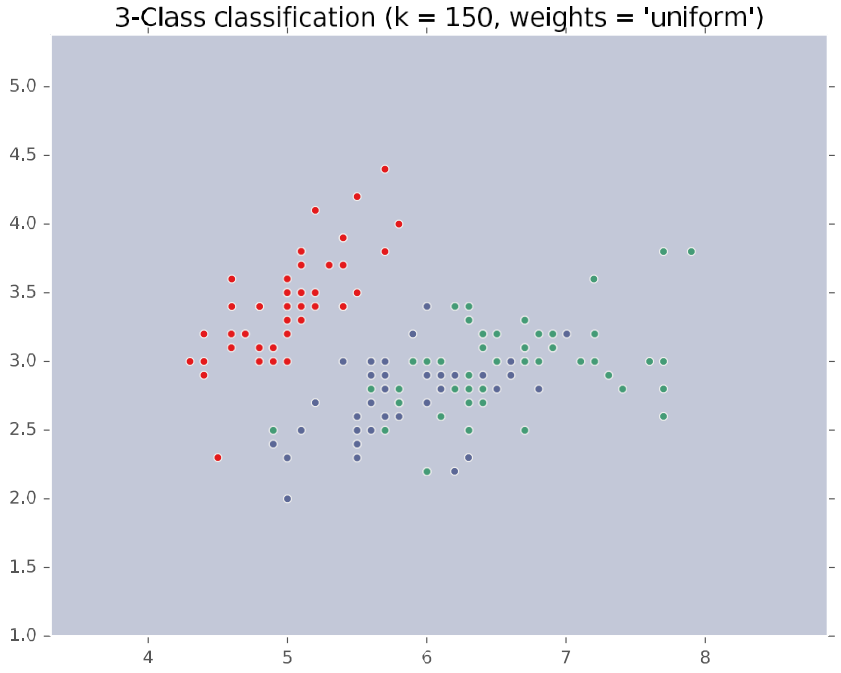
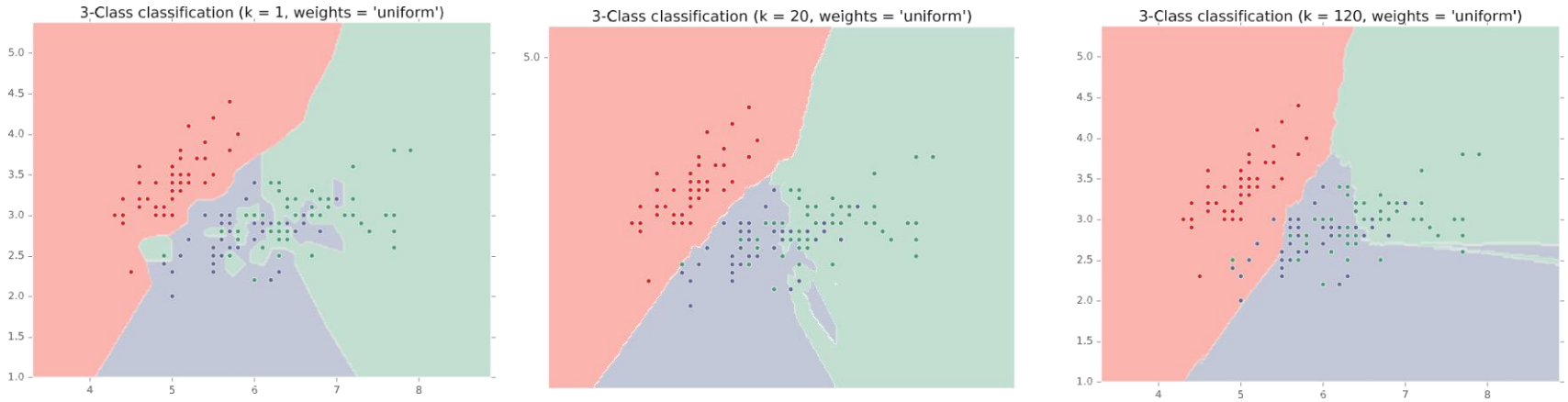


Figure courtesy of Matt Gormley

k=1, 20, 120



Credit to CMU 10601

Figure courtesy of Matt Gormley

Deciding Hyperparameter

What is the best value of **k** to use?

What is the best **distance** to use?

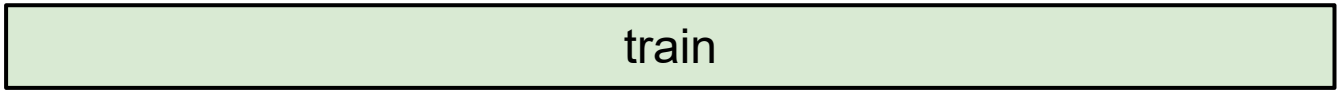
These are **hyperparameters**: choices about the algorithms themselves.

Very problem/dataset-dependent.

Must try them all out and see what works best.

Deciding Hyperparameter

Idea #1: Choose hyperparameters that work best on the **training data**

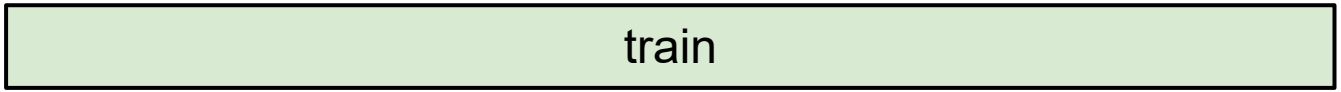


Credit to Stanford CS 229

Deciding Hyperparameter

Idea #1: Choose hyperparameters that work best on the **training data**

BAD: K = 1 always works perfectly on training data

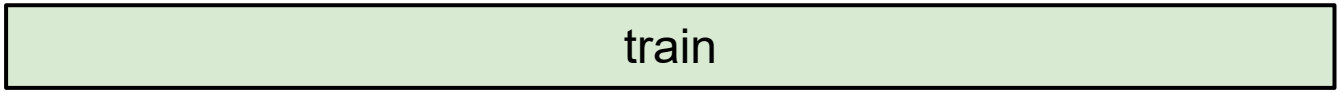


Credit to Stanford CS 229

Deciding Hyperparameter

Idea #1: Choose hyperparameters that work best on the **training data**

BAD: K = 1 always works perfectly on training data



Idea #2: choose hyperparameters that work best on **test** data



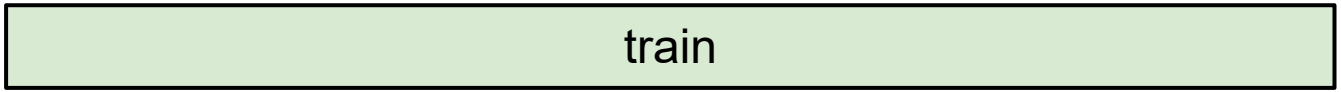
Credit to Stanford CS 229

USC Viterbi School of Engineering

Deciding Hyperparameter

Idea #1: Choose hyperparameters that work best on the **training data**

BAD: K = 1 always works perfectly on training data



Idea #2: choose hyperparameters that work best on **test** data

BAD: No idea how algorithm will perform on new data



Never do this!

Credit to Stanford CS 229

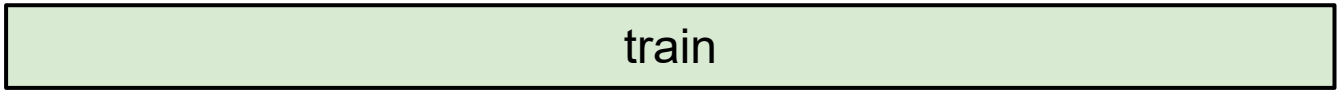
USC Viterbi School of Engineering

Deciding Hyperparameter

Credit to Stanford CS 229

Idea #1: Choose hyperparameters that work best on the **training data**

BAD: K = 1 always works perfectly on training data



Idea #2: choose hyperparameters that work best on **test** data

BAD: No idea how algorithm will perform on new data



Idea #3: Split data into **train**, **val**; choose hyperparameters on val and evaluate on test

Better!



USC Viterbi School of Engineering

Deciding Hyperparameter

train

Idea #4: **Cross-Validation**: Split data into **folds**,
try each fold as validation and average the results

fold 1	fold 2	fold 3	fold 4	fold 5	test
fold 1	fold 2	fold 3	fold 4	fold 5	test
fold 1	fold 2	fold 3	fold 4	fold 5	test

Useful for small datasets, but not used too frequently in deep learning

Credit to Stanford CS 229

USC Viterbi School of Engineering

Example Dataset: **CIFAR10**

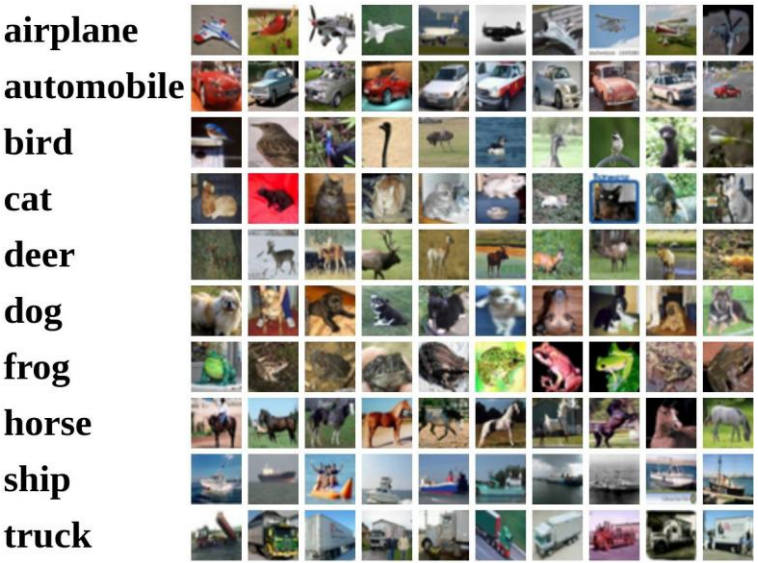
10 classes
50,000 training images
10,000 testing images



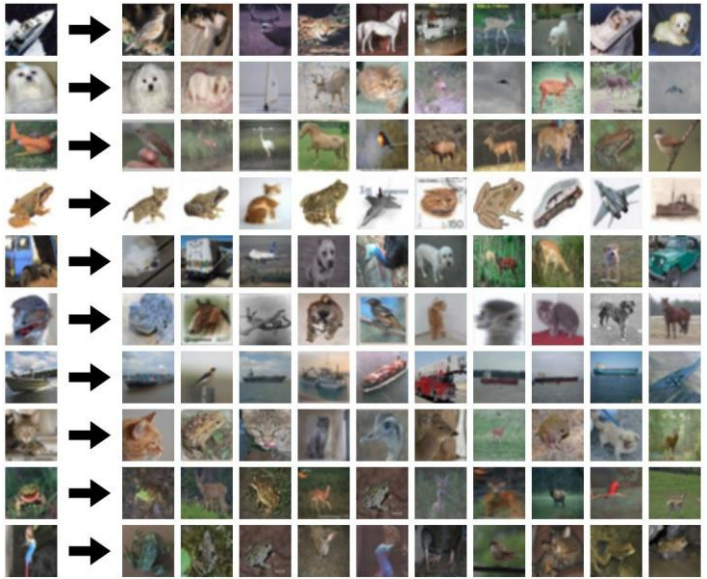
Alex Krizhevsky, “Learning Multiple Layers of Features from Tiny Images”, Technical Report, 2009.

Example Dataset: **CIFAR10**

10 classes
50,000 training images
10,000 testing images



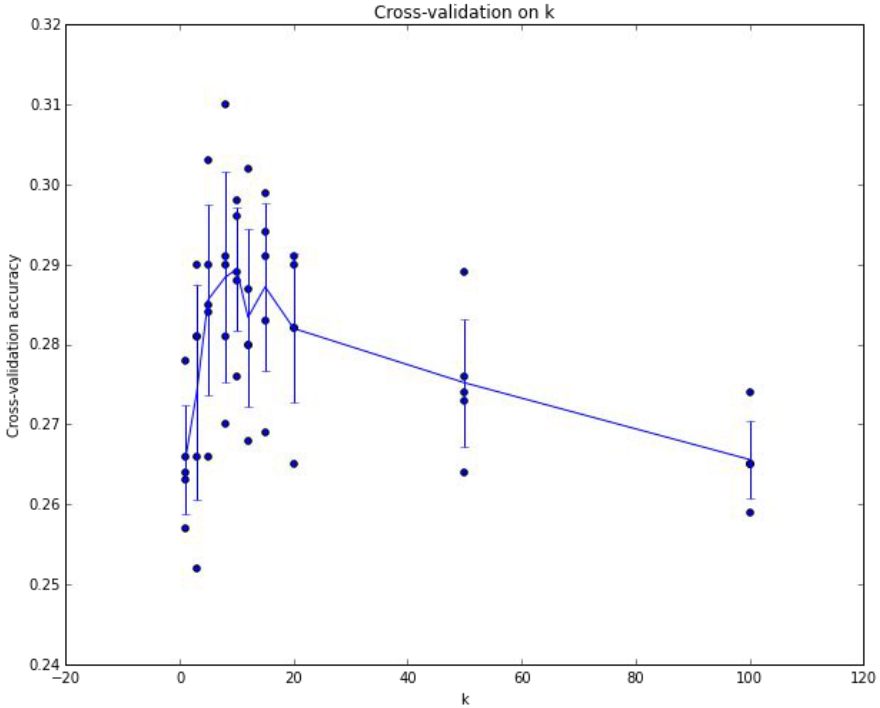
Test images and nearest neighbors



Alex Krizhevsky, "Learning Multiple Layers of Features from Tiny Images", Technical Report, 2009.

USC Viterbi School of Engineering

Deciding Hyperparameter



Example of
5-fold cross-validation
for the value of **k.**

Each point: single
outcome.

The line goes
through the mean, bars
indicated standard
deviation

(Seems that k ~= 7 works best
for this data)

Credit to Stanford CS 229

2026: kNN Can Skip a Model Call

Cache. Store each prompt vector with the answer the model gave.

Look up. For a new prompt, find its nearest cached prompt.

Decide. If the similarity clears a threshold, reuse that answer. Otherwise call the model and store the new pair.

The threshold is the design. A false hit returns a wrong answer; a miss only costs time and money. One global threshold treats easy and ambiguous prompts alike.

2026. vCache learns a threshold per cached prompt under a user-set error bound, reporting up to 12.5x higher cache hit and 26x lower error rates than static-threshold baselines. ICLR 2026.

kNN for Language Models – kNN-LM

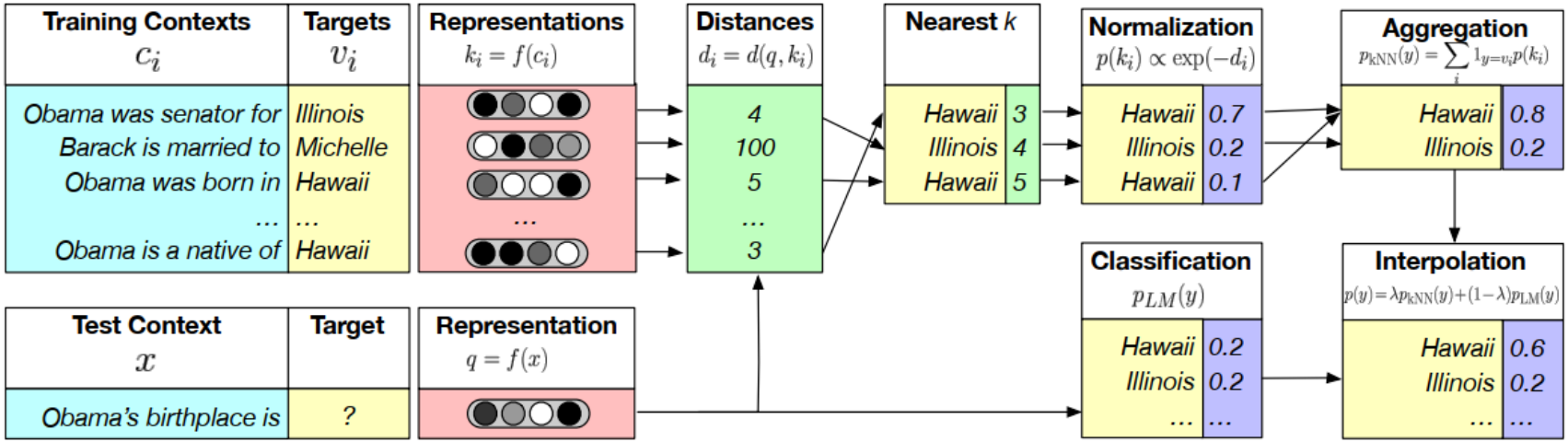


Figure 1: An illustration of kNN-LM. A datastore is constructed with an entry for each training set token, and an encoding of its leftward context. For inference, a test context is encoded, and the k most similar training contexts are retrieved from the datastore, along with the corresponding targets. A distribution over targets is computed based on the distance of the corresponding context from the test context. This distribution is then interpolated with the original model's output distribution.

Khandelwal, Urvashi, et al. "Generalization through Memorization: Nearest Neighbor Language Models." ICLR. 2020.

kNN for Language Models – kNN-LM

Key Idea: kNN-LM enhances language models by retrieving **similar sentences** from the training data (datastore) to aid in predicting the next word. It combines predictions from a neural language model and k-nearest neighbor search for better accuracy.

Khandelwal, Urvashi, et al. "Generalization through Memorization: Nearest Neighbor Language Models." ICLR. 2020.

kNN for Language Models – kNN-LM

How It Works:

1. **Datastore Construction:**
   Store vector representations of training data sequences.
2. **Nearest Neighbor Search:Find k-nearest neighbors for a test input based on vector similarity (e.g., Euclidean distance).**

3. **Prediction Aggregation:**
   Combine next-token probabilities from nearest neighbors with the neural language model's prediction using a weighted interpolation.
4. **Final Decision:**
   A hyperparameter balances contributions from the kNN component and the language model.

Khandelwal, Urvashi, et al. "Generalization through Memorization: Nearest Neighbor Language Models." ICLR. 2020.

Same Geometry, Different Interface

kNN-LM: neighbors vote on the next token. Still alive in specialist work (ACL 2025), but it is not how a general model meets retrieval.

RAG: the nearest passages become readable, citable text in the model's input. That is why a RAG system can show you its sources.

No silver bullet. Across LaRA's eleven models, long context led by about 2.4 points at 32K tokens; retrieval led by 3.68 at 128K.

Production search is approximate. HNSW, ScaNN, and DiskANN trade exact neighbors for speed. What you give up is called recall.

Google states ScaNN uses the same algorithm as Google Search. Microsoft reports one 50-billion-vector index across 1,000+ machines at 26 ms median latency, in Bing.

How Does k Change the Decision?

- What is the effect of k? **control the model capacity/complexity**
- Will a large k make the decision boundary coarse or smooth? **Smoother**
- How will that affect the overfitting and underfitting? **Small k leads to overfitting and large k leads to underfitting**

How do you decide k properly? Also, how to decide these (hyper)parameters of ML models?

- one idea is to monitor the error rate/accuracy

- more systematic approaches are deferred to the later lecture: automated ML, **cross-validation**, Bayesian optimization, etc.,

- $\sqrt{N}$ (the number of total samples)

kNN Summary

- Pros:
  - Intuitive / explainable
  - No training/retraining
  - Provably near-optimal in terms of true error rate
- Cons:
  - Computationally expensive
    - Always needs to store all data: $O(ND)$
    - Finding the k closest points in D dimensions: $O(ND + N\log(k))$
      - Can be sped up through clever use of data structures (trades off training and test costs)
      - Can be approximated using stochastic methods
  - Affected by feature scale Intuitive/explainable

k Nearest Neighbor Summary

What you can do now

- Classify a new point by majority vote over its k neighbors
- Explain what k trades off, and how to choose it
- Name the two things a user must pick: k and the distance
- State where the cost lives: prediction, not training

Classical ML Algorithms

Clustering

What is Clustering?

Discover groups (*clusters*) of related articles



SPORTS

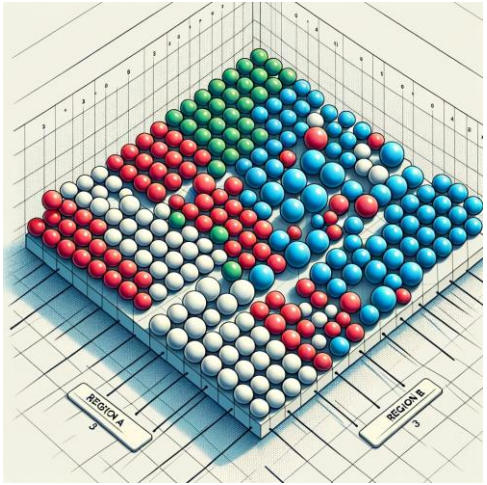


WORLD NEWS

Credit to Stanford CS 229

What is Clustering?

Discover groups (*clusters*) of related items

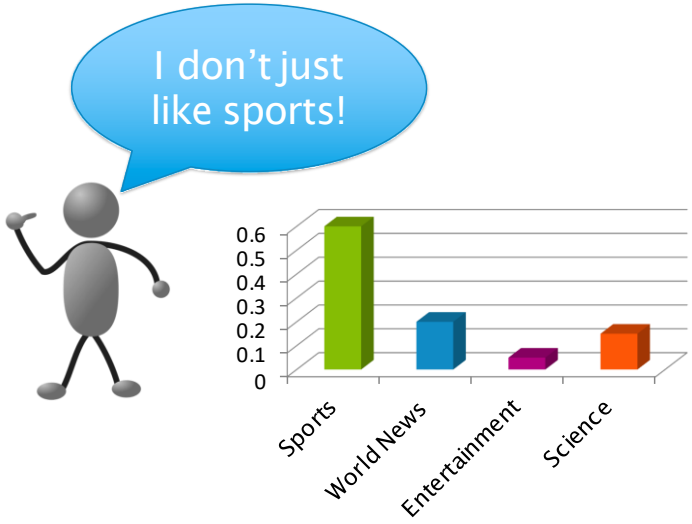




Credit to Stanford CS 229

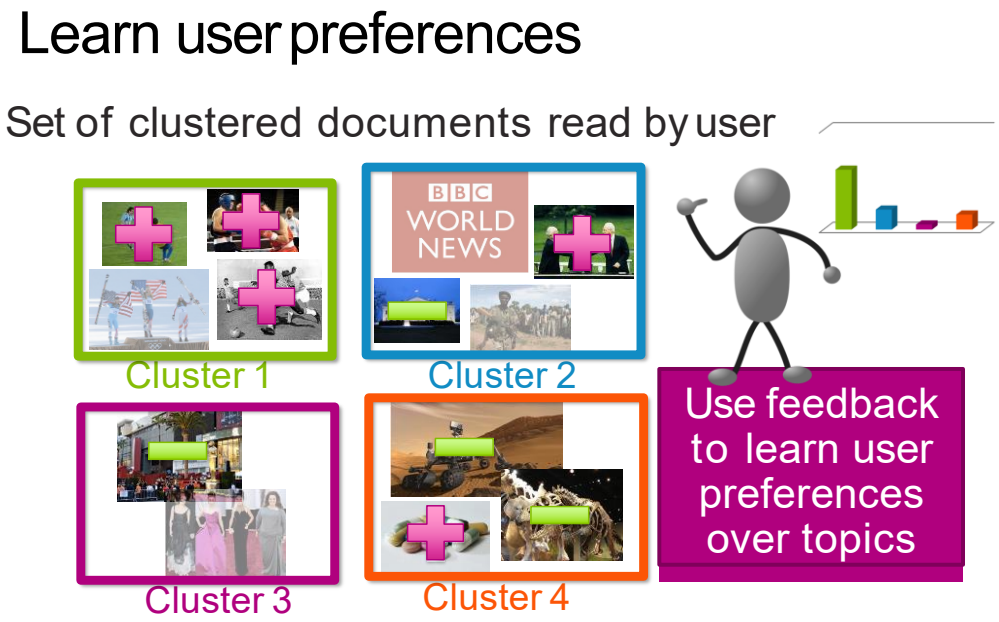
Clustering

Why might clustering be useful?



Credit to Stanford CS 229

Clustering



Credit to Stanford CS 229

Clustering

What if some of the labels are known?

Training set of labeled docs



Clustering: An unsupervised learning task (X only no Y)

Credit to Stanford CS 229

Clustering

Clustering

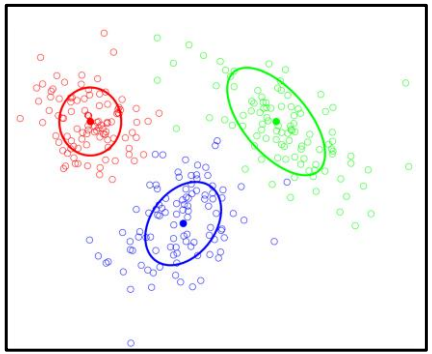
No labels provided

…uncover cluster structure from input alone

Input: docs as vectors x_i

Output: cluster labels z_i

An unsupervised learning task



Credit to Stanford CS 229

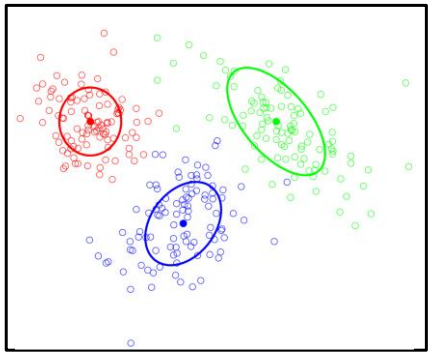
Clustering

What defines a cluster?

Cluster defined by center & shape/spread

Assign observation x_i (doc) to cluster k (topic label) if

- Score under cluster k is lower than under others
- For simplicity, often define score as distance to cluster center (ignoring shape)



Credit to Stanford CS 229

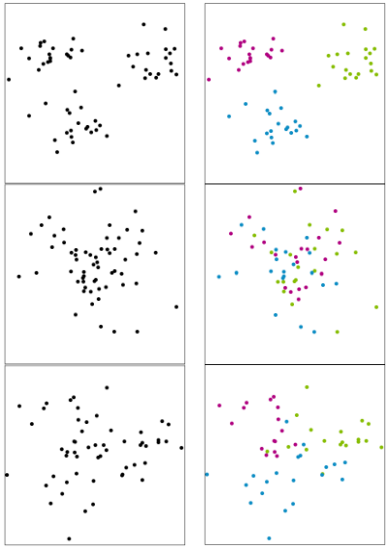
Clustering

Hope for unsupervised learning

Easy

Impossible

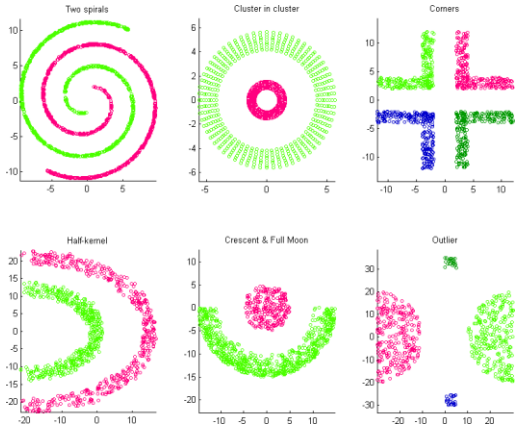
In between



Credit to Stanford CS 229

Clustering

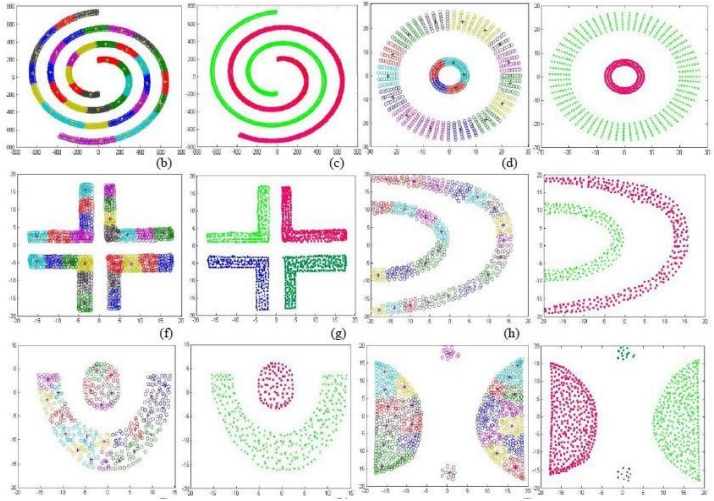
Other (challenging!) clusters to discover…



Credit to Stanford CS 229

Clustering

Other (challenging!) clusters to discover…



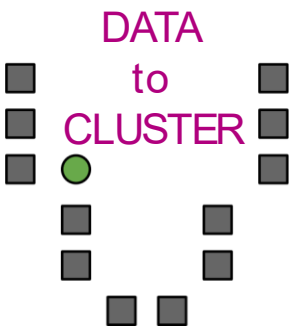
Credit to Stanford CS 229

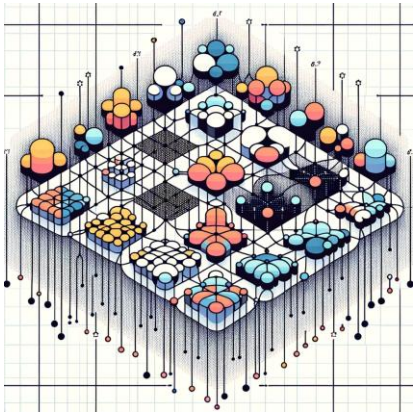
Clustering: k-means

k-means

Assume

- Score= distance to cluster center (smaller better)



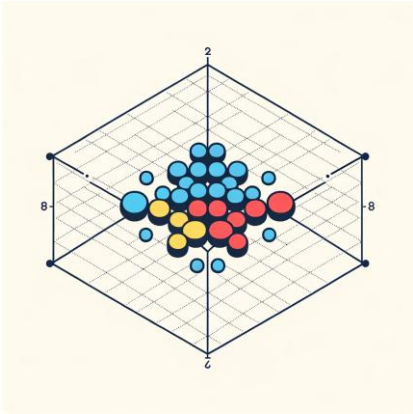


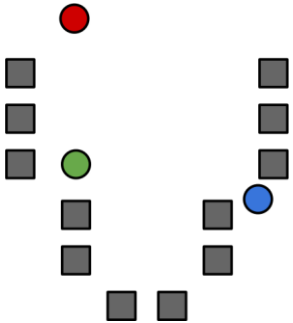
Credit to Stanford CS 229

Clustering: k-means

k-means algorithm

0. Initialize cluster centers
   $\mu_1, \mu_2, \ldots, \mu_k$





Credit to Stanford CS 229

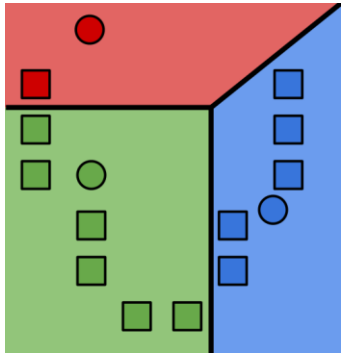
Clustering: k-means

k-means algorithm

1. Initialize cluster centers
2. Assign observations to closest cluster center

$$z_i \leftarrow \arg\min_j ||\mu_j - \mathbf{x}_i||_2^2$$

Inferred label for obs i, whereas supervised learning has given label y_i



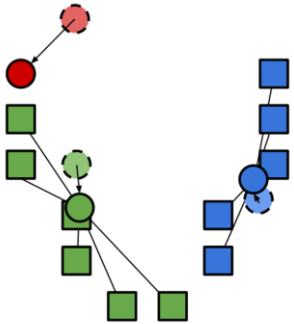
Credit to Stanford CS 229

Clustering: k-means

k-means algorithm

1. Initialize cluster centers
2. Assign observations to closest cluster center
3. Revise cluster centers as mean of assigned observations

$$\mu_j = \frac{1}{n_j} \sum_{i:z_i=j} \mathbf{x}_i$$

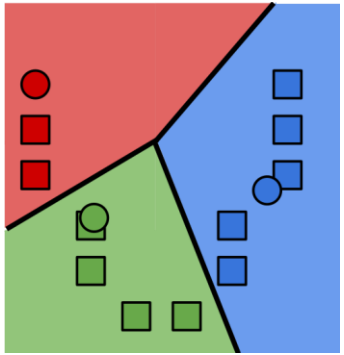


Credit to Stanford CS 229

Clustering: k-means

k-means algorithm

1. Initialize cluster centers
2. Assign observations to closest cluster center
3. Revise cluster centers as mean of assigned observations
4. Repeat 1.+2. until convergence



Credit to Stanford CS 229

Clustering: k-means

Limitations of k-means

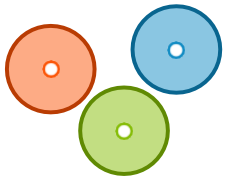
Assign observations to closest cluster center

$$z_i \leftarrow \arg\min_j \|\mu_j - \mathbf{x}_i\|_2^2$$

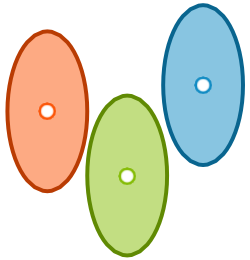
Only center matters

Can use weighted Euclidean, but requires *known* weights

Equivalent to assuming spherically symmetric clusters



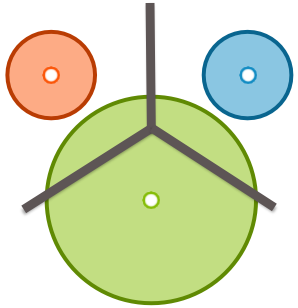
Still assumes all clusters have the same axis-aligned ellipses



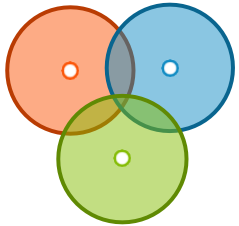
Credit to Stanford CS 229

Clustering: k-means

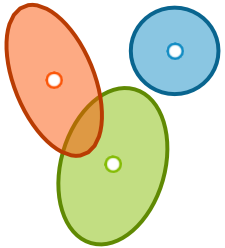
Failure modes of k-means



disparate cluster sizes



overlapping clusters



different shaped/oriented clusters

Credit to Stanford CS 229

Clustering: k-means

What you can do now…

- Describe the input (unlabeled observations) and output (labels) of a clustering algorithm
- Determine whether a task is supervised or unsupervised
- Cluster documents using k-means
- Describe potential applications of clustering

Credit to Stanford CS 229

K-means at Pretraining Scale

CuraWeb, June 2026. A pretraining corpus built from 184.9 billion raw web documents.

Two deduplication stages. MinHash cuts 23.29B documents to 7.97B on surface overlap; semantic deduplication then cuts that to 6.42B, another 19.4%.

k-means carries both halves. The semantic stage runs cluster by cluster on k-means output; about 49,000 clusters later score diversity for sampling.

The cost is real. One assignment pass is 6.42B points against 49K centroids, roughly 3 x 10^14 distance computations before the dimension.

Status: arXiv v1, not peer reviewed. The 1.82-point gain is the whole pipeline, not a k-means ablation; an ICLR 2026 paper reports semantic deduplication alone scoring below the raw corpus.

LLMs for Clustering

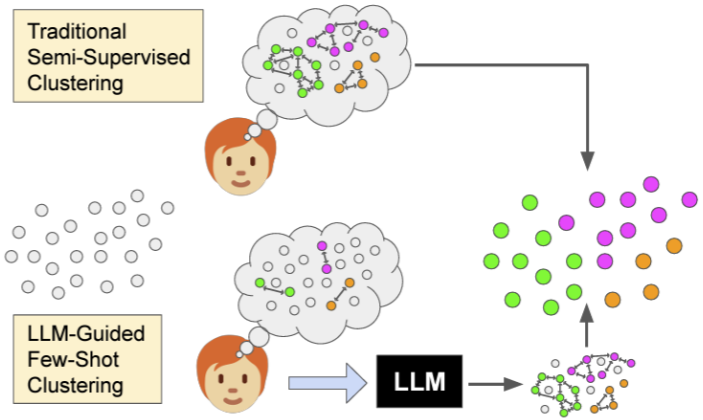


Figure 1: In traditional semi-supervised clustering, a user provides a large amount of feedback to the clusterer. In our approach, the user prompts an LLM with a small amount of feedback. The LLM then generates a large amount of pseudo-feedback for the clusterer.

🔍 **Unsupervised Clustering** → Groups data automatically but **may not match user expectations**.

👩‍🏫 **Semi-Supervised Clustering** → Users provide **guidance** (constraints, seed points) but **requires extensive human feedback**.

⚠️ **The Problem:** Traditional methods either **lack intelligence** or **demand too much expert labeling**.

Viswanathan, V., Gashteovski, K., Lawrence, C., Wu, T., & Neubig, G. (2024). Large language models enable few-shot clustering. TACL, 12, 321-333.

LLMs for Clustering

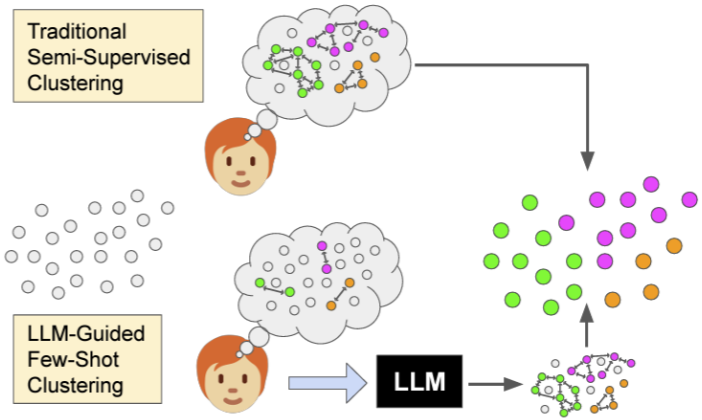


Figure 1: In traditional semi-supervised clustering, a user provides a large amount of feedback to the clusterer. In our approach, the user prompts an LLM with a small amount of feedback. The LLM then generates a large amount of pseudo-feedback for the clusterer.

Traditional vs. LLM-Guided Few-Shot Clustering

- **Traditional Semi-Supervised Clustering:** Requires extensive user feedback to guide the clustering process, making it resource-intensive and time-consuming.
- **LLM-Guided Few-Shot Clustering:** Leverages Large Language Models (LLMs) to amplify minimal user input, generating substantial pseudo-feedback to guide clustering with fewer user interventions.

Viswanathan, V., Gashteovski, K., Lawrence, C., Wu, T., & Neubig, G. (2024). Large language models enable few-shot clustering. TACL, 12, 321-333.

The New Idea: Use LLMs to Make Clustering Smarter

This paper explores using **LLMs at three stages** of the clustering process:
1️⃣ **Before clustering** → LLMs generate key features to **enrich text representation**, improving clustering accuracy.
2️⃣ **During clustering** → LLMs act as a **"pseudo-expert"** to provide **similarity constraints**, guiding the clustering process.
3️⃣ **After clustering** → LLMs **correct errors** in low-confidence cases to **refine the final clustering results**.

💡 **Core Idea:** LLMs simulate **expert feedback**, reducing user workload while improving clustering effectiveness.

Viswanathan, V., Gashteovski, K., Lawrence, C., Wu, T., & Neubig, G. (2024). Large language models enable few-shot clustering. TACL, 12, 321-333.

The New Idea: Use LLMs to Make Clustering Smarter

✅ **Method 1: Using LLMs to Expand Text Representations** 📝✨

- Traditional clustering relies on **word embeddings**, which **may miss key information**.
  LLMs first generate keyphrases (important descriptive words) and add them to the text representation.
  This **helps the clustering algorithm better capture semantic meaning** and group similar texts together.

📊 Example:

> Original text: "How do I locate my credit card?" 🏦💳
> LLM-generated keyphrases: `["card status", "card location"]` 🔑
> Final input vector = Original text vector + Keyphrase vector

Viswanathan, V., Gashteovski, K., Lawrence, C., Wu, T., & Neubig, G. (2024). Large language models enable few-shot clustering. TACL, 12, 321-333.

The New Idea: Use LLMs to Make Clustering Smarter

✅ **Method 2: Using LLMs as a "Pseudo-Expert" to Guide Clustering** 🤖🔍
🔗 Traditional **semi-supervised clustering** requires humans to provide **"must-link" and "cannot-link" constraints**.

👩‍🏫 **Instead of human labeling, the LLM acts as a pseudo-oracle** and generates these constraints based on a few examples.

⚡ **Effect: Reduces the need for manual annotation while improving clustering supervision.**

Viswanathan, V., Gashteovski, K., Lawrence, C., Wu, T., & Neubig, G. (2024). Large language models enable few-shot clustering. TACL, 12, 321-333.

The New Idea: Use LLMs to Make Clustering Smarter

✅ **Method 3: Using LLMs to Correct Clustering Mistakes** 🎯✅

🔄 **Step 1:** Perform **regular clustering** (e.g., K-Means).
🧐 **Step 2:** Identify **low-confidence points** that may belong to multiple clusters.
⚖️ **Step 3:** Use the LLM to **re-evaluate and reassign these points** to the correct cluster.

🔥 **Effect: Has limited impact in the paper's experiments; the gain is smaller than keyphrase generation or pairwise constraints.**

Viswanathan, V., Gashteovski, K., Lawrence, C., Wu, T., & Neubig, G. (2024). Large language models enable few-shot clustering. TACL, 12, 321-333.

LLMs for Clustering

Key Steps:

1.Minimal User Input:

1. The user provides a small amount of feedback (e.g., constraints, examples, or task-specific instructions).

2.LLM Augmentation:

1. A Large Language Model (LLM) processes the user input and generates pseudo-feedback or enriched representations for the data.

3.Data Representation Enhancement:

1. Keyphrases or additional features are generated by the LLM and concatenated with original document embeddings to improve clustering quality.

4.Clustering with Pseudo-Feedback:

1. The clustering algorithm (e.g., K-Means) incorporates the LLM-generated feedback (e.g., pairwise constraints) to refine clusters.

5.Post-Clustering Correction (Optional):

1. Low-confidence cluster assignments are identified and re-evaluated using the LLM as a pseudo-oracle.

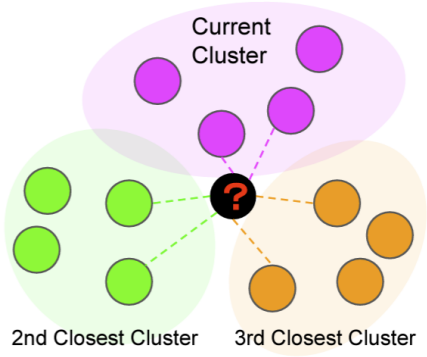


Figure 3: After performing clustering, we identify low-confidence points. For these points, we ask an LLM whether the current cluster assignment is correct. If the LLM responds negatively, we ask the LLM whether this point should instead be linked to any of the top-5 nearest clusters, and correct the clustering accordingly.

Viswanathan, V., Gashteovski, K., Lawrence, C., Wu, T., & Neubig, G. (2024). Large language models enable few-shot clustering. TACL, 12, 321-333.

LLMs for Clustering

	Dataset / Method	OPIEC59k				ReVerb45k			
		Macro F1	**Micro F1**	**Pair F1**	***Avg***	**Macro F1**	**Micro F1**	**Pair F1**	***Avg***
	Optimal Clust.	80.3	97.0	95.5	90.9	84.8	93.5	92.1	90.1
	CMVC	52.8	90.7	84.7	76.1	66.1	87.9	89.4	81.1
	KMeans	53.5 ±0.0	91.0 ±0.0	85.6 ±0.0	76.7	69.6 ±0.0	89.1 ±0.0	89.3 ±0.0	82.7
ours	PCKMeans	58.7 ±0.0	91.5 ±0.0	86.1 ±0.0	78.7	72.0 ±0.0	88.5 ±0.0	87.0 ±0.0	82.5
ours	LLM Correction	58.7	91.5	85.2	78.4	69.9	89.2	88.4	82.5
ours	Keyphrase Clust.	**60.3** ±0.0	**92.5** ±0.0	**87.3** ±0.0	**80.0**	**72.3** ±0.0	**90.2** ±0.0	**90.0** ±0.0	**84.2**

Table 1: Comparing methods for integrating LLMs into entity canonicalization. "CMVC" refers to the multi-view clustering method of Shen et al. (2022), while "KMeans" refers to our simplified reimplementation of the same method. Where applicable, standard deviations are obtained by running clustering 5 times with different seeds.

	Dataset / Method	Bank77		CLINC		Tweet	
		Acc	**NMI**	**Acc**	**NMI**	**Acc**	**NMI**
	SCCL	–	–	–	–	78.2	89.2
	ClusterLLM	71.2	–	83.8	–	–	–
	KMeans	64.0 ±0.0	81.7 ±0.0	77.7 ±0.0	91.5 ±0.0	57.5 ±0.0	80.6 ±0.0
ours	PCKMeans	59.6 ±0.0	79.6 ±0.0	**79.6** ±0.0	92.1 ±0.0	**65.3** ±0.0	**85.1** ±0.0
ours	LLM Correction	64.1	81.9	77.8	91.3	59.0	81.5
ours	Keyphrase Clustering	**65.3** ±0.0	**82.4** ±0.0	79.4 ±0.0	**92.6** ±0.0	62.0 ±0.0	83.8 ±0.0

Table 2: Comparing methods for integrating LLMs into text clustering. "SCCL" refers to Zhang et al. (2021) while "ClusterLLM" refers to Zhang et al. (2023). We use the same base encoders as those methods in our experiments. Where applicable, standard deviations are obtained by running clustering 5 times with different seeds.

✔ **Using LLMs to generate keyphrases is the most effective approach**, improving clustering quality across all datasets.

✔ **Using LLMs as a pseudo-expert also works well**, especially when LLMs can make strong similarity judgments.

✔ **LLM-based post-clustering correction has limited impact**, but it still helps refine results.

Viswanathan, V., Gashteovski, K., Lawrence, C., Wu, T., & Neubig, G. (2024). Large language models enable few-shot clustering. TACL, 12, 321-333.

Clustering Summary

What you can do now

- Separate clustering from classification by what is given
- Run k-means by hand: assign, revise, repeat
- Say what k-means assumes about cluster shape
- Name a case where that assumption fails

Week 2 Is Inside the Agent Loop

An agent is a loop: pick a model, expose a tool, call it, check the result, repeat. That control plane is usually not another large model.

Routing is cost-sensitive classification. A false negative costs quality, a false positive costs money, and the threshold is the design.

The router is this week's algorithms. CARROT routes with kNN classification and kNN regression over cost; a 2026 study picked XGBoost.

Tool selection is top-k retrieval. On ToolRet's 43,000 tools, every retriever scored below 35% Completeness@10.

A guardrail is a thresholded classifier: PromptGuard 2 86M reports 97.5% recall at a 1% false positive rate. Classification, neighbors, trees, and thresholds decide which model runs and what it may do.

The Next Few Lectures

***TAs or myself will cover some basic Python/PyTorch and DL libraries**

We will move to the neural networks. But these basics are important:

- You know the linear models as the components of DL
- You know how to frame an ML problem
- You know loss and optimization – and know overfitting and underfitting
- You know multiple classical ML topics – they can come in handy
- You know why non-linear models are important

Let us move to the neural networks – non-linear, powerful ML models